Universidade Federal do Piauí
Centro de Educação Aberta e a Distância

CONSTRUÇÃO DE SISTEMAS DE GERENCIAMENTO DE BANCO DE DADOS

Flávio Rubens de Carvalho Sousa
José Maria da Silva Monteiro Filho



Ministério da Educação - MEC
Universidade Aberta do Brasil - UAB
Universidade Federal do Piauí - UFPI
Universidade Aberta do Piauí - UAPI
Centro de Educação Aberta e a Distância - CEAD

CONSTRUÇÃO DE SISTEMAS DE GERENCIAMENTO DE BANCO DE DADOS

Flávio Rubens de Carvalho Sousa
José Maria da Silva Monteiro Filho

UFPI
2013

PRESIDENTE DA REPÚBLICA *Dilma Vana Rousseff Linhares*
MINISTRO DA EDUCAÇÃO *Aloizio Mercadante*
GOVERNADOR DO ESTADO *Wilson Nunes Martins*
REITOR DA UNIVERSIDADE FEDERAL DO PIAUÍ *José Arimatéia Dantas Lopes*
PRESIDENTE DA CAPES *Jorge Almeida Guimarães*
COORDENADOR GERAL DA UNIVERSIDADE ABERTA DO BRASIL *João Carlos Teatini de S. Clímaco*
DIRETOR DO CENTRO DE EDUCAÇÃO ABERTA E A DISTÂNCIA DA UFPI *Gildásio Guedes Fernandes*

COORDENADORES DE CURSOS

ADMINISTRAÇÃO *Antonella Maria das Chagas Sousa*
ADMINISTRAÇÃO PÚBLICA *Fabiana Rodrigues de Almeida Castro*
CIÊNCIAS BIOLÓGICAS *Maria da Conceição Prado de Oliveira*
FILOSOFIA *Zoraida Maria Lopes Feitosa*
FÍSICA *Miguel Arcanjo Costa*
LETRAS PORTUGUÊS *José Vanderlei Carneiro*
LETRAS INGLÊS *Lívia Fernanda Nery da Silva*
MATEMÁTICA *José Ribamar Lopes Batista*
PEDAGOGIA *Vera Lúcia Costa Oliveira*
QUÍMICA *Milton Batista da Silva*
SISTEMAS DE INFORMAÇÃO *Leonardo Ramon Nunes de Sousa*

EQUIPE DE DESENVOLVIMENTO

TÉCNICOS EM ASSUNTOS EDUCACIONAIS *Djane Oliveira de Brito*
Ubirajara Santana Assunção
Zilda Vieira Chaves
EDIÇÃO *Roberto Denes Quaresma Rêgo*
PROJETO GRÁFICO *Samuel Falcão Silva*
DIAGRAMAÇÃO *Antonio F. de Carvalho Filho*
REVISÃO ORTOGRÁFICA *Georgea Vale de Queiroz Siqueira*
REVISÃO GRÁFICA *Georgea Vale de Queiroz Siqueira*

CONSELHO EDITORIAL DA EDUFPI

Prof. Dr. Ricardo Alaggio Ribeiro (Presidente)
Des. Tomaz Gomes Campelo
Prof. Dr. José Renato de Araújo Sousa
Profª. Drª. Teresinha de Jesus Mesquita Queiroz
Profª. Francisca Maria Soares Mendes
Profª. Iracildes Maria de Moura Fé Lima
Prof. Dr. João Renór Ferreira de Carvalho

S725c Sousa, Flávio Rubens de Carvalho.
Construção de sistemas de gerenciamento de banco de dados / Flávio Rubens de Carvalho Sousa, José Maria da Silva Monteiro Filho. - Teresina: EDUFPI/CEAD, 2013.
162 p.

ISBN:

1. Banco de Dados - Gerência. 2. Armazenamento de Dados. 3. Recuperação de Banco de Dados. 4. Educação a Distância. I. Monteiro Filho, José Maria da Silva. II. Título.

CDD: 005.74

© 2013. Universidade Federal do Piauí - UFPI. Todos os direitos reservados.

A responsabilidade pelo texto e imagens desta obra é do autor. O conteúdo desta obra foi licenciado, temporária e gratuitamente, para utilização no âmbito do Sistema Universidade Aberta do Brasil, através da UFPI. O leitor se compromete a utilizar o conteúdo desta obra para aprendizado pessoal, sendo que a reprodução e distribuição ficarão limitadas ao âmbito interno dos cursos. A citação desta obra em trabalhos acadêmicos e/ou profissionais poderá ser feita, com indicação da fonte. A cópia desta obra sem autorização expressa, ou com intuito de lucro, constitui crime contra a propriedade intelectual, com sanções previstas no Código Penal.
É proibida a venda deste material.

Apresentação

Um Sistema de Gerenciamento de Banco de Dados, ou SGBD, é um *software* projetado para auxiliar o armazenamento, a recuperação e a utilização de vastos conjuntos de dados. O gerenciamento dessas informações envolve definir as estruturas que serão utilizadas para o armazenamento dos dados e os mecanismos que serão responsáveis pela manipulação desses dados. Como as informações precisam ser compartilhadas entre vários usuários, o SGBD necessita evitar possíveis resultados anômalos. Além disso, o SGBD necessita garantir a segurança das informações armazenadas, apesar das falhas que por ventura possam ocorrer, ou das tentativas de acesso não autorizado.

Os SGBDs são, atualmente, ferramenta indispensável para o armazenamento e o gerenciamento de informações. A disciplina que aborda as técnicas, os algoritmos e a teoria utilizada na construção de sistemas de gerenciamento de banco de dados tornou-se parte integrante do currículo dos cursos de Sistemas de Informação, Ciência da Computação e Engenharia da Computação.

O objetivo deste livro é proporcionar ao leitor entendimento da Construção de Sistemas de Gerenciamento de Banco de Dados. O texto foi escrito de forma simples e objetiva. Cada capítulo é acompanhado de embasamento teórico e prático, bem como de implementações e de exercícios. A bibliografia e a webliografia ao fim das notas são mais do que suficientes para que o leitor se aprofunde na teoria apresentada em cada unidade. Assim, esperamos prover ao leitor um sentido de investigação nesta disciplina rica e vibrante.

Na **Unidade I** são discutidos os principais aspectos relacionados ao projeto físico de sistemas de bancos de dados, envolvendo principalmente o armazenamento de dados, o gerenciamento de *buffer* e a indexação de dados.

Na **Unidade II** é apresentado o problema do processamento de transações em bancos de dados e a teoria relacionada a esta importante questão.

Na **Unidade III** se discute o problema do controle de concorrência em bancos de dados e as principais técnicas utilizadas para solucioná-lo.

Por fim, na **Unidade IV** são apresentados conceitos sobre a recuperação de banco de dados, destacando as principais técnicas utilizadas.

Introdução

Atualmente, os Sistemas de Gerenciamento de Bancos de Dados (SGBDs) desempenham um papel de fundamental importância em praticamente todas as atividades econômicas. Nesse sentido, é difícil encontrar uma atividade econômica que não utilize um SGBD. Assim, os SGBDs são usados nas mais diferentes aplicações comerciais, tais como nos Sistemas de Gestão de Relacionamento com o Cliente (Customer Relationship Management – CRM), nos Sistemas de Gestão Empresarial (Enterprise Resource Planning- ERP), nos Sistemas de Inteligência do Negócio (Business Intelligence – BI), na Mineração de Dados (Data Mining), dentre outras. Além disso, os SGBDs podem ser encontrados em diversas plataformas de hardware, desde servidores, computadores pessoais, equipamentos portáteis, tablets, dispositivos celulares, e, mais recentemente, na nuvem (ou seja, nos ambientes de Computação em Nuvem – Cloud Computing). Assim, os SGBDs se tornaram fundamentais para a tomada de decisões estratégicas e, consequentemente, para a própria sobrevivência das corporações. Além disso, os SGBDs estão presentes nas aplicações de diversas áreas, como por exemplo, a medicina, as engenharias, os transportes, a biologia etc. Esses sistemas também são utilizados pelo Governo para fornecer suporte às implementações das políticas públicas, fazendo parte de diferentes aplicações, tais como: cadastro único de pacientes, Bolsa Família, fila para transplantes, cadastro de doadores de órgãos, declaração de imposto de renda etc.

Para ter ideia da importância dos SGBDs, imagine as consequências caso uma empresa administradora de cartões de crédito perdesse os dados armazenados sobre seus clientes e as operações por eles realizadas, ou um grande banco ou instituição financeira tivesse as informações de seus

clientes corrompidas, ou a Receita Federal do Brasil perdesse os dados dos contribuintes, ou ainda se a relação de famílias cadastradas no programa Bolsa Família fosse perdida. Quais seriam as consequências?

Adicionalmente, os SGBDs são encontrados no núcleo de diversas pesquisas científicas. Eles armazenam os dados coletados por astrônomos (fotos de satélites, por exemplo), geólogos (movimentações sísmicas ou localizações de reservas petrolíferas, por exemplo), biólogos que buscam decifrar o genoma humano ou pesquisam doenças genéticas, bioquímicos que procuram descobrir novos medicamentos, dentre outros.

O sucesso da utilização dos Sistemas de Bancos de Dados advém de um corpo de conhecimento e de tecnologia que se desenvolveu paulatinamente ao longo de várias décadas. Em um Sistema de Banco de Dados, o conjunto dos dados armazenados (denominado Banco de Dados) é gerenciado por um software especializado chamado de Sistema de Gerenciamento de Banco de dados (SGBD). Um SGBD é uma coleção de programas que permite ao usuário definir, construir e manipular Bases de Dados para as mais diversas finalidades. Um SGBD é uma ferramenta poderosa, concebida para armazenar e gerenciar grandes quantidades de dados de forma eficiente e segura.

Os SGBDs estão entre os sistemas de software mais complexos da atualidade. Os principais recursos proporcionados por um SGBD são: armazenamento persistente de grandes volumes de dados, recuperação eficiente dos dados armazenados, interface para execução de consultas, interface para acesso via programas aplicativos, recuperação automática após a ocorrência falhas, gerenciamento do acesso concorrente dos vários usuários, mecanismos para autenticação e autorização, dentre outros.

Dessa forma, compreender como os SGBDs são construídos e como funcionam internamente se torna de fundamental importância para: construir novos SGBDs, adaptar os SGBDs existentes a necessidades específicas, obter o máximo desempenho durante a execução de suas tarefas, realizar ajustes de desempenho (tuning) etc.

Este livro tem por objetivo proporcionar ao leitor os conhecimentos necessários para compreender como os SGBDs são construídos, como executam suas tarefas internamente e como podem ser ajustados para melhorar seu desempenho. Ao longo dos capítulos iremos estudar os módulos que compõem um SGBD, suas funcionalidades, como são implementados e como podem ser ajustados. Esperamos que esta leitura forneça os subsídios necessários para iniciar a formação de novos profissionais (Administradores

de Bancos de Dados, Administradores de Dados etc) e despertar o interesse na fascinante e dinâmica área de Banco de Dados.

Boa Leitura!!
Flávio Rubens de Carvalho Sousa
José Maria da Silva Monteiro Filho

Sumário

09 **UNIDADE 1**
PROJETO FÍSICO DE BANCOS DE DADOS

1 SISTEMAS DE GERENCIAMENTO DE BANCOS DE DADOS
- 1.1 Introdução 15
- 1.2 Arquitetura 16
- 1.3 Classificação dos Sistemas de Banco de Dados 19
- Exercícios 20

2 ARMAZENAMENTO DE DADOS
- 2.1 Introdução 21
- 2.2 Discos Magnéticos 27
- 2.3 Técnicas de RAID 31
- Exercícios 34

3 GERENCIAMENTO DE BUFFER
- 3.1 Sistema de Memória Virtual 38
- 3.2 O Gerenciador de Buffer 40
- 3.3 Mecanismo de Paginação de SGBDs 41
- 3.4 Políticas de Alocação de Páginas 42
- Exercícios 48

41 **UNIDADE 2**
PROCESSAMENTO DE TRANSAÇÕES

1 INTRODUÇÃO
- 1.1 O Problema da Concorrência em BDs 51
- 1.2 Conceito de Transação 53
- 1.3 Estados de uma transação 55
- 1.4. Propriedades da transação 56
- Exercícios 57

2 EXECUÇÕES CONCORRENTES
- 2.1 Acessos Concorrentes 57

2.2 Controle de Concorrência 58
Exercícios 61
3 ISOLAMENTO DE TRANSAÇÕES
3.1 Transações Sequenciais 62
3.2 Execução Correta de Transações Concorrentes 63
3.3 Serializabilidade 66
3.4 Grafo de serialização de um Schedule 68
3.5 Prefixo de um Schedule S 69
Exercícios 70
4 CONFIABILIDADE DE SCHEDULES
4.1 Corretude x Confiabilidade 72
4.2 Schedule Recuperável 73
4.3 Schedule que Evita Abort em Cascata 73
4.4 Schedule Preciso (Strict) 74
Exercícios 75

69

UNIDADE 3
CONTROLE DE CONCORRÊNCIA

1 INTRODUÇÃO
1.1 Conceitos iniciais 79
1.2 Classificação dos Protocolos de Concorrência 79
Exercícios 80
2. DESCRIÇÃO DOS PROTOCOLOS CONSERVADORES
2.1 Protocolos Baseados em Bloqueios 81
2.2 Tratamento de impasse 95
2.3 Granularidade de Bloqueios 100
2.4 Bloqueio de duas fases com múltiplas versões 105
2.5 Arquitetura de GTs que utilizam 2PL 109
2.6 Níveis de isolamento 110
Exercícios 117
3. DESCRIÇÃO DOS PROTOCOLOS AGRESSIVOS
3.1 Protocolos baseados em Marcadores de Tempo (timestamp) 126
3.2 Teste do grafo de serialização 128
3.3 Protocolos otimistas 131
Exercícios 132

41

UNIDADE 4

RECUPERAÇÃO DE BANCO DE DADOS

1 RECUPERAÇÃO DE BANCO DE DADOS
- 1.1 Introdução 137
- 1.2 Tipos de Falhas 138
- 1.3 Recuperação baseada em log 139
- 1.4 Gerenciamento de Buffer 141
- 1.5 Checkpoint 143
- 1.6 Técnicas de Recuperação 145
- 1.7 Técnica Recuperação de Meio de Armazenamento... 153
- 1.8 Recuperação baseada em Páginas Sombras 155
- Exercícios 156

WEBLIOGRAFIA 158
REFERÊNCIAS 159

UNIDADE 1

PROJETO FÍSICO DE BANCOS DE DADOS

Resumindo

Esta unidade discute, em detalhes, os principais aspectos relacionados ao projeto físico de sistemas de bancos de dados. Inicialmente, define-se o que é um Sistema de Gerenciamento de Banco de Dados (SGBD). Em seguida, discute-se a arquitetura para um SGBD e uma classificação com as diferentes arquiteturas é apresentada. Posteriormente, examinamos os conceitos, as técnicas e as estratégias relacionadas ao armazenamento de dados, ao gerenciamento de buffer e à indexação de dados.

1 PROJETO FÍSICO DE BANCOS DE DADOS

1. SISTEMAS DE BANCOS DE DADOS

A finalidade de um sistema de bancos de dados é simplificar e facilitar o acesso aos dados armazenados. Assim, os usuários do sistema não devem se preocupar desnecessariamente com os detalhes físicos da implementação do sistema, tampouco com detalhes acerca do armazenamento dos dados. Contudo, o principal fator na satisfação de um usuário com o sistema de bancos de dados é seu desempenho. Se o tempo de resposta a uma solicitação (consulta) é muito longo, o usuário ficará insatisfeito e pode até deixar de utilizar o sistema. Porém, o desempenho de um sistema de banco de dados depende de diversos fatores, dentre eles destacam-se: a eficiência das estruturas de dados internas usadas para representar os dados no banco de dados; as técnicas utilizadas para a troca de dados entre o disco rígido e a memória principal do sistema; bem como as estruturas de índices utilizadas para recuperar dados de forma mais eficiente.

1.1 Introdução

Um **Banco de Dados (BD**) é uma coleção de dados inter-relacionados, representado informações sobre um domínio específico. Já um **Sistema Gerenciador de Banco de Dados** (como no Brasil) ou Sistema de Gerenciamento de Banco de Dados (**SGBD**) é o conjunto de programas de computador (*softwares*) responsáveis pelo gerenciamento de um banco de dados, o que inclui o armazenamento, o acesso, o controle de concorrência e a recuperação desses dados. Assim, o principal objetivo de um SGBD é retirar da aplicação cliente a responsabilidade de gerenciar o acesso, a manipulação e a organização dos dados. O SGBD disponibiliza uma interface para que os seus clientes possam incluir, alterar ou consultar dados. Um **Sistema de Banco de**

Dados (**SBD**) consiste em uma coleção de dados inter-relacionados (BD) e uma coleção de programas para prover o acesso aos dados armazenados (SGBD). O objetivo principal de um sistema de banco de dados é prover um ambiente que seja adequado e eficiente para uso no armazenamento e na recuperação de informações.

1.2 Arquitetura

Um SGBD é dividido em módulos, onde cada módulo trata um aspecto particular, e tem uma responsabilidade específica. Na maioria dos casos, o sistema operacional (SO) do computador fornece apenas os serviços mais básicos (tais como acesso aos dados armazenados no disco rígido etc) e o sistema de banco de dados precisa ser construído sobre essa base. Portanto, o projeto do sistema de banco de dados precisa incluir considerações sobre a interface (ou seja, sobre a comunicação e interação) entre o sistema de banco de dados e o sistema operacional.

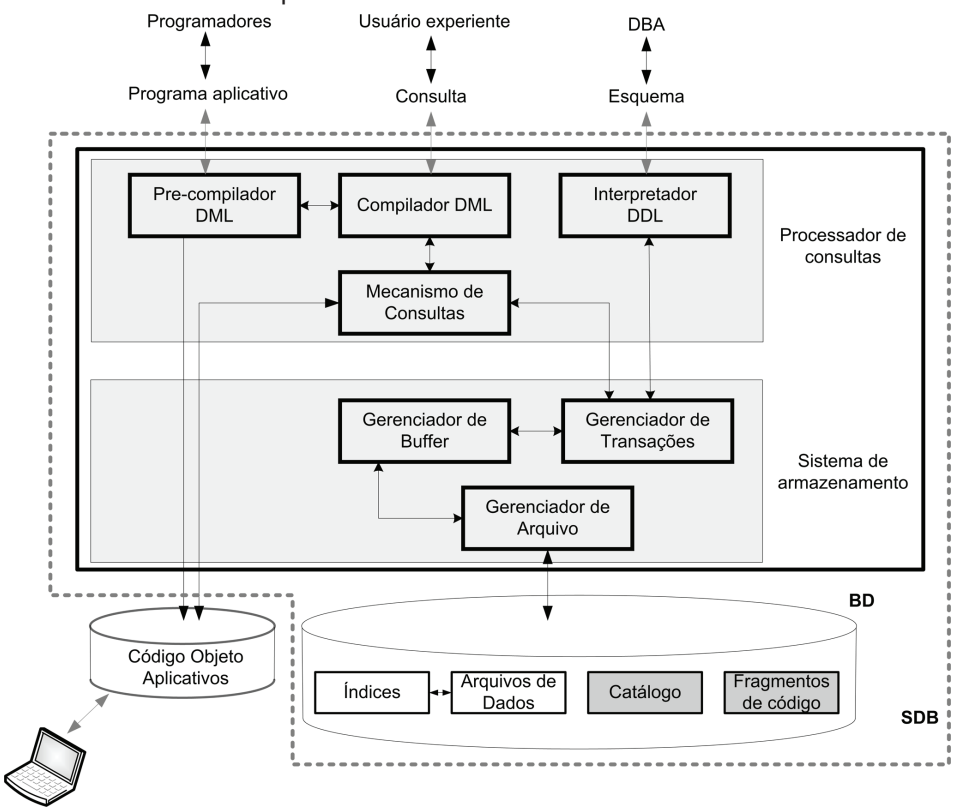


Figura 1.1 – Arquitetura de um SBD

A Figura 1 ilustra os principais componentes funcionais de um sistema de banco de dados. Nessa arquitetura, um SGBD (ou DBMS – *Database Management System*) é formado por dois módulos principais: o Processador de Consultas e o Sistema de Armazenamento.

O processador de consultas é responsável por traduzir as consultas recebidas do usuário ou de uma aplicação, as quais são representadas por comandos em uma determinada linguagem de consulta (como SQL, por exemplo), em operações de baixo nível, que o gerenciador do banco de dados pode interpretar e executar. Além disso, o processador de consultas tenta transformar uma requisição do usuário em uma representação interna que seja equivalente (ou seja, retorne os dados solicitados) à consulta recebida, porém mais eficiente em termos de desempenho. Dessa forma, busca-se encontrar uma boa estratégia para executar a consulta fornecida pelo usuário (ou pela aplicação). O processador de consultas é dividido nos seguintes módulos:

- **Compilador DML**

Analisa sintaticamente e semanticamente os comandos DML (*Data Manipulation Language*) expressos em uma determinada linguagem de consulta (SQL, por exemplo) recebidos do usuário. Em seguida, traduz esses comandos para uma das formas de representação interna de consultas (como por exemplo, a álgebra relacional). Exemplos de comandos DML são: SELECT, INSERT, UPDATE e DELETE.

- **Pré-Compilador DML**

Traduz comandos DML, embutidos em programas aplicativos (aplicações Java, por exemplo) que acessam o sistema de banco de dados (por meio de JDBC ou SQLJ, por exemplo), em chamadas a procedimentos (rotinas) em uma linguagem hospedeira. O pré-compilador precisa interagir com o processador de consultas para gerar o código apropriado.

- **Interpretador DDL**

Interpreta comandos DDL (*Data Definition Language*) e os armazena no catálogo (*log* ou metabase). Um catálogo é uma tabela que armazena metadados. Já os metadados descrevem o banco de dados, ou seja, o esquema do banco de dados. Exemplos de comandos DDL são: CREATE TABLE, ALTER TABLE, DROP TABLE, dentre outros.

- **Mecanismo de Consultas**

Responsável pela otimização e pela geração de planos de execução de consultas.

O Sistema de Armazenamento fornece a interface entre os dados

armazenados no disco (como conjuntos de *bytes* armazenados em trilhas e setores do disco rígido) e os programas aplicativos e de consulta que necessitam acessar esses dados. Para isso, o sistema de armazenamento interage diretamente com o sistema operacional. O Sistema de Armazenamento é dividido nos seguintes módulos:

- **Gerenciador de Transações**

Responsável pelo controle de concorrência e pela recuperação do banco de dados após a ocorrência de falhas (falta de energia durante o funcionamento do SBD, por exemplo).

- **Gerenciador de Buffer**

Responsável por recuperar dados armazenados em um disco rígido e carregá-los na memória principal, em forma de páginas.

- **Gerenciador de Arquivo (File System)**

Responsável pelo armazenamento físico dos dados em disco. O gerenciador de arquivos gerencia a alocação do espaço em disco e as estruturas de dados usadas para representar a informação armazenada no disco. Logo, esse componente define se será utilizada uma alocação sequencial, sequencial indexada ou indexada, por exemplo.

Adicionalmente, diversas estruturas de dados são requeridas como parte da implementação do sistema físico de um banco de dados (BD), incluindo:

- **Arquivos de dados**, que armazenam o banco de dados propriamente dito.
- **Catálogo ou Dicionário de dados**, que armazena metadados sobre a estrutura do banco de dados. Esses metadados incluem: nomes das tabelas; atributos de cada tabela; definição de índice para uma tabela; além de informações estatísticas, como por exemplo, a cardinalidade de uma tabela. O dicionário de dados é usado com bastante frequência. Assim, deve-se dar grande atenção tanto ao projeto quanto à implementação do dicionário. Nesse sentido, deve-se buscar um projeto adequado e uma implementação eficiente do catálogo.
- **Índices**, estruturas que fornecem acesso rápido aos dados armazenados em disco. Isso é possível guardando-se determinados valores (chaves de busca e ponteiros). O funcionamento das estruturas de índices assemelha-se à utilização de um índice de um livro, de um catálogo telefônico ou da agenda de um telefone celular. No caso da agenda telefônica, por exemplo, quando fornecemos a letra “e” iremos obter todos os contatos cujo nome inicia-se pela letra “e”, tornando o acesso a

esses dados mais rápido.

- **Fragmentos de Código**, tais como procedimentos armazenados (*stored procedures*) e gatilhos (*triggers*).

1.3 Classificação dos Sistemas de Bancos de Dados

Os primeiros SGBDs eram hospedados (instalados) em *mainframes* (computadores de grande porte). Todos os módulos que compõem o SBD eram executados no *mainframe*. Todas as funções do SBD eram executadas pelo *mainframe*. Além disso, todos os programas aplicativos que utilizavam o SBD também eram executados no *mainframe*. Assim, a maioria dos usuários fazia acesso aos SBD's via terminais que não possuíam poder de processamento, apenas a capacidade para entrada e saída (visualização) de dados. Todos os processamentos, tanto do SBD quanto dos programas aplicativos, eram realizados remotamente, no *mainframe*. Apenas as informações a serem visualizadas e os controles eram enviados do *mainframe* para os terminais de visualização (denominados também de terminais burros), conectados ao *mainframe* por meio de redes de comunicação. À medida que os preços dos dispositivos de *hardware* foram decrescendo, muitos usuários trocaram seus terminais por computadores pessoais (*Personal Computer* – PC) e por estações de trabalho. Inicialmente, os SGBDs usavam esses computadores da mesma maneira que usavam os terminais, ou seja, o SGBD era centralizado e toda sua funcionalidade, execução de programas aplicativos e processamento da interface do usuário eram executados em apenas uma máquina (denominada servidor). Gradualmente, os SGBDs começaram a explorar a disponibilidade do poder de processamento no lado do usuário (ou cliente), o que levou à arquitetura cliente-servidor. Atualmente, existem várias tendências para arquitetura de Sistemas de Bancos de Dados, nas mais diversas direções. Essas tendências serão discutidas a seguir.

As arquiteturas possíveis para um Sistema de Bancos de Dados podem ser classificadas em dois grandes grupos: **Arquitetura Centralizada** e **Arquitetura Distribuída**.

Na arquitetura centralizada os componentes do SBD residem no mesmo *host* (*hardware*). Os SBDs que utilizam essa arquitetura são denominados **Sistema de Banco de Dados Centralizados**.

Nas arquiteturas distribuídas, busca-se distribuir alguns dos seguintes critérios: função, controle e dados. Nesse contexto, algumas arquiteturas de SBDs são possíveis:

- **Sistema de Banco de Dados Cliente-Servidor**
  Distribuição de funções do SGBD entre clientes e servidor.
- **Sistema de Banco de Dados Paralelos**
  Distribuição do controle de funções do DBMS entre diversos sistemas computacionais.
- **Sistema de Banco de Dados Distribuídos**
  Distribuição de dados através de diversos SBDs homogêneos.
- **Sistema de Banco de Dados Heterogêneos**
  Caracteriza-se pela distribuição de dados através de SBDs heterogêneos e autônomos. Diversas abordagens foram propostas para SBDs Heterogêneos, dentre elas destacam-se:
  - **Sistema de banco de dados múltiplos (MDBS – Multiple Database System)**
  - **Sistema de banco de dados federados (FDBS – Federated Database System)**
  - **Arquitetura de Mediadores**
- **Sistema de Banco de Dados Móvel**
  Caracteriza-se pela distribuição de funções e de dados do SGBD entre clientes e servidor em ambientes de computação móvel.

Exercícios

1. Diferencie Banco de Dados, Sistema de Gerenciamento de Banco de Dados e Sistema de Banco de Dados?

2. Descreva os componentes funcionais de um SGBD.

3. Como podemos classificar os Sistemas de Bancos de Dados?

4. Quais os componentes do SGBD que interagem diretamente com o Sistema Operacional?

5. Quais os componentes do SGBD que interagem com o usuário?

2. ARMAZENAMENTO DE DADOS

Um dos aspectos importantes que distinguem os sistemas de bancos de dados de outros sistemas é a habilidade de um SBD de lidar de forma eficiente com grandes volumes de dados. Embora um sistema de banco de dados ofereça uma visão de alto nível dos dados armazenados, esses dados, em última instância, precisam ser armazenados como *bits*, em trilhas e setores, em um ou mais dispositivos de armazenamento. Atualmente, a grande maioria dos SBDs atuais armazena dados em discos magnéticos. Quando os dados necessitam ser processados, o SBD copia os dados armazenados nos discos magnéticos para a memória principal do computador. Logicamente, após os dados armazenados na memória principal do computador serem atualizados (alterados), esses precisam ser copiados novamente para o disco rígido. Essas operações são denominadas operações de entrada e saída (E/S ou I/O) de dados. Em geral, as leituras e as escritas de dados são realizadas por meio de unidade denominada página de dados. Assim, o SBD lê ou grava uma ou mais páginas de dados por vez.

Além disso, o SBD pode copiar os dados armazenados em disco para fitas e outros dispositivos de armazenamento, por questões de segurança (*backup*), e retornar esses dados para o disco magnético quando for necessário. As características físicas dos dispositivos de armazenamento desempenham um papel importante no modo como os dados são armazenados. Por exemplo, o acesso aleatório a um dado armazenado em disco é muito mais lento que em memória principal. Contudo, a capacidade de armazenamento da memória principal é muito menor que a de um disco rígido. Neste capítulo, discutiremos as principais técnicas utilizadas pelos SBDs para representar, armazenar e recuperar dados.

2.1 Introdução

Na maioria dos sistemas computacionais existem vários tipos de armazenamentos de dados. Esses meios físicos de armazenamento podem ser classificados de acordo com diferentes fatores, como por exemplo: a velocidade com que os dados podem ser acessados, o custo e a confiabilidade. A seguir, destacamos os principais tipos de meios físicos de armazenamento. Esses meios de armazenamento formam uma hierarquia de armazenamento que inclui as seguintes categorias:

- **Memória Interna do Processador**

Compreende um pequeno conjunto de registradores de alta velocidade. Esses registradores são utilizados como memória de trabalho para armazenamento temporário de instruções e dados. Alguns desses registradores são: registrador acumulador, registrador B, registrador contador etc.

- **Memória *Cache***

Dispositivo de memória que funciona como dispositivo de armazenamento temporário e intermediário entre registradores e memória principal. Como exemplo desses dispositivos, podemos citar: Cache L1 (pequena porção de memória estática presente dentro do processador) e Cache L2 (porção de memória bem maior que a Cache L1, embora, em geral, mais lenta. Alguns processadores colocam essa cache fora do processador).

- **Memória Principal (ou Memória Primária)**

Utilizada no armazenamento de programas e dados que estão sendo manipulados pelo sistema computacional. É uma memória relativamente rápida, cara, normalmente apresenta capacidade muito limitada e é volátil, ou seja, seu conteúdo é perdido em caso de queda de energia ou crash do sistema. Os endereços de memória são facilmente e rapidamente acessados pelo conjunto de instruções da CPU. A memória principal é denominada RAM (*Random Access Memory*) ou Memória de Acesso Randômico (Aleatório). Atualmente começam a surgir tecnologias de bancos de dados armazenados inteiramente em memória principal (*in-memory database*).

- **Memória *Flash***

A memória *flash* difere da memória principal porque os dados sobrevivem à falta de energia. A leitura de dados da memória *flash* apresenta aproximadamente a mesma velocidade da leitura de dados em memória principal. Porém, a escrita de dados na memória *flash* é mais complicada. Para escrever sobre um endereço de memória *flash* que já foi escrito anteriormente, é necessário apagar um bloco de memória inteiro ao mesmo tempo e, em seguida, gravar o novo conteúdo. Uma desvantagem da memória *flash* é que ela só pode aceitar número limitado de ciclos de apagamento/escrita, variando de 10.000 a 1 milhão. A memória *flash* é um tipo de memória EEPROM (***E**lectrically-**E**rasable **P**rogrammable **R**ead-**O**nly **M**emory).*

A memória *flash* ganhou popularidade como um substituto para os discos magnéticos, para armazenar pequenos volumes de dados, em equipamentos portáteis como câmeras fotográficas digitais e "*pendrives*" USB (*Universal Serial Bus*). Contudo, atualmente, a memória *flash* começa a ser utilizada para substituir

os discos rígidos em computadores portáteis (*notebooks*).

- **Memória Secundária (Armazenamento em Discos Magnéticos)**

O principal meio para armazenamento de dados em longo prazo é o disco magnético. Esse dispositivo apresenta capacidade de armazenamento maior que a memória principal. Em contrapartida, o acesso a dados em disco é mais lento que na memória principal. Os discos magnéticos apresentam armazenamento não volátil, ou seja, os dados armazenados sobrevivem a faltas de energia e a falhas do sistema. Assim, os discos magnéticos são utilizados para armazenar dados de forma persistente (permanente), podendo armazenar programas aplicativos, grandes arquivos de dados etc. Normalmente, o banco de dados inteiro é armazenado em disco magnético. Logo, o SBD precisa mover os dados do disco para a memória principal, de modo que possam ser acessados. Posteriormente, os dados que foram modificados na memória principal precisam ser gravados (atualizados) em disco. Logicamente, os discos magnéticos também estão sujeitos a falhas. Dessa forma, dados podem ser perdidos. Contudo, essas falhas normalmente ocorrem com muito menos frequência do que as falhas do sistema (tais como falta de energia e "*crash*").

- **Memória Terciária**

A memória terciária é composta por dispositivos mais lentos, tais como: fitas magnéticas e discos óticos (CD – *Compact Disk* e DVD – *Digital Video Disk*). Esse tipo de memória é utilizado para armazenar um grande volume de dados (*backup*).

Nos discos ópticos, os dados são armazenados de forma óptica e são lidos por um laser. Sistemas de ***junkebox*** de discos ópticos contêm algumas unidades de leitura/gravação e vários discos que podem ser carregados em uma das unidades automaticamente (por um braço mecânico).

O armazenamento em fita é usado principalmente para *backup* e arquivamento de dados. Embora a fita magnética seja mais barata do que os discos ópticos, o acesso aos dados é muito mais lento, pois a fita precisa ser acessada sequencialmente desde o início. Assim, o armazenado em fita é chamado de armazenamento de acesso sequencial, enquanto o armazenamento em disco é chamado de armazenamento de acesso direto ou aleatório. As fitas possuem alta capacidade de armazenamento, além disso, podem ser removidas das unidades (leitoras/gravadoras) de fita e transportadas. Logo, são bastante adequadas para o armazenamento de grandes volumes de dados, por longos períodos de tempo. Bibliotecas de fitas (*jukeboxes*) são usadas para manter coleções de dados excepcionalmente grandes, como dados de satélites, que

poderiam incluir até centenas de *terabytes* (1 *terabyte* = 10^{12} *bytes*), ou ainda vários *petabytes* (1 *petabyte* = 10^{15} *bytes*) de dados.

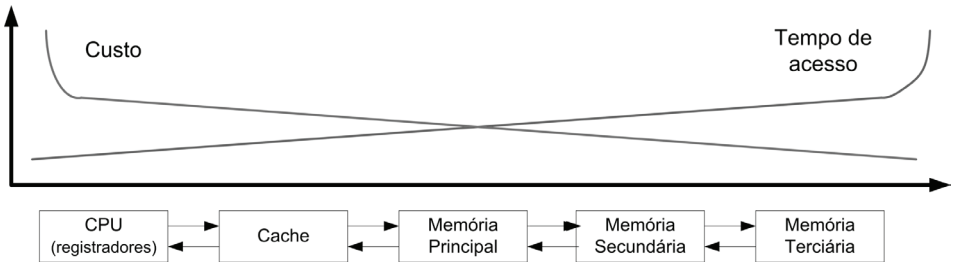


Figura 2.1 – Hierarquia de Meios de Armazenamento

Os diversos meios físicos de armazenamento podem ser organizados em uma hierarquia (Figura 2), de acordo com sua velocidade (tempo de acesso) e seu custo. Os níveis mais altos são caros, mas são rápidos. Ao descermos na hierarquia, o custo por *bit* diminui, a capacidade de armazenamento aumenta e a velocidade diminui (ou seja, o tempo de aceso também cresce).

A seguir iremos descrever as principais características dos meios físicos de armazenamento.

2.1.1 Características dos Meios Físicos de Armazenamento

- **Custo (c)**: O custo de um meio físico de armazenamento é definido por:

  c=CT/CA (dollars/bit)

  em que CT representa o custo total do sistema de memória completo cuja capacidade de armazenamento é dado por CA.

- **Tempo de Acesso (T_A)**: Representa o tempo médio necessário para ler uma quantidade fixa de informação da memória, como por exemplo, uma palavra (*word*). O tempo de acesso é utilizado como medida de *performance* (desempenho) para dispositivos de memória.
- **Taxa de acesso (b_A)**: A taxa de acesso de um dispositivo de memória é dada por $1/T_A$ e representa a quantidade de palavras lidas por segundo.

- **Método de Acesso**:

  (i) *Random-access memory* (RAM): Dispositivos de memória cujas posições podem ser acessadas em qualquer ordem. O tempo de acesso é independente da localização a ser acessada, uma vez que existe um mecanismo de acesso (leitura/gravação) para cada posição de memória. Esse método de acesso é utilizado na memória principal e na memória *flash*, por exemplo.

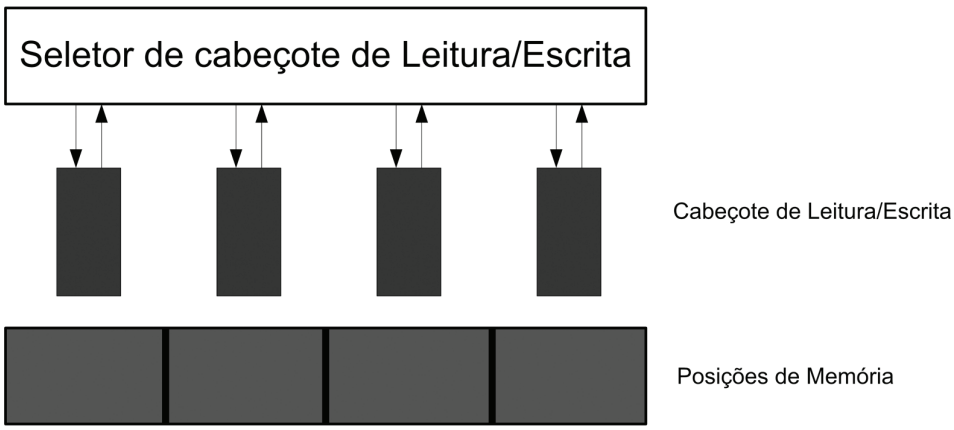


Figura 2.2

(ii) *Serial-access memory* (memória de acesso sequencial): Dispositivos cujas posições só podem ser acessadas em sequências pré-definidas. Esse tipo de memória apresenta um único mecanismo de acesso (leitura/gravação) é compartilhado por várias posições de memória. Esse método de acesso é utilizado nas fitas magnéticas, por exemplo.

- **Alterabilidade**: Característica que representa a capacidade de permitir a alteração do conteúdo de memória.

  (i) *Read-only memory (ROM). Memória somente de leitura.* Esse *tipo de memória é utilizado no BIOS (Basic Input/ Output System ou Sistema Básico de Entrada/Saída). O BIOS é um programa de computador previamente gravado em uma memória ROM. Esse programa é executado automaticamente quando um computador é ligado. O BIOS é responsável pelo teste automático de alguns dispositivos de hardware, bem como por iniciar a carga do sistema operacional.*

(ii) *Read-write memory* (RWM). Memória de leitura e de escrita. Como exemplo desse tipo de memória, podemos citar a memória principal e os discos rígidos, dentre outros.

- **Persistência de Armazenamento**: Os processos físicos envolvidos no armazenamento de dados são algumas vezes instáveis, podendo provocar a perda da informação armazenada após certo período de tempo.

  A seguir discutimos algumas propriedades de memória de que podem destruir a informação armazenada.

  (i) *Destructive read-out* – DRO (leitura destrutiva): Em alguns dispositivos de armazenamento, o método de leitura destrói a informação armazenada. Em dispositivos DRO, cada operação de leitura precisa ser seguida de uma operação de escrita para restaurar o estado original da memória.

  (ii) Memória dinâmica: Certos dispositivos de armazenamento apresentam a seguinte característica: Um "1" armazenado torna-se um "0" devido a algum processo de decaimento. Por exemplo, muitos dispositivos armazenam um "1" por meio de uma carga elétrica em um capacitor. Com o passar do tempo, a carga armazenada no capacitor tende a se descarregar (decair). Logo, esses dispositivos necessitam de um processo de recarga, denominado de *refreshing*.

  (ii) Volatilidade: Fenômeno no qual certos dispositivos de memória têm seu conteúdo destruído por falhas elétricas ou *crash* de sistema. Distinguem-se dois tipos de memória: memória volátil e memória não volátil. Nas memórias voláteis, o seu conteúdo é perdido quando o fornecimento de energia é interrompido. As memórias não voláteis conservam o seu conteúdo mesmo após a interrupção do fornecimento de energia.

2.1.2 Armazenamento de Bancos de Dados

Em geral, os bancos de dados envolvem grandes volumes de dados, os quais devem ser armazenados por longos períodos de tempo. Além disso, os dados necessitam ser acessados e processados diversas vezes. Nesse sentido, os bancos de dados, geralmente, necessitam ser armazenados de forma permanente (ou persistente) em memória secundária, ou seja, em discos magnéticos. A seguir, discutimos, em detalhes, os motivos pelos quais os bancos

de dados necessitam ser armazenados em discos rígidos (ou seja, em meios de armazenamento secundário):

- Frequentemente os bancos de dados são muito grandes (ou seja, envolvem grandes volumes de dados) para caberem inteiramente na memória principal;
- As circunstâncias que causam a perda permanente de dados armazenados são menos frequentes nos meios de armazenamento secundário (discos magnéticos, por exemplo) do que nos meios de armazenamento primário (memória principal, por exemplo).
- O custo de armazenamento por unidade de dados é menor nos discos magnéticos do que nos dispositivos de armazenamento primário;

Por esses motivos, espera-se que os discos magnéticos continuem a ser a mídia mais utilizada para armazenar grandes bancos de dados nos próximos anos. Contudo, atualmente, o armazenamento de bancos de dados tanto em memória *flash* quanto inteiramente em memória principal (*in-memory database*) começam a ser investigados.

Vale destacar ainda que as técnicas utilizadas para armazenar grandes volumes de dados em discos magnéticos são importantes tanto para os administradores de bancos de dados e DBAs (*Database Administrators*) quanto para desenvolvedores de SGBDs. Os DBAs devem conhecer as vantagens e as desvantagens de cada técnica de armazenamento, a fim de poderem projetar, implementar e operar, com eficiência, um banco de dados em um SGBD específico. Em geral, os SGBDs fornecem diversas opções para a organização dos dados, e o processo de projeto físico envolve escolher, dentre as opções disponíveis, as técnicas de organização que melhor se adéquem aos requisitos de uma determinada aplicação. Já os desenvolvedores (ou fabricantes) de SGBDs devem estudar as técnicas de organização de dados a fim que possam implementá-las de forma eficiente e, assim, proporcionar boas opções aos DBAs e usuários de SGBDs.

2.2 Discos Magnéticos

Disco rígido, ou disco duro, (popularmente conhecido como *winchester*) ou ainda HD (do inglês *Hard Disk*), é a parte do computador onde são armazenadas as informações, ou seja, é a "memória permanente" propriamente dita. O disco

rígido é memória física e não volátil. Logo, as informações armazenadas no disco rígido não são perdidas quando o computador é desligado.

O disco rígido é um sistema lacrado contendo discos de metal recobertos por um material magnético onde os dados são gravados por meio de cabeçotes de leitura/gravação. O disco rígido é revestido externamente por uma proteção metálica que é presa ao gabinete do computador por parafusos. São nos discos rígidos que, normalmente, gravamos dados (informações) e programas, os quais devem ser carregados para a memória principal do computador a fim de que sejam utilizados ou executados, respectivamente.

A utilização de discos rígidos é necessária porque o conteúdo da memória principal (RAM) é apagado (perdido) quando o computador é desligado. Dessa forma, na próxima vez em que o computador for ligado, temos um meio de executar novamente os programas e carregar, na memória principal, arquivos contendo os dados. O disco rígido é também denominado de memória de massa ou ainda de memória secundária.

Um disco rígido possui uma ou várias superfícies de gravação/leitura com uma estrutura de gravação composta por cilindros, trilhas e setores. Em cada superfície de um disco, existem várias trilhas que são arranjadas de forma concêntrica (que possuem o mesmo centro). Em geral, existem de 20 a 1500 trilhas por superfície. Cada trilha está subdividida em setores, conforme mostra a Figura 2.3.. Um setor representa a menor unidade de informação que pode ser lida ou escrita (gravada) no disco. O tamanho de um setor pode variar de 32 *bytes* até 4096 *bytes*, dependendo da tecnologia utilizada. Contudo, tipicamente, um setor possui 512 *bytes* e existem de quatro a 32 setores por trilha. Um cilindro é definido como sendo um conjunto de trilhas verticalmente alinhadas e com mesmo raio (e, consequentemente com o mesmo diâmetro).

As superfícies de um disco magnético são recobertas por uma camada magnética extremamente fina. Na verdade, quanto mais fina for essa camada de gravação, maior será sua sensibilidade, e consequentemente maior será a densidade de gravação permitida por ela. A densidade de gravação indica a quantidade de dados que pode ser armazenada em uma determinada área da superfície.

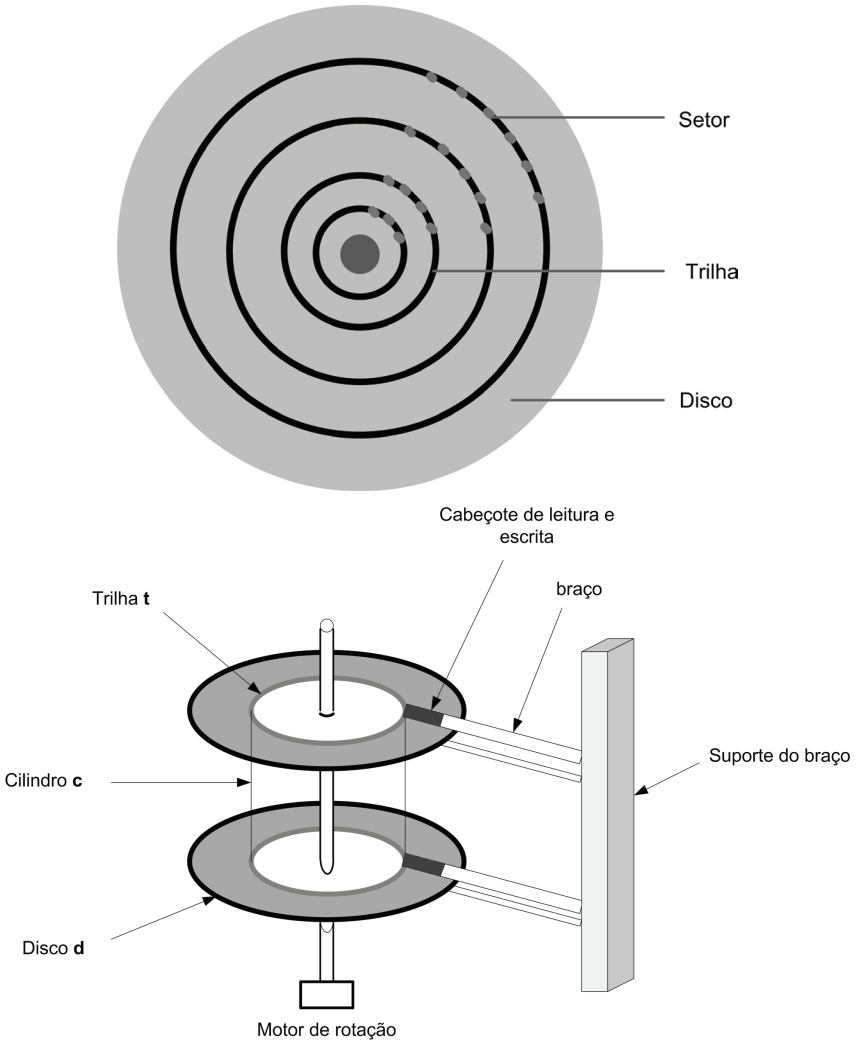


Figura 2.3 Visão geral de um disco rígido

Assim, discos com um mesmo tamanho podem apresentar capacidades de armazenamento diferentes (uma vez que as densidades de gravação desses discos podem ser diferentes).

2.2.1 Acessando uma Unidade de Disco

O motor de rotação faz o disco girar a uma velocidade constante (60, 90 ou 120 rps). Cada superfície do disco possui um cabeçote de leitura/escrita. O cabeçote está montado em um dispositivo mecânico denominado braço do disco. O braço movimenta-se transversalmente pelo disco para acessar as diferentes trilhas do disco.

Uma unidade de disco possui, geralmente, um conjunto de discos (superfícies). Os cabeçotes de leitura/escrita de todas as superfícies de discos de uma unidade movimentam-se em conjunto (simultaneamente) e são acionadas pelo suporte do braço. Assim, quando o cabeçote de leitura/gravação da superfície 1 está posicionado na trilha 10, por exemplo, todos os cabeçotes de leitura/gravação de todas as demais superfícies também estão posicionados sobre a trilha 10 (de suas respectivas superfícies).

A título de exemplo, suponha que os discos de uma unidade A apresentem 1000 trilhas cada um. Em um instante de tempo t todos os cabeçotes acessam a mesma posição nos diferentes discos (superfícies). Assim, os cabeçotes acessam trilhas de diferentes discos ou superfícies, mas que apresentam mesmo diâmetro, ou seja, pertencem ao mesmo cilindro (como por exemplo, o conjunto de todas as trilhas 450 dos discos da unidade A). Por esse motivo, procura-se otimizar o acesso ao disco armazenando-se os dados de um mesmo arquivo em um mesmo cilindro.

A superfície de gravação dos discos (ou pratos) é composta de materiais sensíveis ao magnetismo (geralmente, óxido de ferro). O cabeçote de leitura/gravação manipula as moléculas desse material por meio de seus polos. Para isso, a polaridade dos cabeçotes muda numa frequência muito elevada: quando está positiva, atrai o polo negativo das moléculas e vice-versa. Assim, de acordo com essa polaridade é que são gravados os *bits* (0 e 1). No processo de leitura de dados, o cabeçote simplesmente "lê" o campo magnético gerado pelas moléculas e gera uma corrente elétrica correspondente, cuja variação é analisada pela controladora do HD com a finalidade de determinar os *bits* que estão sendo lidos (0 ou 1).

2.2.2 Medidas de Desempenho de Discos

- **Tempo de procura (T_P) ou Tempo de Busca ou *Seek Time***: Tempo gasto para posicionar o braço do disco sobre a trilha correta (desejada). Como a movimentação do braço do disco é um processo mecânico, o tempo de procura tende a ser elevado, em comparação com os demais tempos envolvidos na leitura de um setor (menor unidade de leitura).
- **Tempo médio de latência rotacional (T_L):** Tempo médio necessário para que o setor desejado passe pelo cabeçote de leitura/escrita. O T_L é definido como 1/2 do tempo de uma rotação completa. Assim, se r é a taxa de rotações/segundo, então $T_L=(2r)^{-1}$.

- **Taxa de transferência de dados (TTr):** Taxa na qual os dados são lidos ou armazenados por segundo. Se N é a capacidade das trilhas em palavras, então TTr= rN.
    - **Tempo de transferência de dados de um setor (T_T)**: Tempo (em segundos) necessário para transferir dados de um setor. Se n é o tamanho (em palavras) de um setor, então $T_T = n(rN)^{-1}$.
    - **Tempo de acesso a um setor (T_A):** Tempo gasto desde um pedido de leitura/escrita até o fim da transferência de dados. A fórmula para determinar o tempo de acesso de um disco é dada por:

$$T_A = T_P + T_L + T_T$$
$$T_A = T_P + (2r)^{-1} + n(rN)^{-1}$$

O acesso a disco é extremamente lento. Logo, nos SGBD atuais, o custo de acesso a disco domina (ou seja, é o principal componente) o custo de execução das consultas. Assim, em relação à execução de uma determinada consulta SQL pelo SGBD, o custo de acesso a disco é maior que o custo de processamento (custo de utilização do processador), por exemplo.

2.3 Técnicas RAID

RAID é a sigla para *Redundant Array of Independent Disks*. Sua definição em português seria "Matriz Redundante de Discos Independentes". A tecnologia RAID combina vários discos rígidos (HDs) para formar uma única unidade lógica. Em outras palavras, um conjunto de HDs que funciona como se fossem um único HD. Essa estratégia permite ter uma alta tolerância contra falhas, pois se um disco tiver problemas, os demais podem continuam funcionando. Atualmente, o RAID é uma tecnologia consolidada. Essa técnica foi proposta por pesquisadores da Universidade de Berkesley, na Califórnia (EUA) no final da década de 1980.

Para que tenhamos um dispositivo RAID, é preciso utilizar pelo menos dois HDs. O sistema operacional, nesse caso, enxergará os discos como uma unidade lógica única. Quando há gravação de dados, os mesmos se repartem entre os discos do RAID (dependendo do nível de RAID utilizado). Com isso, além de garantir a disponibilidade dos dados em caso de falha de um disco, é possível também equilibrar o acesso às informações, de forma que não haja "gargalos" (“*hot spots*”).

As técnicas RAID são baseadas em dois mecanismos principais: o espelhamento e a distribuição paralela de dados. O espelhamento assegura

confiabilidade (através da redundância dos dados), contudo é uma tecnologia extremamente cara. A distribuição paralela de dados possibilita altas taxas de transferência de dados, porém não garante confiabilidade. Assim, as técnicas RAID buscam combinar diferentes graus de distribuição de dados e de redundância.

2.3.1. Os níveis de RAID

A tecnologia RAID funciona de várias maneiras. Tais maneiras são conhecidas como "níveis de RAID".

RAID nível 0 – Esse nível também é conhecido como "*Striping*" ou "Fracionamento". O RAID Nível 0 utiliza uma distribuição paralela e não redundante dos blocos que compõem um arquivo. Nele, os dados são divididos em pequenos segmentos e distribuídos entre os discos. Esse nível não oferece tolerância a falhas, pois não existe redundância. Isso significa que uma falha em qualquer um dos HDs pode ocasionar perda de informações. Por essa razão, o RAID 0 é usado para melhorar a performance (desempenho) do computador, uma vez que a distribuição dos dados entre os discos proporciona grande velocidade na gravação e leitura de informações, uma vez que agora dois setores podem ser lidos simultaneamente (em paralelo), um em cada disco. Assim, quanto mais discos houver, mais velocidade é obtida. Isso porque, se os dados fossem gravados em um único disco, esse processo seria feito de forma sequencial.

RAID nível 1 – Também conhecido como "*Mirroring*" ou "Espelhamento". O RAID 1 funciona adicionando HDs paralelos (espelhos) aos HDs principais existentes no computador. Assim, se, por exemplo, um computador possui dois discos, pode-se aplicar mais um HD para cada um, totalizando quatro. Os discos que foram adicionados trabalham como uma cópia do primeiro. Assim, se uma informação é gravada em dos discos principais, essa informação também é gravada em seu disco espelho. Daí o nome de "espelhamento". Dessa forma, se um dos HDs principais apresentar falha, o seu disco espelho pode assumir imediatamente a função do primeiro, possibilitando que o sistema continue funcionando. Contudo, no RAID Nível 1 a gravação de dados é mais lenta, pois agora cada operação de escrita é realizada duas vezes, no disco principal e no seu disco espelho. No entanto, a leitura dessas informações pode ser rápida, uma vez que se pode acessar o disco principal e sua cópia espelho simultaneamente (em paralelo). O RAID 1 utiliza uma organização de discos com distribuição paralela (baseada em blocos) e redundância (implementada através de replicação).

RAID nível 2 – O RAID 2 utiliza uma organização de discos com distribuição paralela (baseada em *bytes*) e redundância implementada através de *bits* de correção, os quais podem ser implementados por meio de *bits* de paridade. Vale destacar que se pode utilizar tanto paridade par quanto paridade ímpar.

RAID nível 3 – Nesse nível, os dados são divididos entre os discos que compõem a matriz (RAID), exceto um, que armazena informações de paridade. Assim, se cinco discos compõem o dispositivo RAID, quatro discos serão utilizados para armazenar dados, enquanto um disco será utilizado para armazenar os *bits* de paridade. O RAID 3 consegue oferecer altas taxas de transferência e confiabilidade das informações armazenadas. Para usar o RAID 3, pelo menos três discos são necessários (dois discos de dados e um de paridade).

RAID nível 4 – O RAID 4 não apresenta distribuição, ou seja, os blocos são armazenados da mesma forma que em discos comuns. Assim, uma leitura de bloco acessa somente um disco. Isso permite que outras solicitações sejam processadas pelos outros discos de forma paralela. A redundância é implementada através de blocos de paridade. Um disco redundante (disco de paridade) armazena todos os blocos de paridade. Nesse disco, o bloco i armazena *bit* de paridade dos blocos i de todos os discos de dados.

RAID nível 5 – O RAID Nível 5 é bastante semelhante ao RAID Nível 4, exceto pelo fato de que os *bits* de paridade não ficam armazenados em um único disco, mas em todos os discos que compõem a matriz. Isso faz com que a gravação de dados seja mais rápida, pois não é necessário acessar um disco de paridade em cada gravação. Assim, o disco de paridade não se torna um "gargalo" ("*hot spot*"). Apesar disso, como a paridade é distribuída entre os discos, o nível 5 tende a ter um menor desempenho que o RAID 4 nas operações de leitura. O RAID 5 é o nível mais utilizado atualmente.

RAID nível 6 – O RAID Nível 6 é semelhante ao RAID Nível 5. Contudo. O RAID Nível 6 utiliza redundância P + Q, ou seja, armazena informações adicionais para recuperar dados no caso de falha em mais de um disco. Assim, o RAID Nível 6 garante um maior grau de confiabilidade que o RAID Nível 5. Contudo, o RAID 6 requer um mínimo de quatro discos para ser implementado.

RAID nível 10 ou 1 + 0 – Em um RAID 1+0 os dados são primeiramente espelhados, e, em seguida, para cada espelho há a segmentação (distribuição) sobre vários discos.

RAID 0 + 1 – O RAID 0 + 1 é uma combinação dos níveis 0 (*Striping*) e 1 (*Mirroring*), onde os dados são primeiramente distribuídos (segmentados)

entre os discos, para melhorar o desempenho, e, em seguida, os dados são espelhados, para garantir a redundância (melhorando a disponibilidade dos dados). Assim, é possível unir os ganhos de desempenho proporcionados pelo nível 0 com a redundância do nível 1. No entanto, são necessários pelo menos quatro discos para montar um RAID desse tipo. Tais características fazem do RAID 0 + 1 o mais rápido e seguro, porém o mais caro de ser implantado.

2.3.2 Tipos de RAID

Existem dois tipos de RAID, sendo um baseado em *hardware* e o outro baseado em *software*. Cada um desses dois modelos possui vantagens e desvantagens. O RAID baseado em *hardware* é o modelo mais utilizado, pois não depende de sistema operacional – uma vez que, nesse caso, os SOs enxergam o RAID como um único disco, de grande capacidade e são bastante rápidos, o que possibilita explorar integralmente seus recursos. Sua principal desvantagem é o alto custo. O RAID baseado em *hardware* utiliza dispositivos denominados "controladores RAID", que podem ser, inclusive, conectados em *slots* PCI da placa-mãe do computador. Já o RAID baseado em *software* não é muito utilizado na prática, pois apesar de apresentar custo menor, apresenta um desempenho menor (sendo mais lento), sua instalação/configuração é mais complexa e sua utilização depende do sistema operacional. O RAID implementado por *software* depende também dos recursos computacionais do computador em que é utilizado. Logo, seu desempenho fica atrelado à capacidade de processamento e armazenamento do computador (*hardware*) no qual está instalado/configurado.

Exercícios

1. Tanto a memória principal quanto o disco possibilitam o acesso direto a uma determinada localização (página). Entretanto, em média, o acesso à memória principal é mais rápido. Qual a outra diferença fundamental entre esses dispositivos (do ponto de vista do tempo necessário para acessar uma determinada página)?

   O tempo de acesso à memória principal é independente da localização a ser acessada.

2. Caso você tenha um arquivo que frequentemente é percorrido sequencialmente, qual a melhor maneira de armazenar as páginas desse arquivo em um disco?

O ideal seria armazenar as páginas desse arquivo em blocos contíguos no disco e preferivelmente em um mesmo cilindro, pois o acesso ficaria mais rápido devido à menor necessidade de movimentação dos cabeçotes de leitura/gravação.

3. Considere um disco com setores de 512 *bytes* (tamanho), 2.000 trilhas por superfície, 50 setores por trilha, cinco pratos (discos físicos) de dupla face e tempo de busca de 10 ms.

   - Qual a capacidade de uma trilha em *bytes*?

     tamanho do setor x número de setores por trilha =

     512 x 50 *bytes* = 25.600 *bytes*.

   - Qual a capacidade de uma superfície?

     tamanho do setor x número de setores por trilha x número de trilhas =

     512 x 50 x 2000 *bytes* = 51.200.000 *bytes*

   - Qual a capacidade de um disco (prato)?

     tamanho do setor x número de setores por trilha x número de trilhas x número de faces =

     512 x 50 x 2000 x 2 *bytes* = 102.400.000 *bytes*

   - Quantos cilindros existem em um disco?

     Cilindro é o conjunto de trilhas pertencentes a diferentes superfícies e discos de uma mesma unidade, mas que apresentam a mesma posição relativa. No disco considerado na questão, cada cilindro possui 10 trilhas (cinco pratos x duas faces). No entanto, a quantidade de cilindros do disco é igual ao número de trilhas existentes em cada superfície. Logo, o disco possui 2.000 cilindros.

   - Forneça exemplos de tamanhos de blocos válidos. Pode existir um bloco de 256 *bytes*?

     Como pelo enunciado os setores do disco possuem 512 *bytes*, temos como exemplos de tamanhos de blocos válidos: 512, 1024, 2048 ou 4096 *bytes* por bloco. Como o setor é a menor unidade de informação que pode ser escrita no disco (Slide 36) e o setor do disco tem 512 *bytes*, então não é possível existir bloco menor que 512 *bytes*. Logo, NÃO pode existir um bloco de 256 *bytes* para o disco da questão.

   - Podemos ter blocos de 2048 *bytes*? e de 51.200?

     Como pelo enunciado os setores possuem 512 *bytes*, podemos ter blocos de 2048 *bytes* ou mesmo de 51.200, uma vez que 2048 e

51.200 são múltiplos de 512. Nesse caso, um bloco de 2048 *bytes* seria formado por 4 setores e um bloco de 51.200 *bytes* seria formado por 100 setores.

- Se a taxa de rotação for de 5.400 rpm (rotações por minuto), qual o tempo máximo de espera?
  Ta = Tp + Tl + Tt (Slide 41), onde:
  Ta – Tempo de acesso a um setor
  Tp – Tempo de procura = 10 ms = 1 / 100 s
  Tl – Tempo de Latência = $(2r)^{-1}$, onde r é a taxa de rotações/segundo
  Tt – Tempo de transferência = $n(rN)^{-1}$, onde n é o tamanho de um setor e N é a capacidades das trilhas.
  r = 5400 rpm = 90 rotações/segundo
  n = 512 *bytes*
  N = (512 x 50) *bytes*
  Assim, temos:
  $Ta = Tp + (2r)^{-1} + n(rN)^{-1}$
  $Ta = 1 / 100 + (2 \times 90)^{-1} + 512 \times (90 \times 512 \times 50)^{-1}$
  Ta = 1 / 100 + 1 / 180 + 512 / (90 x 512 x 50)
  Ta = 1 / 100 + 1 / 180 + 1 / 4500
  Ta = (45 + 25 + 1) / 4500
  Ta = 71 / 4500 = 0,015777778 s = 15,777777778 ms
- Assumindo-se que uma trilha de dados pode ser transferida a cada rotação, qual é a taxa de transferência?
  TTr = rN
  r = 5400 rpm = 90 rotações/segundo
  N = (512 x 50) *bytes*
  Assim:
  TTr = 90 x 512 x 50 = 2.304.000 *bytes*/s (aprox. 2,2 MB/s)

4. Considere a especificação do disco da questão anterior, e suponha que os blocos utilizados são de 1024 *bytes*. Suponha também que um determinado arquivo que contém 1000.000 registros de 100 *bytes* cada deve ser armazenado no disco em questão, e que os dados de um registro devem ser armazenados em um único bloco.
   - Quantos registros podem ser armazenados em um bloco?
     Tamanho do bloco = 1.024 *bytes*

Tamanho do registo = 100 *bytes*

Assim, até 10 registros (1.024 / 100) podem ser armazenados em um bloco, sendo que 24 *bytes* do bloco ficam ociosos por não serem capazes de armazenar um registro inteiro.

- Quantos blocos são necessários para armazenar o arquivo por inteiro?

  100.000 blocos são necessários para armazenar 1.000.000 de registros
- Quantos registros de 100 *bytes* cada podem ser armazenados usando esse disco?

  Capacidade total do disco = (512 x 50 x 2000 x 2 x 5) *bytes*

  Quantidade total de blocos no disco = (512 x 50 x 2000 x 2 x 5) / 1024 = 50 x 2000 x 5

  Como podemos ter 10 registros em cada bloco, o total de registros é:

  10 x qtd. blocos = 10 x (50 x 2000 x 5) = 5.000.000

  Logo, 5.000.000 de registros podem ser armazenados usando esse disco.
- Qual o tempo necessário para ler um arquivo contendo 100.000 registros de 100 *bytes* cada sequencialmente?

  Ta = Tp + Tl + Tt (*Slide* 41), onde:

  Ta – Tempo de acesso a um setor

  Tp – Tempo de procura = 10 ms = 1 / 100 s

  Tl – Tempo de Latência = $(2r)^{-1}$, onde r é a taxa de rotações/segundo

  Tt – Tempo de transferência = $n(rN)^{-1}$, onde n é o tamanho de um setor e N é a capacidades das trilhas.

  r = 5400 rpm = 90 rotações/segundo

  n = 512 *bytes*

  N = (512 x 50) *bytes*

  Em cada bloco cabem 10 registros de 100 *bytes*. 10.000 é o nº de blocos que precisam ser percorridos para a leitura de 100.000 registros. Como cada bloco (1024 *bytes*) tem dois setores (2 x 512 *bytes*), precisamos percorrer 20.000 setores para a leitura de 100.000 registros.

  Assim temos:

  T = Tp + $(2r)^{-1}$ + qtd. setores x $n(rN)^{-1}$

$T = 1 / 100 + (2 \times 90)^{-1} + 20.000 \times 512 \times (90 \times 512 \times 50)^{-1}$

$T = 1 / 100 + (1 / (2 \times 90)) + 20.000 \times 512 \times (1 / (90 \times 512 \times 50))$

$T = 1 / 100 + 1 / (2 \times 90) + 20.000 / (90 \times 50)$

$T = (45 + 25 + 20.000) / 4500 = 4,46$ s

- Qual o tempo necessário para ler um arquivo contendo 100.000 registros de 100 *bytes* cada em uma determinada ordem randômica?

  Ta = 71 / 4500 [ver questão 3 item g]

  T = qtd. registros x Ta

  T = 100.000 x 71 / 4500 = 1.577,777777778 s

3. GERENCIAMENTO DE *BUFFER*

O gerenciador de *buffer* é o subsistema do SGBD responsável pela alocação da porção de memória onde são armazenados os blocos (páginas) de informações que compõem um banco de dados e que são lidos/gravados nos discos rígidos (armazenamento secundário). Do ponto de vista do desempenho do SGBD, o ideal seria que todas as informações que um SGBD manipula estivessem na memória principal do computador. Como isso não é possível, uma vez que a memória principal é um recurso limitado (pequena capacidade de armazenamento) e caro (de elevado custo financeiro), é necessário gerenciar a alocação do espaço disponível na memória principal para o armazenamento dos blocos que compõem um banco de dados.

O *buffer* é a parte da memória principal disponível (alocada) para o armazenamento de cópias dos blocos (páginas) de um banco de dados. Há sempre uma cópia de cada bloco (página) que é mantida no disco rígido. Isso é necessário devido ao fato da memória principal ser volátil, enquanto a memória secundária é não volátil. Contudo, para que um determinado bloco possa ser efetivamente utilizado (processado) ele necessita estar armazenado na memória principal (mais precisamente no *buffer*). Assim, a cópia de um determinado bloco armazenada em disco pode constituir-se em uma versão mais antiga do que a versão da cópia do mesmo bloco armazenada no *buffer,* uma vez que todas as atualizações sobre os dados ocorrem na memória principal (ou seja, nas cópias dos blocos armazenadas no *buffer*).

3.1 Sistema de Memória Virtual

Quando uma aplicação solicita uma informação do banco e dados, o

SGBD antes de buscá-la no disco, verifica se essa já se encontra em algum bloco (página) no *buffer*. Vale lembrar que todos os dados de um banco de dados são armazenados em blocos (páginas) e que a menor unidade de leitura e gravação em um disco rígido é uma página (bloco). Caso a informação desejada já se encontre em um determinado bloco do *buffer*, o SGBD envia à aplicação o endereço desse bloco, para que esse seja lido/acessado pela aplicação. Caso a informação desejada não se encontre em nenhum dos blocos presentes no *buffer*, a cópia do bloco que contém essa informação e que está armazenada no disco rígido (memória secundária) deverá ser copiada para o *buffer*, criando-se uma cópia desse bloco na memória principal. Nesse momento, o bloco que contém a informação requisitada pela aplicação terá duas cópias: uma em disco e outra no *buffer*.

Para que isto seja possível, o gerenciador de *buffer* precisa alocar (reservar) um espaço no *buffer* para armazenar a cópia do bloco requisitado (que está em disco). Caso haja espaço no *buffer*, essa alocação é realizada sem nenhuma dificuldade. Porém, caso todo o espaço do *buffer* esteja ocupado, será necessário retirar algum dos blocos correntemente armazenados no *buffer* para liberar espaço. Nesse caso, o gerenciador de *buffer* deve verificar se o bloco eleito (escolhido) para ser retirado do *buffer* foi alterado. Em caso afirmativo, ele deverá ser reescrito (gravado) no disco. Caso contrário, ele pode ser simplesmente descartado. Em seguida, uma cópia do bloco requisitado pela aplicação (armazenado em disco) é armazenada no espaço (bloco) que foi liberado no *buffer*. Por fim, o endereço desse bloco (endereço no *buffer*) é enviado para a aplicação, a fim de que essa possa ler (acessar) a informação desejada.

O gerenciador de *buffer* funciona de maneira análoga (semelhante) ao gerenciador de memória virtual de um sistema operacional. Uma diferença importante é que um banco de dados pode ser muito maior que a memória virtual de um sistema computacional qualquer. Outra diferença é que as técnicas utilizadas para gerenciamento do *buffer* são “mais sofisticadas” que as de gerenciamento de memória virtual.

Mais formalmente, um sistema de memória virtual pode ser definido como um sistema de armazenamento hierárquico, com no mínimo dois níveis de memória. A maioria dos sistemas de memória virtual implementa hierarquia de dois níveis, compreendendo:

- **Uma memória principal M_1 (Buffer)**, cuja capacidade de armazenamento é representada por S_1 e cujo custo é dado por

C_1. O tempo de acesso à memória M_1 é dado por T_{A1}.

- **Uma memória secundária M_2 (Disco)**, cuja capacidade de armazenamento é representada por S_2 e cujo custo é dado por C_2. O tempo de acesso à memória M_2 é dado por T_{A2}.

Sabe-se que:

$$S_2 >> S_1 \; ; \; C_1 > C_2 \; ; \; T_{A1} < T_{A2}$$

O objetivo principal de um sistema de memória virtual é dar a ilusão ao usuário do SGBD que existe uma única memória principal com capacidade ilimitada. Além disso, sistema de memória virtual busca obter um compartilhamento eficiente do espaço de memória entre diferentes usuários (Reentrância), apresentar altas taxas de acesso a baixo custo de armazenamento por *bit* e otimizar acesso ao banco de dados.

A reentrância consiste na capacidade de um código de programa (código reentrante) ser compartilhado, na memória, por diversos usuários. Assim, existe apenas uma cópia carregada na memória. Logicamente, esse código não pode ser alterado e cada usuário pode estar em um ponto distinto do programa, manipulando seus próprios dados.

3.2 O Gerenciador de Buffer

O gerenciador de *buffer* funciona com base no seguinte princípio: toda informação armazenada em M_1, em qualquer instante, também está armazenada em M_2. Contudo, o inverso não é verdade. O SGBD comunica-se diretamente com M_1 e M_1 comunica-se com M_2. Caso a informação desejada pelo SGBD não se encontre em M_1, o gerenciador de *buffer* carrega de M_2 para M_1 a informação requerida. Dessa forma, um gerenciador de *buffer* eficiente é aquele em que a Informação desejada deve ser encontrada em M_1 com uma taxa de frequência ótima.

O desempenho (*performance*) de um gerenciador de *buffer* é medida com base na sua taxa de acerto (H) (ou *buffer cache hit ratio*). A taxa de acerto é definida pela probabilidade que os endereços gerados pelo SGBD refiram-se a informação já alocada em M_1.

A taxa de acerto é determinada experimentalmente, como descrito a seguir: um conjunto de programas (consultas) é executado. Os números de referências de endereços satisfeitas por M_1 e M_2 são registrados (contabilizados). Essas quantidades são denotadas por N_1 e N_2. Assim, N_1 é a quantidade de vezes que a informação (bloco/página) desejada já se encontrava em M_1, enquanto N_2 é a quantidade de vezes que a informação (bloco/página) desejada teve que ser

levada (copiada) do disco (M_2) para o *buffer* (M_1).

A taxa de acerto é calculada aplicando-se a fórmula:

$$H= N_1 / (N_1+N_2)$$

Assim, a taxa de acerto ótima deve tender a 1. Já taxa de não acerto é definida por 1-H.

Na prática, os SGBDs buscam atingir taxas de acertos maiores que 95%.

Considere que T_{A1} e T_{A2} sejam os tempos de acesso de M_1 e M_2, respectivamente.

O tempo médio para o SGBD acessar um dado no sistema de memória é dado por:

$$T_A = H.T_{A1} + (1-H).T_{A2} \quad (1)$$

Caso um determinado bloco (página) não seja encontrado em M_1, uma página do banco de dados (em disco) contendo a palavra é transferida para M_1. Seja T_B o tempo de transferência de uma página do disco para o *buffer*, temos que:

$$T_{A2} = T_B + T_{A1} \quad (2)$$

Assim, substituindo (2) em (1), temos:

$$T_A = T_{A1} + (1-H).T_B$$

A razão (r) do tempo de acesso de um sistema de memória virtual com dois níveis é definida como:

$$r= T_{A2} / T_{A1}$$

A eficiência de acesso (e) de um sistema de memória virtual é determinada pela equação:

$$e= 1 / [r+(1-r).H]$$

O objetivo é obter valores para e próximos a 1. Para isso é necessário que H seja próximo a 1.

3.3 Mecanismo de Paginação em SGBDs

A operação mais importante em qualquer sistema de memória virtual é a troca (transferência) de blocos de informação entre os níveis de memória. Essa operação é denominada paginação (ou *swapping*). Essa troca ocorre de acordo com a demanda de processamento, ou seja, de acordo com a ordem e frequência com que as informações são solicitadas.

Por Demanda de Paginação entende-se a propriedade que indica quando uma operação de paginação deve ocorrer, ou seja, quando a paginação é necessária.

A Política de Alocação diz respeito ao método utilizado para determinar o local (endereço) da memória principal (mais precisamente, do *buffer*) onde uma cópia de uma determinada página do disco deve ser carregada.

Existem dois tipos de alocação: Alocação Estática e Alocação Dinâmica. Na alocação estática, cada página de dados está ligada (associada) a uma região fixa da memória principal (*buffer*). Já na alocação dinâmica, a região de memória principal (*buffer*) na qual uma página K é carregada pode variar. A alocação dinâmica pode ser ainda classificada em preemptiva e não preemptiva.

Na alocação dinâmica não preemptiva, a página K a ser carregada só pode ser alocada em um espaço vazio da memória principal (*buffer*), enquanto na alocação dinâmica preemptiva a página a ser carregada pode ser alocada em um espaço ocupado por outra página J. Nesse caso, a página J deve ser realocada em outro endereço ou removida totalmente da memória (e copiada para disco).

3.4 Políticas de Alocação de Páginas

Quando não há mais lugar disponível no *buffer*, um bloco deve ser removido do *buffer* antes que um novo bloco possa ser copiado para ele. O objetivo principal de uma política (ou estratégia) de substituição de páginas (blocos) em um *buffer* é minimizar os acessos a disco.

As principais políticas de alocação dinâmica preemptiva são:

- **FIFO (*First-In First-Out*):** A página a ser carregada é alocada na região de *buffer* ocupada pela página carregada menos recentemente no *buffer*.
- **LRU (*Least Recently Used*):** A página a ser carregada é alocada na região de *buffer* ocupada pela página acessada mais remotamente (ou seja, há mais tempo).
- **MRU (*Most Recently Used*)**: A página a ser carregada é alocada na região de *buffer* ocupada pela página acessada mais recentemente.

Para programas de propósito geral, como um sistema operacional, por exemplo, não é possível prever precisamente quais blocos serão referenciados (requisitados). Por isso, tais sistemas, em geral, utilizam o padrão de referências (requisições) passadas como uma forma de prever as referências futuras. Entretanto, um sistema de banco de dados é capaz de prever o modelo de futuras referências de forma mais precisa que um sistema operacional. Em geral,

a solicitação de um usuário ao sistema de banco de dados envolve diversos passos. O sistema de banco de dados é capaz de determinar antecipadamente quais blocos serão requisitados por meio da verificação de cada um dos passos necessários para executar a operação solicitada pelo usuário. Assim, diferentemente dos sistemas operacionais, que devem se basear no passado para prever o futuro, os sistemas de bancos de dados podem possuir e utilizar informações relativas ao futuro próximo.

Os SGBDs costumam utilizar os algoritmos LRU (*Least Recently Used*) ou Clock2 (*Two Round Clock*) para gerenciar o *buffer* na memória principal.

Contudo, o esquema LRU, em banco de dados, é mais elaborado que em sistemas operacionais, pois o sistema de banco de dados é capaz de determinar antecipadamente quais blocos serão necessários por meio da verificação de cada um dos passos necessários para executar a operação solicitada pelo usuário. Para ilustrar, consideremos a seguinte expressão da álgebra relacional: Departamento Empregado

Considere que a tabela Departamento possui os atributos DEP_CODIGO e DEP_NOME. Assuma ainda que a tabela Empregado possui os atributos EMP_CPF, EMP_NOME e DEP_CODIGO (chave estrangeira utilizada para representar em que departamento um empregado trabalha).

Suponha que o algoritmo de junção a ser utilizado é o *Nested-Loop Join (*mostrado a seguir*)*. Assuma também que as duas relações desse exemplo sejam armazenadas em arquivos separados.

```
Para cada tupla d de departamento faça
        Para cada tupla e de empregado faça
                Se d[EMP_CODIGO] = e[EMP_CODIGO] então
                Início
                        Seja x definida da seguinte forma:
                        x[DEP_CODIGO]:= d[DEP_CODIGO]
                        x[DEP_NOME]:= d[DEP_NOME]
                        x[EMP_CPF]:= e[EMP_CPF]
                        x[EMP_NOME]:= e[EMP_NOME]
                        Inclua a tupla x como parte do resultado de departamento
x empregado
                Fim
        Fim
Fim
```

Vamos mostrar a seguir que as políticas LRU e Clock2 nem sempre constituem a melhor estratégia para substituição de páginas em um sistema de banco de dados. Um dos principais cenários que evidenciam esse fato ocorre quando vários usuários requisitam uma leitura sequencial (*scan*) de um conjunto de dados (uma tabela, por exemplo).

Considere, inicialmente, a seguinte situação. Suponha que o *buffer pool* possua 10 páginas, e que o arquivo a ser lido (tabela empregado, por exemplo) contém nove páginas. Assuma, por simplicidade, que não existe concorrência na requisição das páginas. Assim, durante o primeiro *scan* sobre o arquivo todas as páginas serão levadas do disco para o *buffer pool* (executa operações de I/O). As operações de *scan* subsequentes irão sempre encontrar a página desejada no *buffer*. Nesse caso, o SGBD terá uma taxa de acerto de 0% na primeira operação de varredura (*scan*) e uma taxa de acerto de 100% nas varreduras subsequentes.

Por outro lado, suponha que o arquivo (tabela empregado, por exemplo), a ser lido sequencialmente, possui 11 páginas. Nesse caso, o número de páginas do arquivo a ser lido é maior que o número de páginas disponíveis no *buffer* (ou seja, maior que o tamanho total do *buffer*). Logo, o arquivo não poderá ser carregado integralmente na memória. Dessa forma, observe que durante a execução da primeira operação de varredura (*scan*), quando a página 10 do arquivo for transferida para o *buffer*, esse ficará com todas as suas páginas ocupadas (uma vez que o tamanho do *buffer* é de 10 páginas ou blocos), ou seja, o *buffer* estará completamente preenchido (sem nenhum espaço livre). Logo, quando a página 11 do arquivo for requisitada, uma das páginas presentes no *buffer* deverá ser escolhida para sair do *buffer*, a fim de liberar espaço para que a página 11 seja copiada do disco para o *buffer*. Que página será escolhida para sair do *buffer*? Pelo algoritmo LRU a página escolhida será aquela menos recentemente utilizada, que nesse caso será a página 1 (primeira página a ser carregada na memória, ou seja, no *buffer*). Assim, a página 1 será retirada da memória. Em seguida, uma cópia da página 11 será alocada no espaço do *buffer* anteriormente ocupado pela página 1. Consequentemente, o *buffer* ficará preenchido com as páginas de 2 a 11 do arquivo lido (tabela empregado).

Agora, considere que terá início a segunda execução da varredura (*scan*) sobre a tabela empregado. Nesse caso, qual a primeira página a ser requisitada por essa operação de varredura (*scan*)? Como a operação de varredura é uma operação sequencial, a primeira página requisitada será a página 1 (do arquivo que contém as tuplas da tabela empregado). Porém, a página 1 não está no

buffer. Logo, uma das páginas presentes no *buffer* deve ser selecionada para sair do *buffer*, liberando espaço para que a página 1 possa ser copiada para o *buffer*. Que página será escolhida para deixar o *buffer*? Segundo o algoritmo LRU, a página menos recentemente utilizada. Ora, no *buffer* estão as páginas de 2 a 11 do arquivo lido (tabela empregado). A página menos recentemente utilizada é a página 2. Logo, a página 2 será escolhida para deixar o *buffer*. Em seu lugar será carregada uma cópia da página 1. Nesse instante estão no *buffer* as páginas: 1, 3, 4, 5, 6, 7, 8, 9, 10 e 11. Qual próxima página a ser requisitada pela operação de varredura? A próxima página requisitada será a página 2. Porém, a página 2 não se encontra no *buffer*. Qual a página que será escolhida para deixar o *buffer* pelo algoritmo LRU? A página 3. Qual a próxima página a requisitada pela operação de varredura? A página 3. Observe que esse comportamento se mantém até a leitura da última página (página 11) pela operação de varredura. Portanto, nesse exemplo, todas as execuções da operação de varredura irão apresentar uma taxa de acerto de 0%.

Assim, usando LRU, todas as operações de *scan* (sobre o arquivo) terão que ler todas as páginas do arquivo do disco e levar para o *buffer pool* (operação de I/O). Nessa situação, chamada "*sequential flooding*", LRU é a pior estratégia para substituição de páginas.

Podemos observar, para esse exemplo, que uma vez que uma *tupla* de empregado tenha sido processada, a *tupla* não será necessária novamente. Portanto, uma vez que o processamento de um bloco inteiro de *tuplas* de empregado seja completado, esse bloco não é mais necessário na memória. Entretanto, como ele foi usado recentemente ele ainda permanecerá no *buffer* por um longo período de tempo. O gerenciador de *buffer* deveria ser instruído a liberar o espaço ocupado por um bloco de empregado assim que a última *tupla* tenha sido processada. Essa estratégia de gerenciamento de *buffer* é chamada de arremesso imediato.

Agora considere o algoritmo de junção *Nested Loop Join*. Cada bloco de *tuplas* de empregado precisa ser lido uma vez para cada *tupla* da relação departamento. Assim, quando o processamento de um bloco de *tuplas* de empregado é concluído, esse não será mais necessário até o processamento da próxima *tupla* de departamento, antes disso todos os demais blocos de empregado deverão ser lidos novamente. Entretanto, como esse bloco foi usado recentemente ele ainda permanecerá no *buffer* por certo período de tempo, o que é inteiramente desnecessário.

Na verdade, a estratégia ótima (melhor alternativa) para a substituição de páginas nesse exemplo é a MRU.

Nesse caso, o sistema deve fixar um bloco do arquivo contendo as tuplas da relação empregado sendo correntemente processado. Assim que a última *tupla* desse bloco tiver sido processada, o bloco é liberado e ele se tornará o bloco mais recentemente utilizado. Sendo, portanto, o próximo a ser retirado do *buffer*. De forma similar, esse mecanismo se aplica aos blocos da relação departamento. Surpreendentemente, a estratégia mais utilizada em sistemas de banco de dados é a LRU.

A política FIFO também possui situações nas quais não proporciona bons resultados. Considere um determinado bloco r usado repetidamente, por exemplo, o bloco raiz de um índice i de uma árvore B+. Assuma que uma busca na árvore esteja sendo realizada. O primeiro bloco da árvore a ser lido será a raiz da árvore, bloco r. Assuma agora, que ao percorrer a árvore, os blocos que representam os nós da árvore que foram lidos são transferidos para o *buffer*. Assuma também que durante essa operação o *buffer* seja completamente ocupado (cheio). Considere agora que é necessário ler mais um nó da árvore, ou seja, o bloco que representa esse nó da árvore precisa ser copiado do disco para o *buffer*. Nesse caso, um dos blocos presentes no *buffer* deverá ser escolhido para deixar o *buffer*. Que bloco será selecionado para deixar o *buffer*? Segundo o algoritmo FIFO, o primeiro a ser carregado. Nesse caso, a raiz da árvore, bloco r.

Entretanto, estruturas de índices são usadas com bastante frequência. Considere que uma nova busca precise ser realizada sobre a estrutura de índices i. Nesse caso, qual a primeira página (bloco) do índice i (Árvore B+) a ser lida? A raiz da árvore, bloco r. Porém, o bloco r não se encontra mais no *buffer*, e sim em disco. Logo, o bloco r deverá ser novamente copiado do disco para o *buffer*.

Uma solução para esse problema é fixar a raiz, ou seja, indicar que o bloco r não pode ser removido do *buffer*. Esses blocos são ditos “fixados”. Essa solução é comumente aplicada para blocos que se constituem em “*hot spots*”, ou seja, blocos acessados frequentemente.

A estratégia ideal de substituição de páginas em bancos de dados requer o conhecimento das operações de banco de dados em processamento. Na prática, não se conhece uma única estratégia que manipule bem todos os possíveis cenários. De fato, um grande número de SGBDs utiliza o LRU, apesar das situações em que essa estratégia não proporciona bons resultados.

Vale destacar que a estratégia utilizada por um gerenciador de *buffer* para a substituição de blocos é influenciada por outros fatores além do tempo no qual o bloco foi carregado para o *buffer* (como no caso do FIFO) ou referenciado (como no caso do LRU e MRU). Se o sistema está processando solicitações de vários usuários concorrentemente, o subsistema de controle de concorrência pode ter de atrasar algumas solicitações para assegurar a preservação da consistência do banco de dados. Se o gerenciador de *buffer* receber informações do subsistema de controle de concorrência indicando quais solicitações estão sendo atrasadas, ele pode usar essas informações para alterar sua estratégia de substituição de blocos. Especificamente, os blocos necessários para as solicitações ativas (não atrasadas) podem ser retidos no *buffer*, enquanto os blocos relacionados às solicitações atrasadas podem ser bons candidatos para deixar o *buffer*.

O subsistema de recuperação de falhas também impõe restrições rigorosas à substituição de blocos. Se um determinado bloco b tiver sido modificado (alterado), o gerenciador de *buffer* não tem permissão de escrever de volta no disco a nova versão do bloco b (presente no *buffer)*, uma vez que isso destruiria a versão antiga do bloco b (armazenada em disco). Em vez disso, o gerenciador de *buffer* deve obter permissão do subsistema de recuperação de falhas antes de escrever o bloco b de volta no disco. O subsistema de recuperação de falhas pode exigir (para garantir a consistência dos dados) que determinados blocos (por exemplo, os blocos x e y) tenham que sair do *buffer* antes da saída do bloco b. Assim, o subsistema de recuperação de falhas só dará a permissão para que o bloco b saia do *buffer* após a saída dos blocos x e y.

O gerenciador de *buffer,* além de utilizar os conhecimentos acerca da solicitação que está sendo processada, pode utilizar também informações estatísticas relativas à probabilidade de acesso a determinadas páginas (blocos). O dicionário de dados, por exemplo, é uma das partes do sistema de banco de dados mais frequentemente acessadas. Logo, o gerenciador de *buffer* não deveria tentar remover blocos do dicionário de dados do *buffer* (ou seja, da memória principal) a não ser que outros fatores determinem que isso seja feito. De forma similar, como as estruturas de índice podem ser acessadas mais frequentemente que as próprias páginas de dados (blocos que armazenam as tuplas das relações), o gerenciador de *buffer*, em geral, não deveria remover os blocos de índices da memória principal (*buffer*), a menos que não hajam outras alternativas.

Exercícios

1. Considere um mecanismo de *buffer* com dois níveis. A área de *buffer* em memória principal M_1 apresenta tempo de acesso igual a 10^{-7}s e taxa de acerto igual a 0.7. A memória M_2 está montada através de uma unidade de disco com as seguintes características:
   (a) Palavra de 64 *bits*
   (b) Velocidade de rotação do disco: 5400 r/min
   (c) Capacidade de armazenamento de cada trilha: 64000 *bits*
   (d) Capacidade de cada setor de uma trilha: 64 *bits*

   Suponha que o sistema apresenta um barramento com velocidade de transmissão extremamente alta. Portanto, o tempo gasto para transferir dados através do barramento da unidade de disco até a memória principal é desprezível. Considere o tempo de procura igual a 10^{-6}s. Calcule a eficiência de acesso desse sistema e indique se esse valor está próximo ou não do valor ideal. Justifique sua resposta.

UNIDADE 2

PROCESSAMENTO DE TRANSAÇÕES

Resumindo

Esta unidade discute em detalhes o problema do processamento de transações em bancos de dados. Define, precisamente, os conceitos do modelo convencional de transações. Discute e examina noções de transação e schedules. O critério de corretude convencional, denominado de serializabilidade, é descrito. Vantagens e desvantagens desse modelo são apontadas e discutidas. Aborda também a relação entre corretude e confiabilidade de schedules.

2 PROCESSAMENTO DE TRANSAÇÕES

1. INTRODUÇÃO

Após o acompanhamento desta lição o aluno será capaz de entender como tratar as questões referentes a transações, tipos de isolamento e modos de bloqueios.

1.1 O Problema da Concorrência em BDs

SGBDs são sistemas multiusuários. Dessa forma, a informação contida nele pode ser acessada e modificada de maneira concorrente. Essa situação pode levar o banco de dados a um estado inconsistente (por exemplo, redundância de dados).

Para que possamos entender melhor essa situação, vamos definir que todo banco de dados em seu estado inicial, está em um estado consistente, ou seja, todas as suas restrições de consistência estão preservadas.

A mudança de estado (Figura 1) do Banco de Dados (BD) se dá no instante em que os dados do mesmo sofrem alguma alteração. Sendo assim, as operações de inclusão, exclusão e alteração de dados, alteram o estado do BD e é necessário que qualquer alteração de estado do BD, leve o mesmo a um estado sempre consistente.

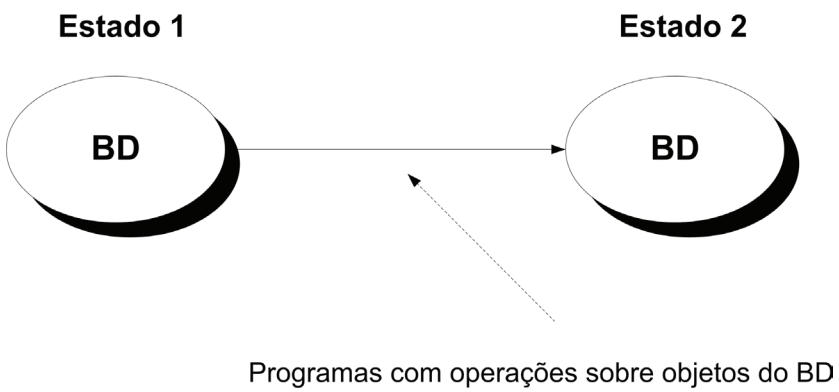


Figura 1. Mudança de estados do BD

O SGBD deve oferecer mecanismos para esse controle sem prejudicar o acesso concorrente aos dados, pois um SGBD multiusuário tem que permitir o acesso simultâneo de vários usuários à base de dados. O SGBD deve oferecer um controle de concorrência para garantir que o resultado de várias modificações à base de dados seja correto, como no caso de uma aplicação que controla reservas de voos.

É dessa necessidade que surge o princípio do controle de concorrência que limita as leituras e modificações simultâneas disparadas ao mesmo dado por diferentes usuários.

A ideia de acesso concorrente ou multiusuário considera que operações de um programa podem ser executadas entre duas operações de outro programa. Como vimos, essas operações causam transições de estado no BD e caso essas transições de estado gerem estados inconsistentes do BD, então elas são ditas transições incorretas (Figura 2).

Portanto, o estado de um banco de dados representa uma visão instantânea do mundo, refletindo apenas seus aspectos estáticos. Entretanto, as mudanças que ocorrem em um mundo real devem ser refletidas no banco de dados. Essas mudanças são capturadas pelo conceito de transição de estados. Uma transição de estado representa o salto de um determinado estado para outro. Essas transições são realizadas pelos programas aplicativos que contêm operações de leitura e de escrita sobre os objetos do banco de dados.

A Figura 2 apresenta o caso de transições incorretas. Acompanhando a execução de acordo com a linha do tempo (seta verde, de cima para baixo), vemos que o programa 1 inicia sua execução realizando duas leituras de dados (*select*) e uma atualização (*update*) antes do programa 2 iniciar suas operações. Quando o programa 2 realiza suas consultas à base de dados, verificando os saldos das contas 12 e 13, respectivamente, nesse instante o programa 1 ainda não alterou o valor do saldo da conta 13. Dessa forma, quando o programa 2 mostrar os valores dos saldos, a atualização feita pelo programa 1 no saldo da conta 13 não aparecerá, provocando uma inconsistência no resultado da consulta.

À medida que mais usuários começam a compartilhar os mesmos dados, aqueles sistemas que não tiveram um planejamento adequado estão sujeitos a apresentar problemas de desempenho, *deadlocks* e bloqueios que diminuem a concorrência no acesso aos dados.

Programa-1	Programa-2
Select saldo into: valor1 From t_conta where num_conta = 12 Select saldo into: valor2 From t_conta where num_conta = 13 valor1 = valor1 + 500 update t_conta set saldo = :valor1 where num_conta = 12	
Valor2 = valor2 +300	Select saldo into :valor1 From t_conta where num_conta = 12 Select saldo into :valor2 From t_conta where num_conta = 13
update t_conta set saldo = :valor2 where num_conta = 13	
	Display (Valor1, Valor2)

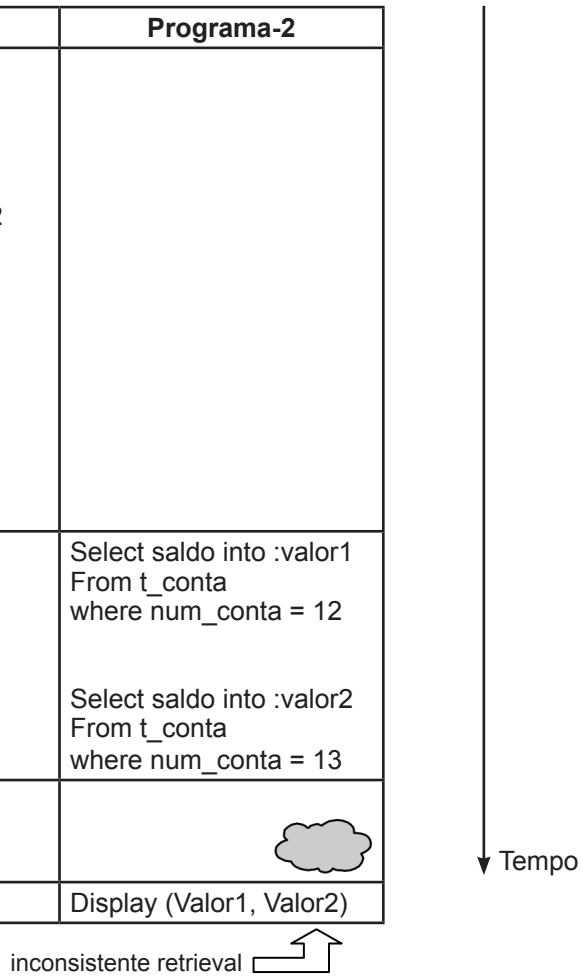


Figura 2. Transições Incorretas

Esse é um problema comum em SGBDs. No decorrer desse capítulo, veremos mais exemplos como esse, bem como a forma existente para tratá-lo.

Para entender o conceito de concorrência em banco de dados, faz-se necessário entender, primeiramente, o conceito de transação.

1.2 Conceito de transação

Transação pode ser entendida como uma “abstração que representa a sequência de operações de bancos de dados resultantes da execução de um programa”. A transação leva em conta apenas as operações de leitura (*read*) e escrita (*write*) que estão presentes nela. Como pode ser visto na Figura 3, o código executado pelo programa-1 é executado em uma transação (T_1) e para essa transação, apenas os acessos ao Banco de Dados serão contabilizados.

Programa-1
Select saldo into :valor1 From t_conta where num_conta = 12 valor1 = valor1 + 500 update t_conta set saldo = :valor1 where num_conta = 12

Figura 3. Código de uma transação

Dessa forma, a expressão que soma 500 à variável *valor1* é desconsiderada pela transação, sendo que apenas as operações de *select* (*read*) e *update* (*write*) serão consideradas como operações da transação T_1. Como forma de simplificação abreviamos a leitura (read) para **r** e a escrita (*write*) para **w**, ambos indexados pelo número da transação, nesse caso T_1. Sendo assim, a sequência de operações executadas na Figura 3, passa a ser representada da seguinte forma:

$$T_1 = r_1(conta_12)\ w_1(conta_12)$$

Como visto, **transação** pode ser entendida como uma série de operações sobre itens no banco de dados que podem alterar o seu estado. As operações são delimitadas com uma instrução de início da transação (*Begin Transaction)* e no final com uma instrução de fim da transação – *Commit* para efetivar as alterações, ou uma instrução *Rollback* quando for necessário descartar todas as alterações realizadas no banco de dados.

Para alguns renomados autores, transação é:

- uma unidade lógica de processamento no BD (Navathe)
- uma unidade de execução do programa que acessa e possivelmente atualiza vários itens de dados (Silberschatz)

Possíveis operações dentro de um banco de dados:

- BEGIN_TRANSACTION – Início da execução de uma transação
- END_TRANSACTION – Fim da Transação
- READ(x) ou WRITE(x) – Operações de leitura de um dado “x” e escrita do dado “x” em um banco de dados
- COMMIT – Transação completou sua execução com sucesso
- ROLLBACK (ou ABORT) – Transação foi abortada, não terminou com sucesso e as mudanças realizadas devem ser desfeitas.

Esse conjunto de operações compreendidos dentro da transação é considerado uma unidade lógica de trabalho, desde que possua um conjunto de propriedades conhecido pelo acrônimo de ACID, descrito mais à frente neste capítulo.

1.3 Estados de uma transação

Uma transação possui diferentes estados, conforme mostra a Figura 4.

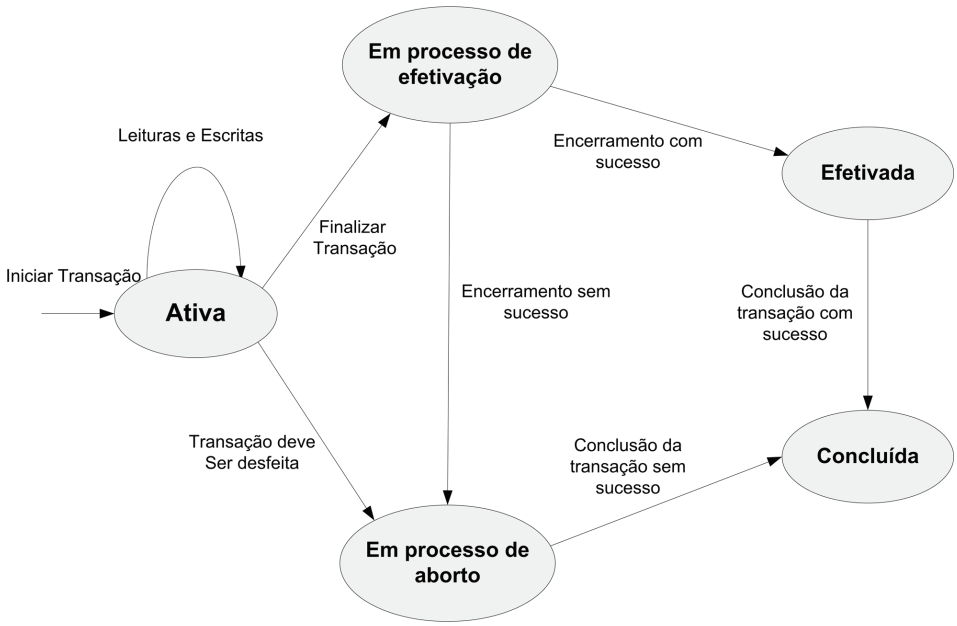


Figura 4. Estados de uma transação

- **Ativa**
  - Estado inicial de toda transação selecionada para execução;
  - Enquanto ativa, uma transação executa uma ou mais operações leitura e escrita.
- **Em processo de efetivação**
  - Entra nesse estado após executar sua última operação (solicitação de COMMIT);
  - Nesse momento, o SGBD precisa garantir que as suas atualizações sejam efetivadas com sucesso (não sofra falhas);
    - Aplicação de técnicas de *recovery*.
- **Efetivada**
  - Entra nesse estado após o SGBD confirmar que todas as modificações da transação estão garantidas no BD (COMMIT OK).
    - Exemplos: gravação em *Log*, descarga de todos os *buffers* em disco.
- **Em processo de aborto**
  - Entra nesse estado se não puder prosseguir a sua execução;
  - Pode passar para esse estado enquanto ativa (I) ou em processo de efetivação (II)

        - Exemplo (I): violação de RI
        - Exemplo (II): pane no S.O;
            - suas ações já realizadas devem ser desfeitas (ROLLBACK).
- **Concluída**
    - Estado final de uma transação;
    - Indica uma transação que deixa o sistema;
        - As informações da transação mantidas em catálogo podem ser excluídas;
            - Operações feitas, dados manipulados, *buffer*s utilizados;
        - Se a transação não concluiu com sucesso, ela pode ser reiniciada automaticamente.

1.4 Propriedades da transação

Nos bancos de dados que trabalham com transações de curta duração, como é o caso dos bancos de Dados Relacionais (BDR), é essencial que sejam respeitadas as propriedades de ACID (Atomicidade, Consistência, Isolamento, Durabilidade), descritas a seguir:

- **Atomicidade**: "tudo ou nada". Todas as ações da transação acontecem ou nenhuma pode acontecer;
- **Consistência**: se toda a transação é consistente, e o BD inicia consistente, então o BD termina consistente;
- **Isolamento**: a execução de uma transação é isolada de outras transações, ou seja, as alterações realizadas por uma transação não serão visualizadas pelas outras transações até que elas sejam efetivamente atualizadas (*commit*);
- **Durabilidade**: se uma transação é finalizada, seu efeito persiste.

Exemplo bancário – considere a seguinte transação: Suponha que T_2 seja uma transação que transfere R$ 100,00 da conta X para a conta Y.

T_2:
```
read (X);
X := X – 100;
write (X);
read (Y);
Y := Y + 100;
write (Y)
```

Analisando cada uma das propriedades ACID no exemplo bancário:

- Atomicidade
    - Suponha os valores X = R$ 1.000 e Y = R$ 2.000
    - Suponha uma falha no meio da transação
        - R$ 900,00 e R$ 2.000,00 (somente a retirada foi

efetuada)

- A atomicidade garante que todas as operações serão efetuadas ou nenhuma delas será. Para essa suposta falha, a operação de retirada é desfeita retornando o saldo da conta X para R$ 1.000

- Consistência
  - Se os dados estiverem consistentes antes da transação, elas devem permanecer consistentes depois da transação também.
- Isolamento
  - Operações não podem se intercalar, gerando um estado inconsistente.
  - Mesmo que uma operação leia informações da outra, a consistência deve ser garantida.
- Durabilidade
  - Quando uma transação executou com sucesso, é preciso garantir que nenhuma falha resulte na perda dos dados já armazenados.

Exercícios

1. Conceitue transação em um banco de dados?

2. Quais os estados de uma transação?

3. Defina as propriedades ACID.

2. EXECUÇÕES CONCORRENTES

2.1 Acessos concorrentes

Os acessos concorrentes devem ser tratados pelos bancos de dados para garantir a integridade das alterações realizadas nas informações.

Em um mesmo momento diversos usuários podem estar realizando as mais variadas atualizações no mesmo banco de dados, e em alguns casos alterando as mesmas informações. Para solucionar esse problema o banco de dados possui diferentes mecanismos, possibilitando ao desenvolvedor a escolha da melhor forma para a aplicação tratar acessos concorrentes à mesma informação.

Programa-1	Programa-2
Select saldo into :valor From tab_conta where num_conta = 12	
valor = valor +500	Select saldo into :valor From tab_conta where num_conta = 12
update tab_conta set saldo = :valor where num_conta = 12	valor = valor – 100
	update tab_conta set saldo = :valor where num_conta = 12
Saldo inicial (conta 12) = 400 Saldo final correto = 800 Saldo produzido no BD = 300	

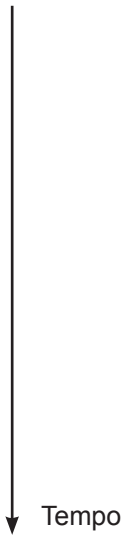


Figura 5. Perda de atualização

Na Figura 5 é apresentado um erro provocado pelo acesso concorrente ao banco de dados, a perda de atualização ou *lost update*. Esse problema ocorre com bastante frequência e se não for tratado devidamente pode trazer sérios problemas de inconsistência para os dados. Nesse caso, ocorreu que a atualização (*update* da conta 12) do programa-1 foi sobreposta pela atualização (*update* da conta 12) do programa-2, perdendo a adição de 500 ao valor inicial. Ou seja, o banco de dados foi levado de um estado inicial consistente a um novo estado, agora inconsistente.

Quando várias transações executam simultaneamente, a propriedade de ISOLAMENTO pode não ser mais preservada, pode haver interferências. Um modo de evitar o problema de transações executando simultaneamente é executar transações de modo serial, uma após a outra.

Para garantir que ela seja PRESERVADA, o sistema precisa controlar a interação entre as transações simultâneas. Daí a necessidade de se fazer o CONTROLE DE CONCORRÊNCIA.

2.2 Controle de Concorrência

É o gerenciamento das operações concorrentes no BD, garantindo o isolamento das transações executadas concorrentemente. Permite que vários usuários acessem simultaneamente os dados compartilhados. Esses usuários entram em concorrência quando tentam acessar os mesmos dados ao mesmo tempo.

Porque fazer o controle de concorrência:

- Melhor vazão (*throughput)* e utilização dos recursos

- Transação consiste de muitas etapas como: atividades de E/S, utilização da CPU que podem ser executadas em paralelo. Tudo isso aumenta o *throughput*, isto é, o número de transações executadas em determinada quantidade de tempo e a utilização dos recursos.

- Tempo de espera reduzido
  - Pode haver transações curtas e longas. Se a execução for serial, uma transação curta pode esperar muito tempo até uma que transação longa termine. Se elas forem executadas simultaneamente, compartilhando os ciclos da CPU, o tempo médio de resposta será reduzido.

No SGBD, o responsável pelo controle de concorrência é o escalonador (*Scheduler*). Ele é o responsável por fazer o entrelaçamento (*interleaving*) de operações. Por entrelaçamento, entenda que operações de um programa podem ser executadas entre duas operações de outro programa.

O *Schedule* (história) representa a ordem cronológica em que as instruções são executadas no sistema. O escalonador (*Schedule*r) define uma sequência de operações de um conjunto de transações concorrentes, preservando a ordem das operações em cada uma das transações.

Escalonador serializável: As operações de cada transação são executadas de maneira consecutiva sem nenhuma intercalação nas operações (Figura 6).

Solução ineficiente!!!

- Várias transações podem esperar muito tempo para serem executadas.
- A CPU pode ficar muito tempo ociosa.
  - Enquanto uma transação faz I/O, por exemplo, outras transações poderiam ser executadas.

execução serial

T1	T2
read (X)	
X = X - 20	
write (X)	
read (Y)	
Y = Y + 20	
write (Y)	
	read (Y)
	X = X + 10
	write (X)

Figura 6. Escalonador serializável

Obs. 1: A execução em série não permite nunca que um BD fique em um estado inconsistente.
Obs. 2: Planos seriais desperdiçam tempo da CPU e são inaceitáveis na prática.

Escalonador não serializável: as operações de um conjunto de transações são executadas de maneira intercaladas.

- escalonamento (*Schedule*) não serial e íntegro.

execução não-serial ou concorrente

T1	T2
read (X)	
X = X - 20	
write (X)	
	read (X)
	X = X + 10
	write (X)
read (Y)	
Y = Y + 20	
write (Y)	

Figura 7. Escalonador não serializável

O objetivo da capacidade de serialização é de encontrar escalonadores não seriais que permitam uma execução concorrente, sem interferir uma transação na outra e consequentemente, sem gerar um estado inconsistente.

Execução concorrente de transações implica em um entrelaçamento das operações dessas transações. Sendo assim, uma execução correta das transações depende do entrelaçamento correto de suas operações (Figura 6).

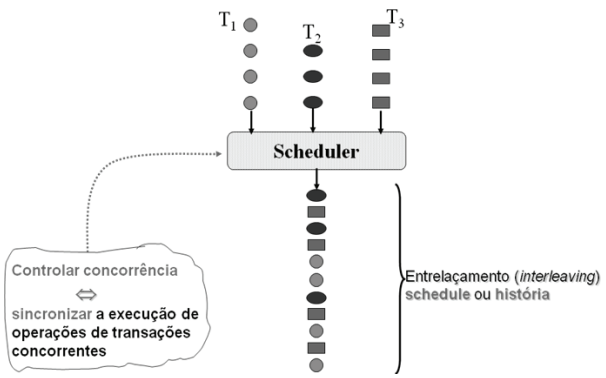


Figura 8. Entrelaçamento de operações.

Quando um entrelaçamento está correto? Dizemos que uma execução correta produz um estado consistente no BD.

Problemas de um escalonamento não serial mal definido (inválido):

- atualização perdida (*lost-update*)

Uma transação T_y grava em um dado atualizado por uma transação T_x

T1	T2
read (X)	
X = X - 20	
	read (Z)
	X = Z + 10
write (X)	
read (Y)	
	write (X) ← a atualização de X por T1 foi perdida!
Y = Y + 30	
write (Y)	

Figura 7. Escalonador não serializável

- leitura suja (*dirty-read*)

T_x atualiza um dado X, outras transações posteriormente leem X, e depois T_x falha (Figura 8).

T1	T2
read (X)	
X = X - 20	
write (X)	
	read (X) ← T2 leu um valor de X que não será mais válido!
	X = X + 10
	write (X)
read (Y)	
abort (Y)	

Figura 8. Exemplo leitura suja

Exercícios

1. O que é uma transação?
   Uma sequência de ações que são consideradas uma unidade atômica (indivisível) de trabalho.

2. O que é conflito entre transações?
   O conflito entre transações ocorre quando mais de uma transação tenta acessar o mesmo item de dado.

3. O que é mecanismo de controle de concorrência?
   É um mecanismo que garante a consistência e o isolamento dos dados, dada a atomicidade das transações.

4. Por que estudar mecanismos de controle de concorrência?
   - Trabalhamos em ambientes multitarefas;
   - É necessário compartilhar dados;
   - É necessário manter a consistência dos dados.

3. ISOLAMENTO DE TRANSAÇÕES

3.1 Transações Sequenciais

Conforme vimos anteriormente, transações executadas de forma sequencial – uma seguida da outra – são ditas transações corretas. Entretanto, a execução de *schedules* seriais limita o acesso concorrente aos dados e, por consequência, diminui o *throughput* (quantidade de trabalho realizado em um intervalo de tempo).

O ideal seria permitir a execução concorrente de transações, mas de tal forma que resultados consistentes sejam produzidos no banco de dados.
Podemos dizer, que temos:

- Premissa
  - A execução de uma transação é correta, se executada isoladamente. Pois, produz sempre um estado consistente.
- Teorema
  - Toda execução serial de transações é correta

$$S=T_1 T_2 T_3 \ldots T_{n-1} T_n$$

Este é o padrão (conceito básico) para corretude de schedules!!!

A Figura 9 apresenta o exemplo da execução serial de transações. Primeiramente é executada toda a transação T1 e só depois é executada a transação T2.

T1	T2
Select saldo into :valor1 From t_conta where num_conta = 12 Select saldo into :valor2 From t_conta where num_conta = 13 valor1 = valor1 + 500 update t_conta set saldo = :valor1 where num_conta = 12 Valor2 = valor2 +300 update t_conta set saldo = :valor2 where num_conta = 13	
	Select saldo into :valor1 From t_conta where num_conta = 12 Select saldo into :valor2 From t_conta where num_conta = 13

Figura 9 – Execução serial das transações T1 e T2

Como vimos, apesar de correto esse método não é o desejável para um ambiente multiusuário. Precisamos de um critério de corretude (ou correção) que garanta mais paralelismo. Nesse caso, precisamos de transações executando concorrentemente e de maneira correta.

3.2 Execução correta de transações concorrentes

Partindo do teorema de que toda execução serial de transações é correta, temos que “um escalonamento não serial de um conjunto de transações deve produzir resultado **equivalente** a alguma execução **serial** dessas transações”. Logo:

- Seja um *schedule **S***
  - Se ***S*** produz um estado de banco de dados igual ao produzido por alguma execução serial do mesmo conjunto de transações, então a execução ***S*** é correta (Figura 10).

entrada:
X = 50
Y = 40

execução serial

saída:
X = 40
Y = 60

T1	T2
read (X)	
X = X - 20	
write (X)	
read (Y)	
Y = Y+ 20	
write (Y)	
	read (X)
	X = X + 10
	write (X)

entrada:
X = 50
Y = 40

execução não-serial ***serializável***

saída:
X = 40
Y = 60

T1	T2
read (X)	
X = X - 20	
write (X)	
	read (X)
	X = X + 10
	write (X)
read (Y)	
Y = Y+ 20	
write (Y)	

Figura 10 – Execução serial e concorrente das transações T1 e T2.

Logo, escalonamento serializável é equivalente à execução serial das mesmas transações.

Dessa premissa, surge o conceito de **SERIALIZABILIDADE**. Serializabilidade é a ilusão de que transações são executadas como ações atômicas (sem interferências ou *interleavings*).

- Equivalência de *schedules*
  - Dado um conjunto $\Im = \{T_1, T_2, ..., T_n\}$ de transações
    - Encontrar *schedules* cujas execuções produzam o mesmo estado no BD que a execução de algum *Schedule* serial sobre $\Im$.

Dessa forma, reduzimos o problema de identificar *schedules* corretos ao problema de definir equivalência entre *schedules*.

Existem três diferentes noções de equivalência que podem ser encontradas na literatura. São elas:

- Equivalência de estado final
- Equivalência de visão
- Equivalência de conflito

Para o estudo de equivalências, a notação $r_i(x)$ e $w_i(x)$ será utilizada para representar operações de leitura e de escrita executadas por uma transação *Ti* sobre um objeto x. OP(T_i) representa o conjunto de todas as operações executadas pela transação T_i. Além das operações de leitura e de escrita uma transação T_i deverá conter após todas as operações de leitura e de escrita uma operação *commit* (c_i) ou *abort* (a_i), mas não ambas.

A relação de ordem $<_i$ representa a precedência entre duas operações de uma transação T_i. Por exemplo, $r_i(x) <_i w_i(x)$ indica que a operação $r_i(x)$ ocorreu antes da operação $w_i(x)$.

3.2.1. Equivalência de estado final

- Dois *Schedules* sobre um conjunto $\Im = \{T_1, T_2, ..., T_n\}$ são equivalentes de estado final ($S \approx_{FS} S'$) se, e somente se,

  (i) têm as mesmas operações pertencentes às transações de $\Im$ e
  (ii) produzem o mesmo estado final no BD, se forem executados sobre um mesmo estado inicial

  - **PROBLEMA:** *Schedules* não capturam o conceito de estados inicial e final.

3.2.2 Equivalência de visão

- Dois *Schedules* ***S*** e ***S´*** sobre um conjunto $\Im = \{T_1, T_2, ..., T_n\}$ são equivalentes de visão ($S \approx_V S'$) se, e somente se,

  (i) têm as mesmas operações pertencentes a transações de $\Im$ e

  (ii) para cada operação $r_i(x)$ de $T_i \in \Im$, o valor lido por r_i é o mesmo nos dois *Schedule*s
  (iii) Se $w_i(x)$ é a última operação de escrita sobre x em ***S***, então $w_i(x)$ é a última operação de escrita sobre x em ***S´***

  - **PROBLEMA**: Controle sobre todas as operações de leitura e de escrita.

Para entendermos o próximo tipo de equivalência (equivalência em conflito), precisamos entender primeiro o conceito de operações em conflito.

- **Operações em conflito (conflitantes):** Duas operações estão em conflito se e somente se:
  1. São operações de transações distintas
  2. São executadas sobre um mesmo objeto e
  3. Pelo menos uma das duas operações é uma operação de escrita (*write*)

Considere o exemplo mostrado a seguir.

$T_1 = r_1(x) w_1(x)$

$T_2 = r_2(x) w_2(y)$

$S = r_2(x) r_1(x) w_2(y) w_1(x)$

OBS: Observe que as operações $r_2(x)$ e $w_1(x)$ estão em conflito, uma vez que pertencem a transações diferentes (a primeira operação, $r_2(x)$, pertence à transação T2, enquanto a segunda operação, $w_1(x)$, pertence à transação T1; as duas operações atuam sobre o mesmo objeto (objeto ou item x) e pelo menos uma das operações é uma operação de escrita (no caso desse exemplo, a segunda operação é uma operação de escrita).

3.2.3 Equivalência de conflito

- Dois *Schedules* ***S*** e ***S´***, definidos sobre um conjunto de transações $\Im=\{T_1, T_2, ..., T_n\}$, são equivalentes de conflito ou por conflito ($\boldsymbol{S} \approx_c \boldsymbol{S´}$) se, e somente se,
  (i) ***S*** e ***S´*** têm as mesmas operações pertencentes às transações de $\Im$ e
  (ii) ***S*** e ***S´*** ordenam (sequenciam) as operações em conflito da mesma forma (ou seja, dada duas operações que estejam em conflito no *Schedule* S essas operações devem aparecer na mesma ordem no *Schedule* S'. Assim:
  - Para qualquer duas operações em conflito $p_i \in OP(T_i)$ e $q_j \in OP(T_j)$, com $i \neq j$, se $\mathbf{p_i <_S q_j}$, então $\mathbf{p_i <_{S'} q_j}$

Considere o exemplo mostrado a seguir:

$T_1=r_1(y)r_1(x)w_1(y)$ $T_2=r_2(x)r_2(y)w_2(x)$

$S = r_1(y)r_1(x)r_2(x)w_1(y)r_2(y)w_2(x)$

$S' = r_1(y)r_1(x)w_1(y)r_2(x)r_2(y)w_2(x)$

Observe que:

Operações em conflito do *Schedule* S	Operações em conflito do *Schedule* S'
$r_1(x) <_S w_2(x)$ $w_1(y) <_S r_2(y)$	$r_1(x) <_S w_2(x)$ $w_1(y) <_S r_2(y)$

Analisando as operações em conflito dos dois *Schedule*s, verificamos que os dois *schedules* são equivalentes em conflito.

3.3 Serializabilidade

As três diferentes noções de equivalência de *Schedule*s estudadas anteriormente (equivalência por estado final, por visão e por conflito) implicam (dão origem) a três diferentes noções de corretude (correção) de *Schedule*s (Figura 10), as quais são detalhadas a seguir.

- **Serializabilidade por estado final** (FSR)
  - Um *schedule* S é serializável por estado final, se ele é equivalente de estado final a algum *Schedule* serial.
- **Serializabilidade por visão** (VSR)
  - Um *schedule* S é serializável por visão, se ele é equivalente de visão a algum *Schedule* serial.

- **Serializabilidade por conflito** (CSR)
  - Um *schedule* S é serializável por conflito, se ele é equivalente de conflito a algum *Schedule* serial.

Os três critérios de corretude podem ser utilizados para produzir *Schedules* corretos. Então, surge uma pergunta: Qual critério a adotar?

- Vale destacar que os três critérios de corretude apresentam diferentes graus de paralelismo, uma vez que permitem diferentes níveis de entrelaçamento.

  Para ilustrar, considere o seguinte exemplo:

  $S_S = r_2(x)w_2(x,5)r_1(x)w_1(x,10)$ e o valor inicial de x é 2

  $S_1 = r_1(x)r_2(x)w_2(x,5)w_1(x,10)$

  - Observe que S_1 produz o mesmo estado final que S_S, então S_1 é serializável por estado final. Logo, S_1 é correto.
  - Porém, note que S_1 é não é serializável por visão (uma vez que o valor lido por $r_1(x)$ é diferente nos dois *schedules*) e nem por conflito.

Assim, os três critérios de corretude relacionam-se da seguinte forma:

FSR ⊃ VSR ⊃ CSR

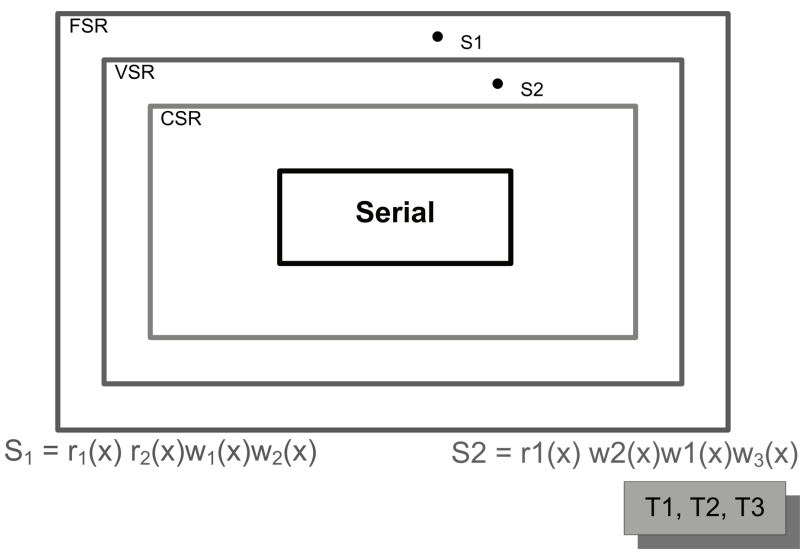


Figura 10 – Equivalência de schedules

- Além disso, os três critérios de corretude apresentam diferentes graus de complexidade. Dessa forma, os critérios de corretude (correção) FSR e VSR podem ser classificados como um problema NP completo, tendo no pior caso complexidade assintótica de ***n!***, onde n é o número de transações.

- Por outro lado, identificar se um *schedule* é CSR poder ser realizado em tempo polinomial. Por esse motivo, o critério comumente adotado pelos SGBDs é o da serializabilidade por conflito.

Agora, podemos definir mais formalmente a serializabilidade por conflito:

- **Serializabilidade por conflito**
  - **S** é serializável por conflito, se **S** é equivalente por conflito a algum *schedule* serial $\mathbf{S_S}$ sobre o mesmo conjunto de transações

Para ilustrar a serializabilidade por conflito, considere o exemplo a seguir:

Exemplo:

T_1=$\mathbf{r_1(y)r_1(x)w_1(y)}$ T_2=$\mathbf{r_2(x)r_2(y)w_2(x)}$

T_3=$\mathbf{r_3(y)r_3(x)w_3(z)}$

S = $\mathbf{r_3(y)r_1(y)r_1(x)r_2(x)w_1(y)r_2(y)r_3(x)w_2(x)w_3(z)}$ → *Schedule* original

S_S=$\mathbf{r_3(y)r_3(x)w_3(z)r_1(y)r_1(x)w_1(y)r_2(x)r_2(y)w_2(x)}$ → *Schedule* serial (T_3,T_1,T_2)

Operações em conflito do *Schedule* S	Operações em conflito do *Schedule* S_S
$r_3(y) <_S w_1(y)$	$r_3(y) <_S w_1(y)$
$r_1(x) <_S w_2(x)$	$r_3(x) <_S w_2(x)$
$w_1(y) <_S r_2(y)$	$r_1(x) <_S w_2(x)$
$r_3(x) <_S w_2(x)$	$w_1(y) <_S r_2(y)$

Figura 11 – Serializabilidade por conflito

Como verificamos (Figura 11) por meio da forma como os dois *Schedule*s ordenam suas operações em conflito, podemos dizer que o *Schedule* S é serializável por conflito. Essa é uma das formas de verificação da serializabilidade por conflito de um *Schedule*.

Porém, essa não é a única forma. Podemos verificar se um *Schedule* é serializável por conflito por meio do seu Grafo de Serialização, onde não precisamos comparar um dado *Schedule* com outro *schedule* serial.

3.4 Grafo de serialização de um schedule S

- Considere que S é um *schedule* definido para um conjunto de transações $\Im=\{T_1, T_2, ..., T_n\}$
- Assuma que SG(S) representa o grafo de serialização para o *schedule* S. Como todo grafo, SG(S) pode ser definido como um conjunto de vértices (V) e arestas (E)

- Assim, SG(S)=(V,E) tal que:
  - SG(S) é um grafo direcionado
  - V= $\Im$
  - $T_i \rightarrow T_j \in E \leftrightarrow$
    i. $T_i, T_j \in \Im$,
    ii. $\exists\ p \in OP(T_i)$, $q \in OP(T_j)$, tal que *p* conflita com *q* e $p <_S q$

O grafo de serialização de um *schedule* é um grafo direcionado, onde cada transação é um vértice do grafo e as operações em conflito são as arestas que conectam os vértices (transações). Um exemplo desse grafo pode ser observada na Figura 12.

- Grafo de serialização de um *Schedule* S
  - S está definido para um conjunto $\Im=\{T_1, T_2, T_3\}$

$S=r_3(y)r_1(y)r_1(x)r_2(x)w_1(y)r_2(y)r_3(x)w_2(x)w_3(z)$

Vértices (Transações):	T_1, T_2, T_3
Arestas (operações em conflito):	$r_3(y) < w_1(y)$ → aresta de 3 para 1 $r_1(x) < w_2(x)$ → aresta de 1 para 2 $w_1(y) < r_2(y)$ → aresta de 1 para 2 $r_3(x) < w_2(x)$ → aresta de 3 para 2

Figura 12 – Grafo de serialização

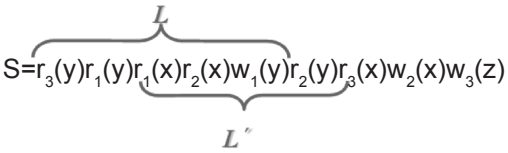
- **Teorema**
  - Um *schedule* S é serializável por conflito ⇔ **o grafo de serialização de S não contém ciclos**
    - Para verificar existência de ciclos em grafos existem algoritmos que podem ser executados em tempo polinomial, ou seja, cuja complexidade assintótica é $O(n^2)$, onde *n* representa o número de transações.

3.5 Prefixo de um Schedule

Com base no grafo de serialização, chegamos a outro importante conceito que é muito útil para identificar corretude de *Schedule*s dinamicamente, é o conceito de Prefixo de um *Schedule*.

Vamos considerar o prefixo *L* de um *Schedule S*:

1. $OP(L) \subseteq OP(S)$
2. $\forall p,q \in OP(L),\ p <_L q \leftrightarrow p <_s q$
3. $\forall q \in OP(L), \forall p \in OP(S),\ p <_s q \rightarrow p \in OP(L)$



Propriedade *P* de *Schedule* fechada por prefixo

- Se *P* é válida para o *Schedule* completo S, *P* também é válida para qualquer prefixo de S

Esse conceito, na verdade, mostra que a Serializabilidade por conflito é uma propriedade fechada por prefixo:

- **Se algum prefixo não é serializável por conflito, o *schedule* completo também não será!** *Schedule* incorreto.

Dessa forma, à medida que as operações vão sendo escalonadas, se ocorrer um ciclo em qualquer parte do *Schedule*, o mesmo já pode ser considerado incorreto. Não precisa checar todo o *Schedule*.

Exercícios

1. Verifique se o seguinte *Schedule* é serializável por estado final e por visão. Justifique sua resposta.

   T1=r1(y)r1(x)w1(y) T2= w2(u)r2(x)r2(y)w2(x)

   T3=r3(y)r3(x)w3(u)w3(z) T4= r4(v)w4(u)

   S=r4(v)r3(y)r1(y)w2(u)r2(x)r2(y)w1(y)w4(u)w2(x)r1(x)r3(x)w3(u)w3(z)

2. Verifique a corretude do seguinte *Schedule*:

   $S´=r_3(y)r_1(y)r_1(x)r_2(x)w_1(y)r_2(y)w_2(x)r_3(x)w_3(z)$

3. Verifique se o seguinte *schedule* é serializável por conflito. Justifique sua resposta.

   T1=r1(y)r1(x)w1(y) T2= w2(u)r2(x)r2(y)w2(x)

   T3=r3(y)r3(x)w3(u)w3(z) T4=r4(v)w4(u)

S=r4(v)r3(y)r1(y)r1(x)w2(u)r2(x)w1(y)r2(y)w4(u)r3(x)w2(x)w3(u)w3(z)

4. Prove que se dois *Schedules* são equivalentes, então seus grafos de serialização são iguais. Prove também que o inverso não é verdade.

5. Suponha duas transações T1 e T2:

```
BEGIN TRANSACTION T1
UPDATE Banco
SET Saldo = ((SELECT Saldo FROM Banco WHERE Conta = 10) –
100)
WHERE Conta = 10
UPDATE Banco
SET Saldo = ((SELECT Saldo FROM Banco WHERE Conta = 20) +
100)
WHERE Conta = 20
COMMIT TRANSACTION T1

BEGIN TRANSACTION T2
UPDATE Banco
SET Saldo = ((SELECT Saldo FROM Banco WHERE Conta = 20) –
200)
WHERE Conta = 20
UPDATE Banco
SET Saldo = ((SELECT Saldo FROM Banco WHERE Conta = 30) +
200)
WHERE Conta = 30
COMMIT TRANSACTION T2
```

a) As instruções de leitura e de escrita dos dados da transação T1, obtidas a partir da SQL, são as seguintes:

```
READ Conta 10
WRITE Conta 10
READ Conta 20
WRITE Conta 20
```

Seguindo o exemplo acima, obtenha as instruções de leitura e de escrita dos dados da transação T2.

b) Determine as possíveis escalas de execução concorrente serializáveis em conflito e/ou em visão.

4. CONFIABILIDADE DE SCHEDULES

4.1 Corretude x Confiabilidade

Até agora, temos discutido o modelo clássico para a execução de transações concorrentes sem considerar a presença de falhas. Na prática, as transações são executadas em ambientes onde podem ocorrer falhas. Logo, um mecanismo eficiente para o processamento de transações deve suportar a existência de falhas.

Outra contribuição importante do modelo clássico de processamento de transações é o conceito de *schedule* confiável. A confiabilidade está relacionada ao grau de consistência. A seguir definiremos formalmente a noção de *schedule* confiável.

Dizemos que um *Schedule* é confiável se as seguintes condições são satisfeitas:

i. O *abort* de uma determinada transação T_i não afeta a semântica das transações que já executaram sua operação de *commit*, e

ii. Quando uma determinada transação T_i aborta, os valores dos objetos atualizados anteriormente por T_i devem ser restaurados como se essa transação nunca tivesse sido executada. Os efeitos de uma transação abortada devem ser completamente desfeitos.

Torna-se importante destacar que os conceitos de serializabilidade e confiabilidade de *schedules* são propriedades ortogonais. Existem *schedules* serializáveis que não são confiáveis e vice-versa. Contudo, do ponto de vista prático os *schedules* devem ser corretos (serializáveis) e confiáveis. Vejamos o exemplo da Figura 13:

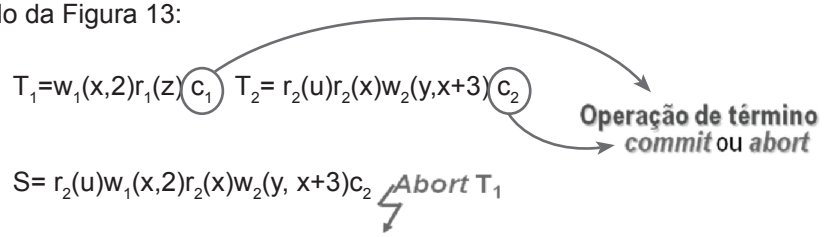


Figura 13 – Exemplo de escalonamento

- Após o *abort* de T_1
  - $r_2(x)$ torna-se uma leitura inconsistente (*dirty read*)
  - Desfazer execução de T_1
    - Violação da semântica da operação $r_2(x)$ ($r_2(x)$ leu o valor que já tinha sido alterado por $w_1(x)$)
  - Abortar T_2
    - Violação da semântica da operação c_2 (nenhuma

transação pode ser abortada após ter realizado *Commit*)

- Execução de S é correta (serializável), porém
  - **Execução de S não é confiável**

A seguir iremos caracterizar melhor os *Schedules* confiáveis. De fato, existem três diferentes níveis de confiabilidade de *Schedules*: **Recuperável**, **Evita *Abort* em Cascata** e **Preciso**.

4.2 *Schedule* Recuperável

O nível mais genérico é chamado de recuperação de *schedules*. Um *schedule* S é dito recuperável se a condição abaixo é satisfeita:

Se $w_i(x) <_S r_j(x)$, a operação *commit* da transação T_i precede a operação de *commit* da transação T_j, isto é, $c_i <_S c_j$.

Schedules recuperáveis garantem que, uma vez que uma transação T_i executa sua operação de *commit*, seus efeitos nunca serão desfeitos.

Por exemplo:

$T_1=w_1(x,2)r_1(z)c_1 \quad T_2= r_2(u)r_2(x)w_2(y, x+3)c_2$

$S_1= r_2(u)w_1(x,2)r_2(x)r_1(z)w_2(y, x+3)c_1c_2$

- Se $w_1(x) <_{S1} r_2(x) \rightarrow c_1 <_S c_2$
- S_1 é recuperável

4.3 *Schedule* que evita *abort* em cascata

Entretanto, mesmo em *schedules* recuperáveis, o *abort* de uma transação T_i pode provocar efeitos colaterais indesejáveis. Por exemplo, se $w_i(x) <_S r_j(x)$, e a transação T_i aborta após a execução da operação $r_j(x)$ e antes de T_j executar sua operação de *commit*, o valor lido por $r_j(x)$ não é válido. Isto implica que a transação T_j também deve ser abortada. Esse fenômeno é chamado de *abort* em cascata. A fim de evitar a ocorrência desses fenômenos nós necessitamos de um nível de confiabilidade mais forte. Um *schedule* é dito que evita *aborts* em cascata se:

Sempre que $w_i(x) <_S r_j(x)$, então $c_i <_S r_j(x)$ ou $a_i <_S r_j(x)$

Por exemplo:

$S_1= r_2(u)w_1(x,2)r_2(x)w_2(y, x+3)$ 

Nesse caso, o abort de T_1 força o abort de T_2.

Exemplo 2:

$S_2 = r_2(u)w_1(x,2)r_1(z)c_1r_2(x)w_2(y, x+3)c_2$

- Se $w_i(x) <_s r_j(x) \rightarrow c_i <_s r_j(x)$
- S_2 evita aborts em cascata

4.4 *Schedule* preciso (*strict*)

Algumas vezes, a propriedade que evita o abort em cascata ainda não é suficientemente forte para garantir um nível de confiabilidade desejável. O *Schedule* S mostrado a seguir ilustra esse fato.

$S = w_i(x)\ w_j(x)\ c_j\ a_i$

Como T_i foi abortada, o valor do objeto x deve ser restaurado para o valor anterior à execução da transação T_i. Infelizmente, isto não é possível, pois a transação T_j já executou sua operação de *commit* e sobrescreveu o valor escrito por T_i. Esse problema pode ser evitado fazendo com que a operação $w_j(x)$ espere até que a transação T_i execute sua operação de *commit* ou de *abort*, só então a operação $w_i(x)$ poderá ser executada. Essa condição é garantida pelo nível mais forte de confiabilidade, que é chamado "*strictness*". Um *schedule* é chamado estrito ("*strict*") se:

Sempre que $w_i(x) <_S p_j$ então $c_i <_S p_j$ ou $a_i <_S p_j$, onde $p_j \in \{r(x), w(x)\}$

Observe que um *schedule* estrito ("*strict*") evita *aborts* em cascata e é recuperável. Isto se deve ao fato de que a classe de *Schedule*s estritos ("*strict*") é uma subclasse dos *schedules* que evitam *aborts* em cascata, o qual por sua vez é uma subclasse dos *schedules* recuperáveis.

Exemplo 1:

$S_3 = w_1(x,2)w_2(x,5)\ c_2$ T_1

- Não é possível recuperar o valor que x tinha antes da operação $w_1(x,2)$

Exemplo 2:

$S_4 = w_1(x,2)c_1w_2(x,5)c_2$

- Se wi(x) <S opj(x) → ci<S opj(x) ou ai<S opj(x), onde opj ∈ {rj, wj}
- S4 é preciso

E como funcionam os DBMSs existentes?

Os atuais DBMSs produzem *schedules*:

- Corretos → **Serializáveis por conflito**
- Confiáveis → **Precisos** (grau de confiabilidade)

Exercícios

1. Classifique os *schedules* abaixo de acordo com as seguintes classes de *Schedules*: serializáveis por conflito, recuperáveis, que evitam *aborts* em cascata, e estritos (strict).
   a) $r_1(x)\ r_2(y)\ r_3(x)\ w_3(x)\ r_2(x)\ r_1(y)\ w_1(x)\ c_1\ c_2\ c_3$
   b) $r_1(x)\ r_1(y)\ w_1(x)\ r_2(y)\ w_3(y)\ w_1(x)\ r_2(y)\ c_1\ c_2\ c_3$
   c) $r_4(v)r_3(y)r_1(y)r_1(x)\ w_2(u)r_2(x)w_1(y)r_2(y)w_4(u)r_3(x)w_2(x)w_3(u)w_3(z)\ c_1\ c_2\ c_3\ c_4$
   d) $r_1(x)\ w_2(x)\ w_1(x)\ a_2\ c_1$

2. Considere o seguinte *schedule* S (incompleto) :

   $r_1(x)\ r_1(y)\ w_1(x)\ r_2(y)\ w_3(y)\ w_1(x)\ r_2(y)$

   Assumindo que as três transações executam suas operações de commit, mostre o grafo de serialização para o *Schedule* S.
   Para cada uma das opções baixo modifique o *Schedule* S a fim de criar um novo *Schedule* (completo) T tal que:
   - T evita aborts em cascata e não é recuperável.
   - T é recuperável.
   - T é serializável por conflito.

   Para criar o *schedule* T você pode adicionar novas operações em qualquer posição do *Schedule* S, mas não pode alterar a ordem das operações já existentes. Caso não seja possível criar o *Schedule* T como se pede, comente brevemente.

UNIDADE 3

CONTROLE DE CONCORRÊNCIA

Resumindo

Esta unidade discute em detalhes o problema de controle de concorrência em bancos de dados. Vários algoritmos para controle de concorrência são apresentados e discutidos. A apresentação de cada método é acompanhada de uma discussão sobre problemas adicionais, ocasionados pelo método, que possam impedir o término normal das transações. Uma classificação desses protocolos também é apresentada e discutida.

3 CONTROLE DE CONCORRÊNCIA

1. INTRODUÇÃO

1.1 Conceitos Iniciais

Os modelos de processamento de transações são implementados através dos protocolos de controle de concorrência. Esses protocolos são responsáveis por produzir estórias (*schedules*) em conformidade com o critério de corretude (ou correção) adotado. Pelo termo "produzir *schedules*"', entendemos que um protocolo de controle de concorrência recebe como entrada as operações pertencentes a um determinado conjunto de transações e produz como saída uma sequência corretamente entrelaçada dessas operações. Por exemplo, o protocolo 2PL (*Two Phase Locking*) é um protocolo de controle de concorrência que implementa o modelo clássico de processamento de transações, logo, utiliza serializabilidade como critério de correção. Assim, o protocolo 2PL produz *schedules* serializáveis. Os protocolos de controle de concorrência constituem o núcleo dos escalonadores (*schedulers*).

1.2 Classificação dos Protocolos de Controle de Concorrência

Os protocolos de controle de concorrência podem ser classificados como **agressivos** ou **conservativos** (ou conservadores). Um protocolo agressivo tenta escalonar as operações imediatamente. Dessa forma, um protocolo agressivo evita adiar a execução das operações de uma transação. Entretanto, essa estratégia pode levar a situações onde não é possível produzir uma execução correta de todas as transações ativas. Nesse caso, um protocolo agressivo rejeita operações de uma ou mais transações, levando essas transações ao estado "*aborted*". Assim, os protocolos agressivos podem gerar taxas de *aborts* inaceitáveis.

Adicionalmente, os protocolos agressivos podem ser classificados em duas subclasses: **pessimistas** e **otimistas**. Um *scheduler* que implementa

um protocolo pessimista deve decidir se aceita ou rejeita uma operação tão logo ele receba essa operação. Por outro lado, um *scheduler* que utiliza um protocolo otimista escalona as operações que recebe de forma imediata. Após um determinado período de tempo, o *scheduler* verifica se o *schedule* que está sendo gerado é correto ou não. Caso o *schedule* seja correto, o *scheduler* continua escalonando operações. Caso contrário, o *scheduler* deve abortar uma ou mais transações. Um exemplo clássico para essa classe de protocolos são os certificadores.

Por sua vez, um protocolo conservativo tende a adiar a execução de determinadas operações com a finalidade de escaloná-las corretamente. Dessa forma, esses protocolos, apesar de não induzir o *abort* de transações, podem levar as aplicações a tempos de espera inaceitáveis, principalmente se essas forem transações de longa duração A Figura 1.1 mostra uma visão geral dessa classificação.

Fn

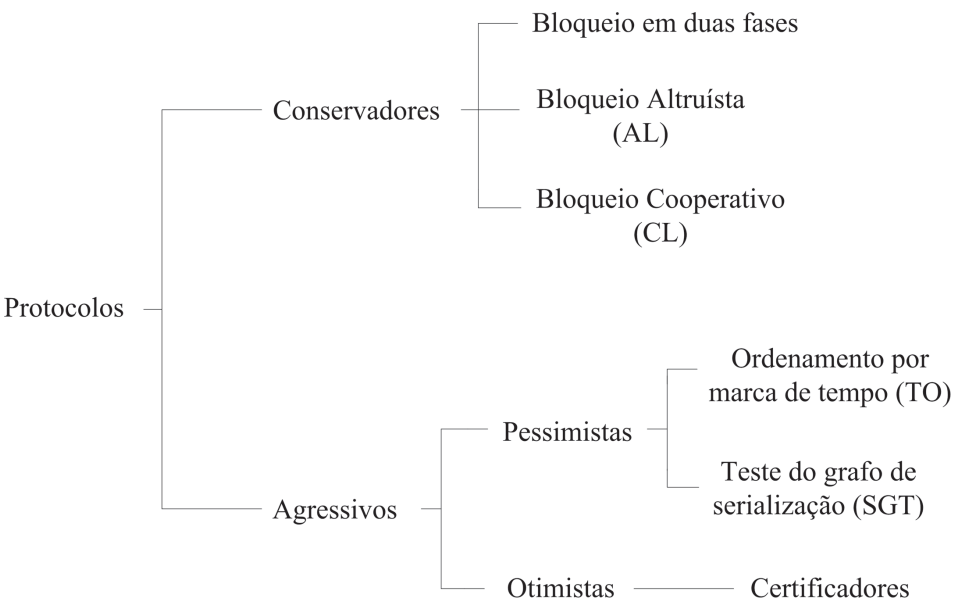


Figura 1.1 – Classificação dos Protocolos de Controle de Concorrência

Exercícios

1. Quais as principais características de um protocolo agressivo?
2. Quais as principais características de um protocolo conservativo?
3. Quais as principais características de um protocolo otimista?
4. Quais as principais características de um protocolo pessimista?

2. DESCRIÇÃO DOS PROTOCOLOS CONSERVADORES

Este capítulo discute os protocolos baseados em bloqueios, os quais são utilizados no controle de concorrência em SGBDs relacionais. Inicialmente os problemas de gerência de bloqueios e de tratamento de impasses (*deadlocks*) são abordados. Em seguida, um método que utiliza o conceito de "bloqueio" para assegurar a geração de *schedules* (execuções) serializáveis, denominado bloqueio em duas fases, é apresentado. Por fim, a correção do método é provada.

2.1 Protocolos Baseados em Bloqueios

2.1.1 Conceitos Iniciais

Nos protocolos baseados em bloqueio, uma transação T_i somente pode acessar um item de dado caso tenha anteriormente obtido um bloqueio sobre esse item. Assim, a fim de acessar um determinado item, a transação T_i precisa primeiramente bloqueá-lo. Caso o item de dado esteja bloqueado por outra transação (T_j, por exemplo) então T_i deverá esperar até que o bloqueio seja liberado. Dessa forma, o *scheduler* garante que somente uma transação pode acessar um determinado item de dado por vez.

Para ilustrar a utilização do conceito de bloqueio, suponha inicialmente que só há um modo de bloqueio, ou seja, que cada item de dado (objeto) só pode estar em dois estados:

i) **bloqueado:** permite acesso ao objeto apenas pela transação que detém o bloqueio;

ii) **livre:** ainda não está bloqueado por nenhuma transação. Logo, esse item de dado ainda não pode ser acessado por nenhuma transação, uma vez que para acessar um determinado item de dado uma transação precisa inicialmente obter um bloqueio sobre esse item.

Nesse caso, é necessário introduzir apenas duas novas ações elementares ao nosso repertório (que contém até o momento as operações $r_i(x)$, $w_i(x)$, a_i e c_i):

i) ***li(x)*** bloqueia (*lock*) o item de dado (objeto) *x para a transação* T_i;

ii) $\boldsymbol{u_i(x)}$ a transação T_i libera (*unlock*) o bloqueio que detém sobre o item de dado (objeto) x;

As informações necessárias para gerenciar a concessão e liberação de bloqueios são, em geral, armazenadas em uma estrutura de dados denominada "***tabela de bloqueios***". *Essa tabela é* modelada como uma coleção de triplas (x, T_i, F) onde:

i) x é o nome de um item de dado (objeto);

ii) T_i é o nome da transação que mantém o bloqueio sobre *x;*

iii) F é uma *fila de espera* para *x,* contendo os nomes de todas as transações que esperam a liberação de *x.*

2.1.2 Protocolo de Bloqueio Básico

A seguir apresentamos um protocolo de controle de concorrência rudimentar, baseado na ideia de bloqueio.

Passo 1: Inicialmente a tabela de bloqueios está vazia (λ indica a fila, F, vazia);
Passo 2: Ao receber solicitação para bloquear o objeto *x* para a transação T_i, por meio da operação $l_i(x)$, pesquise a tabela de bloqueios procurando uma tripla cujo primeiro elemento seja *x*:

i) se nenhuma tripla for encontrada (ou seja, se *x* está livre), bloqueie *x* para T_i, acrescentando a tripla *(x,* T_i, λ*)* à tabela.

ii) caso contrário, acrescente T_i ao final da fila de espera para *x* na tripla encontrada.

Passo 3: Ao receber solicitação da transação T_i por meio da operação $u_i(x)$ para liberar o objeto x, pesquise a tabela de bloqueios procurando uma tripla cujos dois primeiros elementos sejam *x e* T_i:

i) se nenhuma tripla for encontrada, ignore a liberação de *x*.

ii) caso contrário, seja *(x,* T_i, *F)* a tripla encontrada:

a) se a fila *F* estiver vazia (F = λ), retire a tripla da tabela, liberando *x*.

b) se a fila *F* não estiver vazia (F ≠ λ), passe o controle de *x, ou seja, bloqueie x,* para a primeira transação T_j em F, em seguida, retire T_j de F.

Exemplo 1

A fim de ilustrar o funcionamento do protocolo de bloqueio básico considere inicialmente o seguinte conjunto de transações:

$T_1 = r_1(x)\ r_1(y)\ w_1(x)\ w_1(y)\ c_1$
$T_2 = r_2(x)\ c_2$

Considerando que antes de qualquer operação sobre um determinado item de dado é necessário obter um bloqueio sobre esse item e que a liberação dos bloqueios ocorrerão assim que o item de dado não for mais utilizado, podemos representar as transações T_1 e T_2 da seguinte forma:

$T_1 = l_1(x)\ r_1(x)\ l_1(y)\ r_1(y)\ w_1(x)\ w_1(y)\ u_1(x)\ u_1(y)\ c_1$

$T_2 = l_2(x)\ r_2(x)\ u_2(x)\ c_2$

Considere agora a seguinte execução (*schedule*):

$S_1 = r_1(x)\ r_2(x)\ c_2\ r_1(y)\ w_1(x)\ w_1(y)\ c_1$

ou, mais precisamente:

$S_1 = l_1(x)\ r_1(x)\ l_2(x)\ r_2(x)\ u_2(x)\ c_2\ l_1(y)\ r_1(y)\ w_1(x)\ w_1(y)\ u_1(x)\ u_1(y)\ c_1$

Assuma que essa execução (S_1) representa a ordem com que as operações enviadas pelas transações T_1 e T_2 chegaram ao escalonador (o qual utiliza o protocolo descrito acima).

Observe que o *schedule* S_1 não poderia ser obtido pelo protocolo discutido, uma vez que a operação $l_2(x)$, e consequentemente $r_2(x)$, não pode ser executada até que a transação T_1 libere o bloqueio obtido sobre o item x através da operação $u_1(x)$.

Nesse caso, o *schedule* gerado pelo protocolo poderia ser representado da seguinte forma:

$S_2 = l_1(x)\ r_1(x)\ l_1(y)\ r_1(y)\ w_1(x)\ w_1(y)\ u_1(x)\ l_2(x)\ r_2(x)\ u_2(x)\ c_2\ u_1(y)\ c_1$

A Figura 2.1 ilustra o grafo de serialização para o *schedule* S_2. Como o grafo de serialização do *schedule* S_2 não possui ciclo, podemos concluir que S_2 é serializável por conflito (e, por conseguinte, correto).

T1 → T2

Figura 2.1 – Grafo de Serialização do *Schedule* S_2.

Porém, se montarmos o grafo de serialização para o *schedule* S_1 (figura 2.2), veremos que esse também não possui ciclo. Assim, podemos concluir que S_1 também é serializável por conflito (e, por conseguinte, correto). Contudo, o *schedule* S_1 não seria gerado pelo protocolo apresentado, apesar de apresentar um grau de concorrência maior que o *schedule* S_2. Esse maior grau de concorrência decorre do fato de que no *schedule* S_1 a operação $r_2(x)$ é executada bem antes do que no *schedule* S_2.

T1 ← T2

Figura 2.2. Grafo de Serialização do *Schedule* S_1.

Bloqueio Compartilhado e Exclusivo

O protocolo de bloqueio básico pode ser melhorado incorporando-se modalidades (ou modos) diferentes de bloqueio. Uma opção seria adotar duas

modalidades de bloqueio:

i) **Bloqueio de leitura** ou compartilhado (*read lock* – rl)

ii) **Bloqueio de escrita** ou exclusivo (*write lock* – wl)

Isto significa que agora um item de dado poderá estar em três estados:

i) **bloqueado para leitura** (ou de forma compartilhada);

ii) **bloqueado para escrita** (ou de forma exclusiva);

iii) **livre** (ou seja**,** não está bloqueado por nenhuma transação).

Uma forma precisa de definir a compatibilidade das modalidades de bloqueio seria através de uma matriz de compatibilidades, a qual indica quando duas transações pode podem bloquear o mesmo item de dado e quando não o podem fazer (figura 2.3):

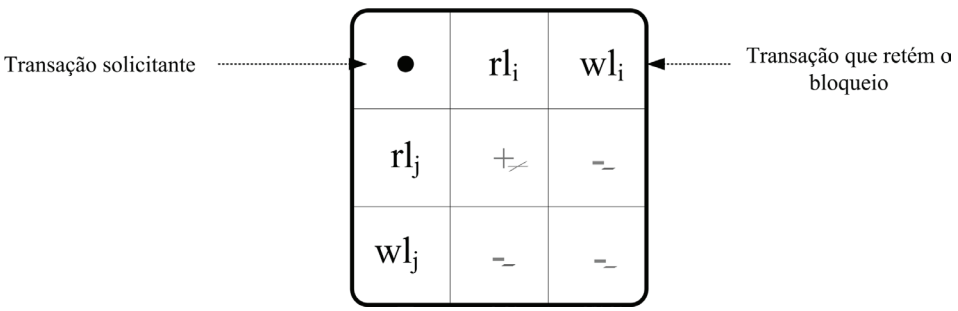


Figura 2.3 – Matriz de Compatibilidade de Bloqueios

A justificativa para as modalidades de bloqueio, anteriormente definidas, é simples. Duas ou mais transações poderão ler o item x simultaneamente, sem perigo de conflito, bloqueando-o para leitura (modo compartilhado). Por outro lado, se uma transação T_i deseja atualizar (gravar ou escrever) o item x, essa operação conflita com qualquer outra operação de leitura ou escrita executada sobre x por outra transação T_j. Logo T_i deverá bloquear x em modo exclusivo (para escrita), não permitindo assim que nenhuma outra transação bloqueie x em qualquer modo (leitura ou escrita).

Cabe observar que a matriz de compatibilidades se refere a ações de transações diferentes, e não se aplica a operações de uma mesma transação. Assim, se uma transação T_i já mantém um item x bloqueado para leitura (bloqueio compartilhado) e desejar bloqueá-lo para escrita (bloqueio exclusivo), poderá fazê-lo se for a única transação que no momento mantém x bloqueado.

Nesse caso, é necessário substituir a operação $l_i(x)$ por duas novas operações:

i) ***rli(x)*** bloqueia (*lock*) o item de dado (objeto) x em modo de leitura (compartilhado) para a transação T_i;

ii) **wl_i(x)** bloqueia (*lock*) o item de dado (objeto) x em modo de escrita (exclusivo) para a transação T_i;

Além disso, a tabela de bloqueios deve ser modelada agora como uma coleção de *tuplas (x,modo,T, F)* onde:

i) x é o nome de um item de dado (objeto);

ii) modo representa a modalidade pela qual o item de dado x foi bloqueado (leitura ou escrita);

iii) T conjunto das transações que mantêm bloqueios sobre *x.* Se x estiver bloqueado para escrita T será um conjunto unitário, ou seja, T conterá uma única transação. Se x estiver bloqueado para leitura o conjunto T poderá conter mais de uma transação;

iv) F é uma *fila de espera* para *x,* contendo pares de valores do tipo <T_i, modo> onde T_i representa o nome de uma transação e modo indica a modalidade do bloqueio requisitado por T_i.

Agora, o protocolo de bloqueio básico, utilizando bloqueios de leitura e de escrita, pode ser descrito da seguinte forma:

Passo 1: Inicialmente a tabela de bloqueios está vazia (λ indica a fila, F, vazia);
Passo 2: Ao receber solicitação para bloquear o objeto *x* para a transação T_i, por meio de uma operação $rl_i(x)$ ou $wl_i(x)$, genericamente representada por $pl_i(x)$, pesquise a tabela de bloqueios procurando uma *tupla* cujo primeiro elemento seja *x*:

i) se nenhuma *tupla* for encontrada (ou seja, se *x* está livre), bloqueie *x* para T_i, no modo requisitado, acrescentando a *tupla (x,* modo*,* T_i*,* λ*)* à tabela.

ii) caso contrário, o objeto x já se encontra bloqueado por uma ou mais transações. Nesse caso, verifique se o modo (leitura ou escrita) do bloqueio solicitado por T_i é compatível com o modo pelo qual o item x já se encontra bloqueado:

1) em caso afirmativo, bloqueie *x* para T_i, no modo requisitado, atualizando a *tupla (x,* modo*,* T*, F)* à tabela, ou seja, insira T_i em T.
2) caso contrário, acrescente T_i ao final da fila de espera para *x* na tripla encontrada, ou seja, insira <T_i, modo> em F.

Passo 3: Ao receber solicitação da transação T_i por meio da operação $u_i(x)$ para liberar o objeto x, pesquise a tabela de bloqueios procurando uma *tupla* cujo primeiro elemento seja *x e* $T_i \in$ T.

i) se nenhuma *tupla* for encontrada, ignore a liberação de *x*.

ii) caso contrário, seja *(x,* modo, T*, F)* a *tupla* encontrada, onde $T_i \in$ T.

a) se T – {T_i} = Φ

- se a fila *F* estiver vazia (F = λ), retire a *tupla* da tabela, liberando *x*.

- se a fila *F* não estiver vazia (F ≠ λ), passe o controle

de *x, ou seja, bloqueie x,* para a primeira transação T_j em F, em seguida, retire T_j de F, adicione T_j em T e faça modo ← o modo do bloqueio solicitado por T_j.

Exemplo 2

A fim de ilustrar o funcionamento do protocolo de bloqueio básico com dois modos de bloqueio (leitura e escrita), considere inicialmente o seguinte conjunto de transações:

$T_1 = r_1(x)\ r_1(y)\ w_1(x)\ w_1(y)\ c_1$

$T_2 = r_2(x)\ c_2$

Considerando que antes de qualquer operação sobre um determinado item de dado é necessário obter um bloqueio sobre esse item e que a liberação dos bloqueios ocorrerão assim que o item de dado não for mais utilizado, podemos representar as transações T_1 e T_2 da seguinte forma:

$T_1 = rl_1(x)\ r_1(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ wl_1(y)\ w_1(y)\ u_1(x)\ u_1(y)\ c_1$

$T_2 = rl_2(x)\ r_2(x)\ u_2(x)\ c_2$

Considere agora a seguinte execução (*schedule*):

$S_1 = r_1(x)\ r_2(x)\ c_2\ r_1(y)\ w_1(x)\ w_1(y)\ c_1$

ou, mais precisamente:

$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ u_2(x)\ c_2\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ wl_1(y)\ w_1(y)\ u_1(x)\ u_1(y)\ c_1$

Assuma que essa execução (S_1) representa a ordem com que as operações enviadas pelas transações T_1 e T_2 chegaram ao escalonador.

Observe que o *schedule* S_1, que não poderia ser obtido pelo protocolo discutido anteriormente (com um único modo de bloqueio), passa a ser possível na versão do protocolo de bloqueio básico com dois modos de bloqueio (leitura e escrita). Assim, podemos observar que essa nova versão do protocolo apresenta um grau de concorrência maior, se comparado à versão anterior.

A figura 2.4 ilustra o grafo de serialização para o *schedule* S_1.

T1 ← T2

Figura 2.4 – Grafo de Serialização do *Schedule* S_1.

Exemplo 3

Sejam A e B duas contas que são acessadas pelas transações T_1 e T_2. A transação T_1 transfere R$ 50,00 da conta B para a conta A e tem a forma:

$T_1 = rl_1(B)\ r_1(B)\ wl_1(B)\ w_1(B, B\text{-}50)\ u_1(B)\ rl_1(A)\ r_1(A)\ wl_1(A)\ w_1(A, A+50)\ u_1(A)\ c_1$

A transação T_2 apresenta o saldo total das contas A e B, isto é, a soma A+B e é definida por:

T_2 = $rl_2(A)$ $r_2(A)$ $u_2(A)$ $rl_2(B)$ $r_2(B)$ $u_2(B)$ **Display (A+B)** c_2

Suponha que o saldo inicial sejam R$ 100,00 e R$ 200,00 para as contas A e B, respectivamente. Se essas duas transações são executadas serialmente, na ordem T_1, T_2 ou T_2, T_1, então a transação T_2 mostrará o valor R$ 300,00. Contudo, se essas transações forem executadas concorrentemente, a execução S_1 (*schedule*) abaixo poderá ocorrer e o valor R$ 250,00, que não é correto, será exibido pela transação T_2.

S_1 = $rl_1(B)$ $r_1(B)$ $wl_1(B)$ $w_1(B, B-50)$ $u_1(B)$ $rl_2(A)$ $r_2(A)$ $u_2(A)$ $rl_2(B)$ $r_2(B)$ $u_2(B)$ **Display (A+B)** c_2 $rl_1(A)$ $r_1(A)$ $wl_1(A)$ $w_1(A, A+50)$ $u_1(A)$ c_1

Assuma que essa execução (S_1) representa a ordem com que as operações enviadas pelas transações T_1 e T_2 chegaram ao escalonador.

T_1	T_2
$rl_1(B)$	
$r_1(B)$	
$wl_1(B)$	
$w_1(B, B-50)$	
$u_1(B)$	
	$rl_2(A)$
	$r_2(A)$
	$u_2(A)$
	$rl_2(B)$
	$r_2(B)$
	$u_2(B)$
	Display(A+B)
	c_2
$rl_1(A)$	
$r_1(A)$	
$wl_1(A)$	
$w_1(A, A+50)$	
$u_1(A)$	
c_1	

Figura 2.5 – Execução Concorrente S_1 (Exemplo 3).

Esse comportamento incorreto, denominado leitura suja (*dirty read*), ocorreu devido ao fato das transações T_1 e T_2 terem liberado os bloqueios obtidos sobre os itens de dados A e B durante o decorrer da execução concorrente. Logo, podemos concluir que o protocolo de bloqueio básico não evita a ocorrência de leituras sujas.

Exemplo 4

Considere agora o seguinte conjunto de transações:

$T_1 = rl_1(x)\ r_1(x)\ u_1(x)\ rl_1(y)\ r_1(y)\ u_1(y)\ wl_1(x)\ w_1(x)\ u_1(x)\ c_1$

$T_2 = rl_2(x)\ r_2(x)\ u_2(x)\ wl_2(x)\ w_2(x)\ u_2(x)\ c_2$

Considere agora a seguinte execução (*schedule*):

$S_1 = rl_1(x)\ r_1(x)\ u_1(x)\ rl_2(x)\ r_2(x)\ u_2(x)\ rl_1(y)\ r_1(y)\ u_1(y)\ wl_1(x)\ \mathbf{w_1(x)}\ u_1(x)\ wl_2(x)\ \mathbf{w_2(x)}\ u_2(x)\ c_1\ c_2$

Assuma que essa execução (S_1) representa a ordem com que as operações enviadas pelas transações T_1 e T_2 chegaram ao escalonador.

Note que a execução (história) S_1 apresenta o fenômeno da atualização perdida (*lost update*) uma vez que o resultado da operação de escrita $\mathbf{w_1(x)}$ será sobrescrito pela operação $\mathbf{w_2(x)}$.

Além disso, observe que o protocolo de bloqueio básico irá considerar a execução S_1 correta e, consequentemente, irá concluir com sucesso (“commitar”) as transações T_1 e T_2. Contudo, se construirmos o grafo de serialização para a execução S_1 (figura 2.6) veremos que esse apresenta um ciclo, o que indica que a execução S_1 não é serializável por conflito. Logo, nesse caso, o protocolo de bloqueio básico considerou correta uma execução incorreta (ou seja, que não é serializável por conflito). Esse comportamento ocorreu devido ao fato das transações T_1 e T_2 terem liberado os bloqueios obtidos sobre o item de dado x durante o decorrer da execução concorrente para posteriormente obter novo bloqueio sobre o mesmo item de dado. Assim, podemos concluir que **o protocolo de bloqueio básico não assegura a geração de execuções (*schedules*) serializáveis. Além disso, as execuções geradas por esse protocolo podem apresentar os fenômenos leitura suja e perda de atualização.**

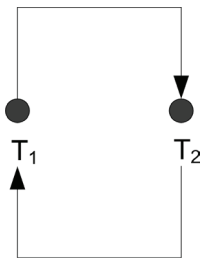


Figura 2.6 – Grafo de Serialização do *Schedule* S_1 no Exemplo 4.

Exemplo 5

Considere novamente que A e B são duas contas acessadas pelas transações T_1 e T_2. A transação T_1 transfere R$ 50,00 da conta B para a conta A e tem a forma:

$T_1 = rl_1(B)\ r_1(B)\ wl_1(B)\ w_1(B, B-50)\ u_1(B)\ rl_1(A)\ r_1(A)\ wl_1(A)\ w_1(A, A+50)\ u_1(A)\ c_1$

A transação T_2 apresenta o saldo total das contas A e B, isto é, a soma A+B e é definida por:

$T_2 = rl_2(A)\ r_2(A)\ u_2(A)\ rl_2(B)\ r_2(B)\ u_2(B)$ **Display (A+B)** c_2

Suponha agora, que os desbloqueios sejam realizados ao final da transação. A transação T_3 é similar à transação T_1, com desbloqueio ao final da transação, e é definida como:

$T_3 = rl_1(B)\ r_1(B)\ wl_1(B)\ w_1(B, B-50)\ rl_1(A)\ r_1(A)\ wl_1(A)\ w_1(A, A+50)\ u_1(B)\ u_1(A)\ c_1$

A transação T_4 corresponde à T_2, com desbloqueio ao final da transação, e é definida como:

$T_4 = rl_2(A)\ r_2(A)\ rl_2(B)\ r_2(B)$ **Display (A+B)** $u_2(A)\ u_2(B)\ c_2$

Observe agora a execução a seguir (figura 2.7).

T_3	T_4
$rl_1(B)$	
$r_1(B)$	
$wl_1(B)$	
$w_1(B, B-50)$	
	$rl_2(A)$
	$r_2(A)$
	$rl_2(B)$
$rl_1(A)$	
$r_1(A)$	
$wl_1(A)$	

Figura 2.7 – Execução Concorrente S_1 (Exemplo 5).

Observe que no momento em que a transação T_4 tenta obter um bloqueio de leitura sobre o item de dado B ($rl_2(B)$), esse já se encontra bloqueado em um modo incompatível (escrita) pela transação T_3. Logo, a transação T_4 irá esperar até que a transação T_3 libere seu bloqueio de escrita sobre o item B. Assim, a transação T_4 terá sua execução suspensa até que ela possa obter um bloqueio

de leitura sobre o item B. Contudo, a transação T_3 continua sua execução. No momento em que a transação T_3 tenta obter um bloqueio de escrita sobre o item de dado A ($wl_1(A)$), esse já se encontra bloqueado em um modo incompatível (leitura) pela transação T_4. Logo, a transação T_3 irá esperar até que a transação T_4 libere seu bloqueio de escrita sobre o item A. Assim, a transação T_3 terá sua execução suspensa até que ela possa obter um bloqueio de leitura sobre o item A. Nesse momento, chega-se a uma **situação de impasse (*deadlock*)**, ou seja, nenhuma das duas transações prossegue sua execução. A transação T_4 está esperando pela transação T_3 e a transação T_3 está esperando pela transação T_4.

Para solucionar alguns desses problemas, surgiu a ideia do protocolo de bloqueio em duas fases.

2.1.3. Protocolo de Bloqueio em Duas Fases (Básico)

O protocolo de bloqueio em duas fases (*two phase lock* – 2PL) é semelhante ao protocolo de bloqueio básico, contudo exige que cada transação emita suas solicitações de bloqueio e desbloqueio em duas fases:

1 – Fase de expansão: uma transação está nessa fase quando ela pode obter bloqueios, mas não pode liberar nenhum;

2 – Fase de encolhimento: uma transação pode liberar bloqueios, mas não consegue obter nenhum bloqueio novo.

Gráfico de bloqueios de T_i:

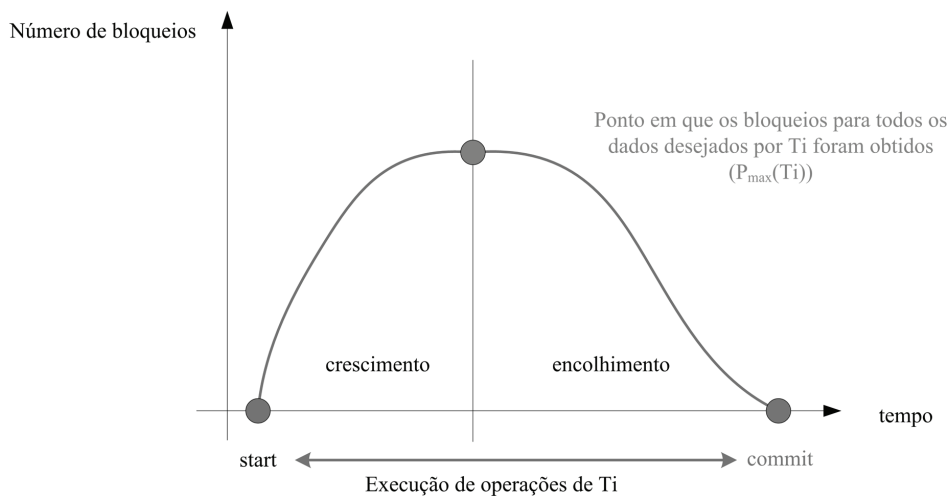


Figura 2.6 – Fases do protocolo 2PL.

Inicialmente uma transação está em fase de expansão. A transação adquire os bloqueios de que precisa. Tão logo a transação libera um bloqueio ela entra em fase de encolhimento. Considere qualquer transação, o ponto da

escala no qual a transação obteve seu último bloqueio (fim da fase de expansão) é chamado ponto de bloqueio da transação. Assim, as transações podem ser ordenadas de acordo com seus pontos de bloqueio.

Agora, considere novamente a execução concorrente (*schedule*) S_1 discutida no Exemplo 3 (figura 2.5):

S_1 = $rl_1(B)$ $r_1(B)$ $wl_1(B)$ $w_1(B, B-50)$ **$u_1(B)$** $rl_2(A)$ $r_2(A)$ $u_2(A)$ $rl_2(B)$ $r_2(B)$ $u_2(B)$ Display(A+B) c_2 **$rl_1(A)$** $r_1(A)$ $wl_1(A)$ $w_1(A, A+50)$ $u_1(A)$ c_1

Observe que essa execução concorrente não seria produzida pelo protocolo 2PL básico, pois uma vez que a transação T_1 libera o bloqueio obtido sobre o item B (**$u_1(B)$**) está transação entra na fase de encolhimento e não pode mais obter bloqueios. Assim, a transação T_1 não poderia obter o bloqueio sobre o item A (**$rl_1(A)$**). Assim, podemos observar que **o protocolo 2PL básico evita o fenômeno leitura suja**. Essa propriedade decorre do seguinte fato: quando uma transação T_i obtém um bloqueio de escrita sobre um determinado item de dado x, nenhuma outra transação T_J poderá ler o item x até que a transação T_i libere o bloqueio de escrita obtido sobre x e, a pós T_i liberar o bloqueio sobre x, T_i entra na fase de encolhimento, não podendo mais obter nenhum bloqueio.

Considere novamente a execução concorrente (*schedule*) S_1 discutida no Exemplo 4:

S_1 = $rl_1(x)$ $r_1(x)$ $u_1(x)$ $rl_2(x)$ $r_2(x)$ $u_2(x)$ $rl_1(y)$ $r_1(y)$ $u_1(y)$ $wl_1(x)$ **$w_1(x)$** $u_1(x)$ $wl_2(x)$ **$w_2(x)$** $u_2(x)$ c_1 c_2

Observe que essa execução concorrente também não seria produzida pelo protocolo 2PL básico, pois uma vez que as transações (T_1 e T_2) liberam o bloqueio obtido sobre o item x, essas transações entram na fase de encolhimento e não podem mais obter bloqueios. Assim, podemos observar que **o protocolo 2PL evita também o fenômeno da atualização perdida (*lost update*)**.

Contudo, caso as operações das transações T_1 e T_2 fossem enviadas a um escalonador 2PL seguindo a mesma ordem apresentada em S_1 teríamos uma execução S_2 na qual, para cada transação (T_1 e T_2), as liberações de bloqueios seriam realizadas em algum momento após a obtenção do último bloqueio. Assim a transação T_1, por exemplo, somente iria realizar uma liberação de bloqueio após obter seu último bloqueio. O mesmo ocorrendo com a transação T_2.

S_2 = $rl_1(x)$ $r_1(x)$ $rl_2(x)$ $r_2(x)$ $rl_1(y)$ $r_1(y)$ $wl_1(x)$ $wl_2(x)$ ...

Observando a execução concorrente S_2 podemos notar que a transação T_1, ao tentar obter um bloqueio de escrita sobre o item x ($wl_1(x)$), terá sua execução suspensa e ficará esperando até que a transação T_2 libere o bloqueio de leitura ($rl_2(x)$) anteriormente obtido sobre o item x. A transação T_2 segue sua execução.

Contudo, no momento em que a transação T_2 tenta obter um bloqueio de escrita sobre o item de dado x, T_2 terá sua execução suspensa e ficará esperando até que a transação T_1 libere o bloqueio de leitura ($rl_1(x)$) anteriormente obtido sobre o item x. Dessa forma, ocorrerá uma situação de impasse (*deadlock*) onde nenhuma das duas transações (T_1 e T_2) conseguirá prosseguir com sua execução. Logo, podemos concluir que **o protocolo de bloqueio em duas fases não garante a ausência de *deadlock*s**.

Exemplo 6

Considere as seguintes as seguintes transações:

$T_1=w_1(x,2)\ r_1(z)\ c_1$

$T_2= r_2(u)\ r_2(x)\ w_2(y,x+3)\ c_2$

Suponha agora a execução concorrente ilustrada pelo *schedule* S_1.

$S_1= rl_2(u)\ r_2(u)\ wl_1(x)\ w_1(x,2)\ u_1(x)\ rl_2(x)\ r_2(x)\ wl_2(y)\ w_2(y, x+3)\ u_2(u)\ u_2(x)\ u_2(y)\ c_2\ a_1$

Observe que o *schedule* S_1 pode ser gerado pelo protocolo 2PL básico, pois uma vez que a transação T_1 libera o bloqueio obtido sobre o item de dado x, essa entra na fase de encolhimento e não solicita mais nenhum bloqueio. Contudo, a execução S_1 não é confiável (recuperável) uma vez que $w_1(x,2) <_{S1} r_2(x)$ e c_1 não precede c_2 em S_1. Assim, o *abort* da transação T_1 deveria causar o *abort* de T_2, uma vez que T_2 leu um valor escrito por T_1. Contudo a transação T_1 já "commitou" (concluiu) e não pode mais ser desfeita. Logo, podemos concluir que **o protocolo 2PL básico pode gerar execuções (*schedules*) não confiáveis, ou seja, não recuperáveis.** Consequentemente, **o protocolo 2PL básico não evita *aborts* em cascata e nem gera execuções precisas (*strict*)**.

Contudo, o protocolo 2PL básico assegura a geração de execuções (*schedules*) serializáveis por conflito.

2.1.3 Protocolo de Bloqueio em Duas Fases Preciso (Strict)

O protocolo de bloqueio em duas fases severo exige, em adição ao bloqueio feito em duas fases, que **todos os bloqueios de modo exclusivo** tomados por uma transação sejam **mantidos até que a transação seja efetivada**. Assim, a transação está na fase de expansão até terminar. Essa exigência garante que qualquer dado escrito por uma transação que não foi ainda efetivada seja bloqueado de modo exclusivo até que a transação seja efetivada, evitando que qualquer outra transação lei os dados em transição. Assim, todos os bloqueios exclusivos são liberados somente após a operação

de *commit* ou *abort*.

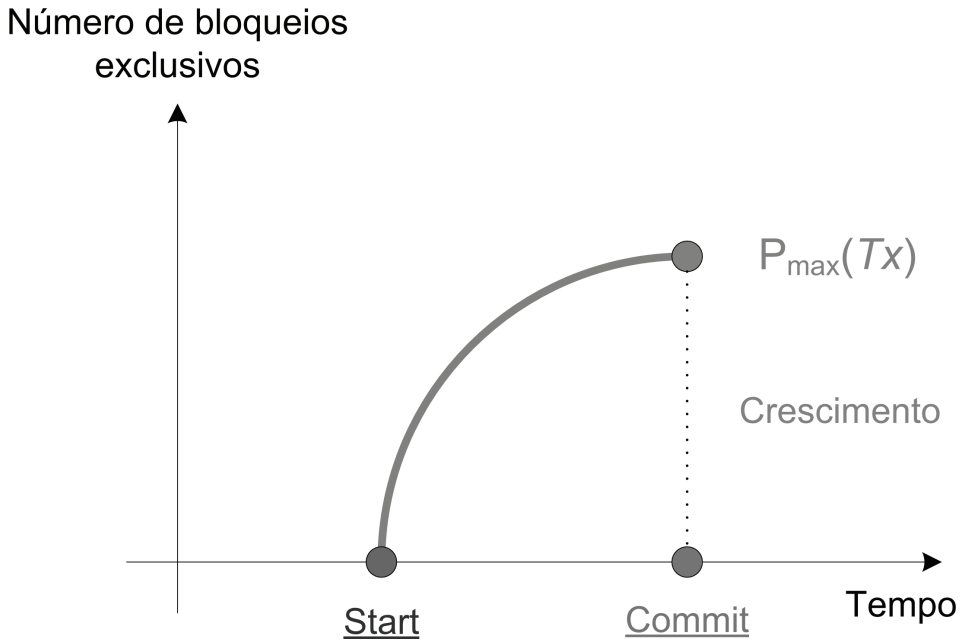


Figura 2.7 – Fases do protocolo 2PL Preciso.

A grande vantagem do protocolo 2PL Preciso é que esse gera execuções serializáveis e confiáveis (estritas). Contudo, esse protocolo não está livre da ocorrência de *deadlocks*.

Considere novamente a execução S_1 do exemplo 6:

S_1= $rl_2(u)$ $r_2(u)$ $wl_1(x)$ $w_1(x,2)$ $u_1(x)$ $rl_2(x)$ $r_2(x)$ $wl_2(y)$ $w_2(y, x+3)$ $u_2(u)$ $u_2(x)$ $u_2(y)$ c_2 a_1

Observe que esse *schedule* não seria gerado pelo protocolo 2PL Preciso, pois a operação $u_1(x)$ só poderia ser realizada após o *commit* ou *abort* de T_1. Consequentemente, a transação T_2 não conseguiria obter o bloqueio de leitura sobre o item x ($rl_2(x)$) e ficaria esperando até que a transação T_1 libere o bloqueio de escrita anteriormente obtido sobre o item x. Assim, a execução gerada pelo protocolo 2PL Preciso seria:

S_2= $rl_2(u)$ $r_2(u)$ $wl_1(x)$ $w_1(x,2)$ a_1 ...

2.1.4 Protocolo de Bloqueio em Duas Fases Rigoroso

Esse protocolo exige que os bloqueios (tanto compartilhados quanto exclusivos) sejam mantidos até que a transação seja efetivada. Pode facilmente ser verificado que, com o bloqueio em duas fases rigoroso, as transações podem ser serializadas na ordem de sua efetivação. A maioria dos sistemas de banco de dados implementa os dois tipos de bloqueios em duas fases. A desvantagem

desse protocolo é que o grau de concorrência é ainda mais limitado que no protocolo 2PL Preciso. Além disso, o protocolo 2PL Rigoroso não evita a ocorrência de *deadlocks*.

2.1.5 Protocolo de Bloqueio em Duas Fases Conservador

Nesse protocolo, também conhecido como 2PL Conservador ou 2PL Estático, uma transação deve bloquear todos os itens de dados que deseja antes de iniciar qualquer operação. Caso não seja possível bloquear todos os itens necessários nenhum bloqueio é realizado e a transação não inicia sua execução. Nesse caso, a transação deve esperar e tentar novamente em um momento futuro.

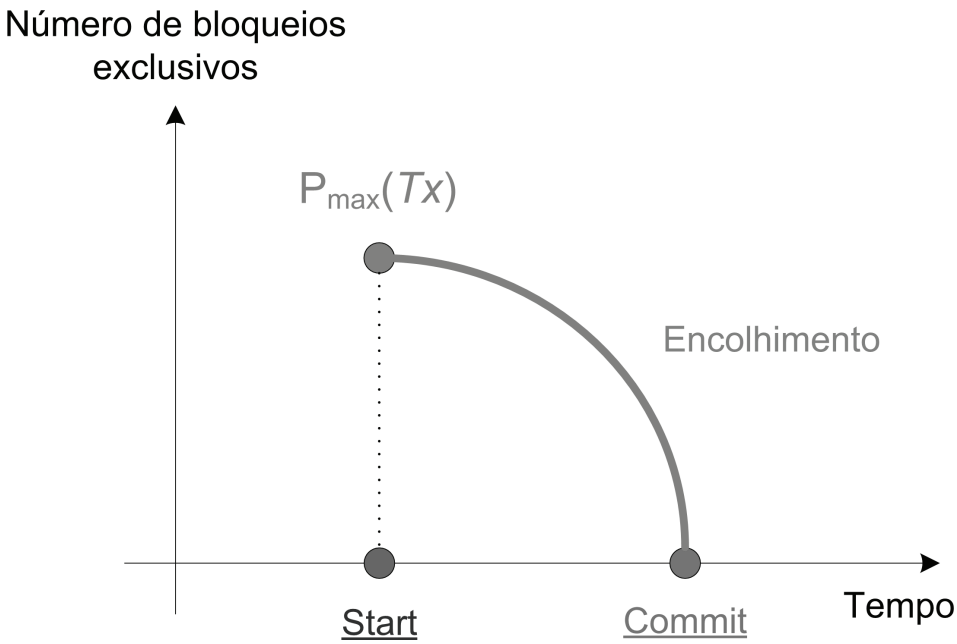


Figura 2.8. Fases do protocolo 2PL Conservador.

A grande vantagem do protocolo 2PL conservador é evitar a formação de *deadlocks*. Pois, quando uma transação inicia sua execução ela já detém todos os bloqueios de que necessita, logo ela não entrará em *deadlock* com nenhuma outra transação durante sua execução. Assim, quando a transação inicia, ela já está na fase de encolhimento. Contudo, o problema desse protocolo é que a espera pela aquisição de todos os bloqueios necessários pode apresentar um desempenho inviável na prática, o que decorre do baixo grau de concorrência proporcionado pelo protocolo.

2.2 Tratamento de Impasse (*deadlock*)

O protocolo 2PL Conservador assegura que os escalonamentos (*schedules*) gerados não apresentam situações de impasse (*deadlock*). Contudo, as restrições impostas pelo protocolo diminuem sobremaneira o grau de concorrência. Por esse motivo, o protocolo 2PL Conservador praticamente não é utilizado pelos SGBDs. Por outro lado, as demais versões do protocolo 2PL não estão livres das situações de impasse. Logo, torna-se necessário desenvolver e utilizar alguma técnica para tratar os impasses (*deadlocks*).

Nesse sentido, duas abordagens são possíveis:

- **Detecção e Resolução**: Essa abordagem não evita a ocorrência de *deadlocks*. Mas, pelo contrário, aceita que eles possivelmente irão ocorrer. Logo, será necessário, primeiramente, detectar a ocorrência de um *deadlock,* e, em seguida, usar alguma estratégia para solucionar o impasse.
- **Prevenção**: Essa abordagem utiliza técnicas que asseguram que uma situação de impasse (*deadlock*) nunca irá ocorrer.

2.2.1 Detecção e Resolução

1. Detecção por *Timeout*

Nessa técnica, é definido um tempo máximo pelo qual uma transação pode esperar por um bloqueio. Assim, se o tempo de espera de uma transação T_i por um determinado bloqueio atingir o tempo limite estipulado, o escalonador (*scheduler*) supõe que T_i esteja envolvida em uma *deadlock*.

Uma vez que o escalonar supõe que uma transação T_i esteja envolvida em uma situação de impasse, ele precisa solucionar (resolver) essa situação. A solução consiste em abortar a transação T_i.

Essa estratégia apresenta dois problemas principais:

- Pode ser que a transação T_i não estivesse envolvida em um *deadlock*;
- A detecção de *deadlocks* pode demorar longos períodos de tempo, ou seja, uma situação de *deadlock* só é percebida após o tempo máximo de esperar ser ultrapassado. Assim, a situação de impasse não é detectada no momento em que ela ocorre, e sim em um momento posterior.

2. Grafo de Espera (*Wait-For*)

Nessa estratégia, o escalonador mantém um grafo onde os nós representam as transações e as arestas são construídas da seguinte forma:

- Se uma transação T_i está esperando que outra transação T_j libere um bloqueio, então uma aresta $T_i \rightarrow T_j$ deve ser inserida no grafo.
- Se a inclusão da aresta $T_i \rightarrow T_j$ produzir um ciclo no grafo, T_i e T_j estão envolvidas em um *deadlock*.

A solução para a situação de impasse identificada consiste em abortar a transação mais recente. Ou seja, a transação mais nova é escolhida como vítima.

Exemplo 7

Considere o seguinte conjunto de transações:
$T_1 = rl_1(x)\ r_1(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ u_1(x)\ u_1(y)\ c_1$
$T_2 = rl_2(x)\ r_2(x)\ wl_2(x)\ w_2(x)\ u_2(x)\ c_2$

Considere agora a seguinte execução (*schedule*):
$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ wl_2(x)\ ...$

A figura 2.9 ilustra o grafo *wait-for* para o *Schedule* S_1. Observe que a transação T_1 ao tentar obter um bloqueio de escrita sobre o item x ($wl_1(x)$) irá esperar pela transação T_2, uma vez que T_2 já detém um bloqueio (leitura), $rl_2(x)$, em modo incompatível com o bloqueio desejado por T_1. Nesse momento, uma aresta $T_1 \rightarrow T_2$ será no grafo *wait-for*. Em seguida, a transação T_2 tenta obter um bloqueio de escrita sobre o item x ($wl_2(x)$). Contudo, como a transação T_1 já detém um bloqueio (leitura), $rl_1(x)$, em modo incompatível com o bloqueio desejado por T_2. Logo, T_2 vai esperar por T_1 e uma aresta $T_2 \rightarrow T_1$ será inserida no grafo *wait-for*. Porém, essa nova aresta introduz um ciclo no grafo *wait-for*. Assim, o escalonador irá detectar que T_1 e T_2 estão envolvidas em uma situação de impasse (*deadlock*). A transação T_2, por ser mais recente (iniciou sua execução após a transação T_1) será escolhida como vítima, e, consequentemente, será abortada.

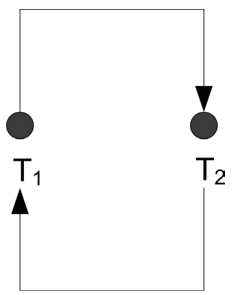


Figura 2.9 – Grafo *Wait-For* para o *Schedule* S_1 no Exemplo 7.

2.2.2 Prevenção de Impasses (*Deadlocks*)

1. Bloqueio de Atualização (*Update*)

A estratégia do bloqueio de atualização (*update*) surgiu da constatação de que 90% das situações de impasse (*deadlocks*) eram produzidos pela conversão de bloqueios de leitura em bloqueios de escrita, o que ocorre quando uma transação obtém um bloqueio de leitura, efetua a leitura e, posteriormente, tenta obter um bloqueio de escrita sobre o mesmo item de dado. A ideia básica dessa técnica consiste em utilizar um novo tipo de bloqueio: o bloqueio de atualização (ou *update*). Um bloqueio de atualização consiste em um bloqueio de leitura com a intenção de executar uma operação de escrita no futuro. O objetivo é reduzir as conversões de bloqueio de leitura para bloqueio de escrita. A figura 2.10 mostra a tabela de compatibilidade de bloqueios considerando agora a operação "bloqueio de atualização", representada por ul_i.

●	rl_i	ul_i	wl_i
rl_j	+	-	-
ul_j	+	-	-
wl_j	-	-	-

Figura 2.10 – Tabela de Compatibilidade de Bloqueios.

Observe como ficaria a execução discutida no Exemplo 7 se o conceito de bloqueio de atualização fosse utilizado:

$T_1 = ul_1(x)\ r_1(x)\ rl_1(y)\ r_1(y)\ w_1(x)\ u_1(x)\ u_1(y)\ c_1$

$T_2 = ul_2(x)\ r_2(x)\ w_2(x)\ u_2(x)\ c_2$

$S_1 = ul_1(x)\ r_1(x)\ ul_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ u_1(x)\ u_1(y)\ c_1\ r_2(x)\ w_2(x)\ u_2(x)\ c_2$

Note que a transação T_2, ao tentar obter um bloqueio de atualização sobre o item de dado x ($ul_2(x)$), ficará esperando pela transação T_1, uma vez que T_1 já detém um bloqueio sobre o item x ($ul_1(x)$) incompatível com o tipo de bloqueio solicitado por T_2 ($ul_2(x)$). Assim, nesse momento, T_2 ficará suspensa até que a transação T_1 libere seu bloqueio sobre o item x. Contudo, a transação T_1 continua sua execução normalmente. Já a transação T_2 só continuará sua execução quando a transação T_1 entrar na fase de encolhimento e liberar o

bloqueio sobre o item x. Logo, a situação de impasse não ocorrerá.

Podemos perceber que essa estratégia evita a formação de *deadlocks*. Contudo, reduz substancialmente o grau de concorrência.

1. Solução por *Timestamp*

A ideia dessa abordagem para evitar a formação de *deadlocks* baseia-se na utilização de marcadores de tempo (*timestamps*). A seguir descreveremos mais detalhadamente essa técnica.

Primeiramente, precisamos definir uma ordenação temporal entre as transações. Para isso, cada transação T_i possui um marcador de tempo (*timestamp*), representado por $ts(T_i)$, o qual irá representar a ordem em que as transações iniciaram suas execuções. Dessa forma:

- Se T_i inicia sua execução antes de T_j então:
  - $ts(T_i) < ts(T_j)$

Ou seja, nesse caso, T_i é uma transação mais antiga que T_j.

Quando uma transação T_i solicitar um bloqueio sobre um item de dado já bloqueado por T_j, duas estratégias são possíveis:

2.2.2.1 wait-die:

Se $ts(T_i) < ts(T_j)$ então //T_i é mais antiga que T_j
T_i espera
Se não //T_i é mais recente que T_j
Abortar T_i //Transação mais recente é “abortada”.
Fim Se

Nessa estratégia, as transações mais antigas podem esperar por uma transação mais recente. Já as transações mais novas não esperam por uma transação mais antiga, sendo canceladas. Assim, à medida que uma transação fica mais antiga, ela tende a esperar pelas transações mais jovens.

2.2.2.2 wound-wait:

Se $ts(T_i) < ts(T_j)$ então //T_i é mais antiga que T_j
Abortar T_i //Transação mais recente é “abortada”.
Se não //T_i é mais recente que T_j
T_i espera
Fim Se

Nessa estratégia, as transações mais recentes podem esperar por uma transação mais antiga. Já as transações mais antigas não esperam por uma transação mais nova, nesse caso, as transações mais antigas provocam o *abort* das transações mais jovens.

Vale destacar que nas duas estratégias sempre a transação mais recente é escolhida como vítima. Assim, as transações mais antigas são

priorizadas. Essa escolha decorre da expectativa que as transações mais jovens tenham executado uma quantidade menor de operações, logo o trabalho a ser refeito deve ser menor. Contudo, essa escolha apresenta o risco de que uma determinada transação T_j (jovem) seja sempre escolhida como vítima, sendo sempre reiniciada, ao envolver-se em *deadlocks* com transações mais antigas. Assim, T_j poderia ser abortada continuamente, fenômeno denominado de ***livelock***, ou enfraquecimento.

A solução para essa situação consiste em ao reiniciar uma transação T_j não reiniciar o seu *timestamp* ($ts(T_j)$), ou seja, manter um único *timestamp* para cada transação. Dessa forma, é razoável esperar que as transações mais antigas que T_j em algum momento concluam sua execução, ficando T_j como a transação mais antiga, passando a ter prioridade sobre as transações mais recentes que ela.

Exemplo 8

Considere o seguinte conjunto de transações:

$T_1 = rl_1(x)\ r_1(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ u_1(x)\ u_1(y)\ c_1$

$T_2 = rl_2(x)\ r_2(x)\ wl_2(x)\ w_2(x)\ u_2(x)\ c_2$

Considere agora a seguinte execução (*schedule*):

$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ wl_2(x)$ ...

Vamos assumir que:

- $ts(T_1) = 1$
- $ts(T_2) = 2$

Assim, a transação T_1 é mais antiga que T_2.

Supondo que a estratégia ***wait-die*** seja utilizada teríamos:

$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ wl_2(x)\ a_2$ ...

Nesse exemplo, vale destacar que quando uma transação T_1 solicita um bloqueio de escrita sobre o item de dado x ($wl_1(x)$), esse já está bloqueado por T_2 ($rl_2(x)$) em um modo incompatível. Nesse caso, teremos:

$ts(T_1) < ts(T_2)$

Logo, T_1 irá esperar por T_2, ou seja, T_1 irá esperar até que T_2 libere o seu bloqueio sobre x ($rl_2(x)$). A transação T_2 prossegue sua execução.

Quando a transação T_2 solicita um bloqueio de escrita sobre o item de dado x ($wl_2(x)$), esse já está bloqueado por T_1 ($rl_1(x)$) em um modo incompatível. Nesse caso, teremos:

$ts(T_2) < ts(T_1)$

Logo, T_2 será cancelada (abortada) e seus bloqueios liberados. O escalonamento gerado é mostrado a seguir.

$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ wl_2(x)\ a_2\ w_1(x)\ u_1(x)\ u_1(y)\ c_1\ rl_2(x)$

$r_2(x)$ $wl_2(x)$ $w_2(x)$ $u_2(x)$ c_2

Agora, suponha que a estratégia ***wound-wait*** seja utilizada.

Nesse caso, quando uma transação T_1 solicita um bloqueio de escrita sobre o item de dado x ($wl_1(x)$), esse já está bloqueado por T_2 ($rl_2(x)$) em um modo incompatível. Nesse caso, teremos:

$ts(T_1) < ts(T_2)$

Logo, a transação T_2 será cancela (abortada) e o escalonamento (*schedule*) gerado é mostrado a seguir.

$S_1 = rl_1(x)$ $r_1(x)$ $rl_2(x)$ $r_2(x)$ $rl_1(y)$ $r_1(y)$ $wl_1(x)$ a_2 $w_1(x)$ $u_1(x)$ $u_1(y)$ c_1 $rl_2(x)$ $r_2(x)$ $wl_2(x)$ $w_2(x)$ $u_2(x)$ c_2

Assim, podemos observar que as duas abordagens evitam a formação de *deadlocks*. Contudo, ambas aumentam de forma significativa a quantidade de *aborts*.

2.3 Granularidade de Bloqueios

Nos protocolos de controle de concorrência descritos até agora, cada operação de bloqueio era realizada sobre um único item de dado, de forma individual. Assim, para bloquear uma tabela inteira com 2000 itens de dados serão necessários no mínimo 2000 bloqueios (uma vez que um determinado item de dado pode estar bloqueado de forma compartilhada, leitura, por mais de uma transação). Logo, a tabela de bloqueios gerenciada pelo escalonador deverá ter no mínimo 2000 entradas, o que gera uma sobrecarga (*overhead*) considerável.

Dessa forma, há circunstâncias em que pode ser vantajoso o agrupamento de diversos itens de dados, a fim de tratá-los como uma unidade de sincronização individual. Por exemplo, para bloquear uma tabela por completo poderia ser utilizada uma única operação de bloqueio, o que reduziria o *overhead* de gerenciamento da tabela de bloqueios. Nesse caso, a granularidade do bloqueio seria a tabela inteira. Por outro lado, se uma transação T_i precisa acessar somente alguns itens de dados, não é necessário bloquear toda a tabela, porque, desse modo, estaríamos limitando a concorrência.

Assim, torna-se necessário definir um mecanismo que permita ao SGBD utilizar níveis múltiplos de granulação (ou granularidade). O ponto de partida para isso é a identificação das diferentes granularidades (figura 2.11).

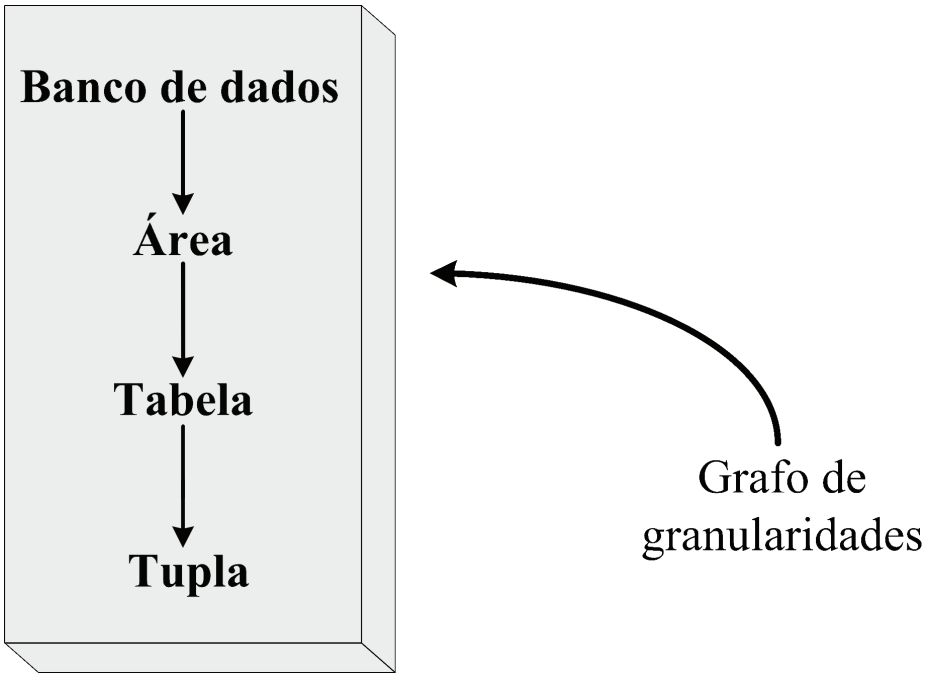


Figura 2.11 – Relacionamento entre as Diferentes Granularidades.

A ideia principal consiste em que o bloqueio de um determinado objeto (nó) no grafo de granularidades implica que seus descendentes são implicitamente bloqueados (com o mesmo tipo de bloqueio). Além disso, propagam-se para os seus ascendentes os efeitos do bloqueio.

A fim de ilustração considere a árvore da figura 2.12, a qual consiste em quatro (4) níveis de nós. O nível mais alto representa o banco de dados (BD) como um todo. Abaixo, há nós do tipo área; o BD é constituído exatamente dessas áreas. Cada área, por sua vez, possui nós do tipo arquivo/relação/tabela como filhos. Cada área é constituída exatamente daquelas tabelas que são seus nós filhos. Nenhuma tabela está em mais de uma área. Finalmente, cada tabela possui nós do tipo registro/*tupla*. Nenhum registro pode estar em mais de um arquivo.

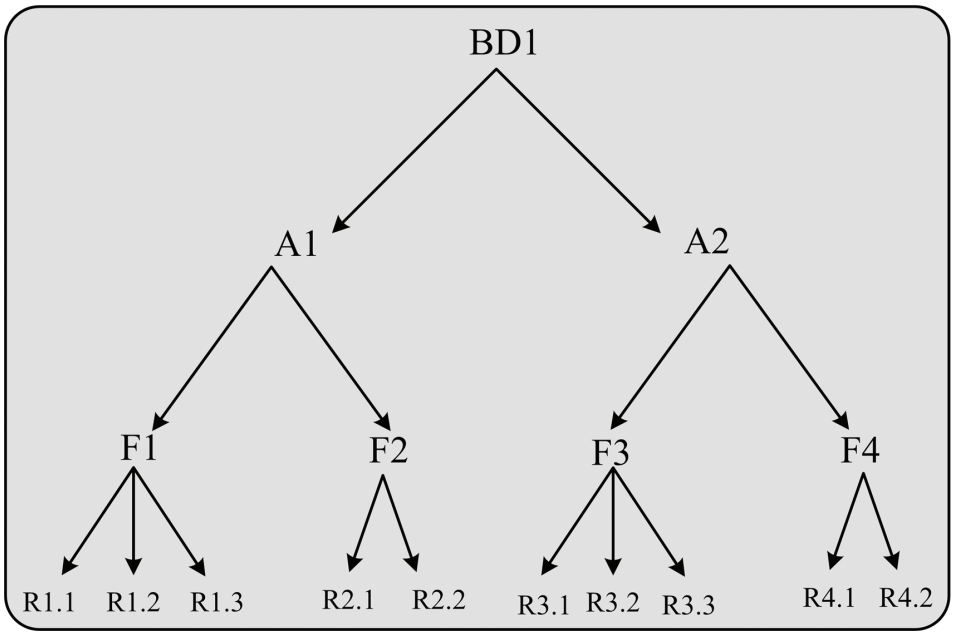


Figura 2.12 – Estrutura do Banco de Dados.

A partir dessa ideia foi concebido o **protocolo de bloqueio em duas fases com granularidade múltipla**. Nesse protocolo, cada nó de uma árvore pode ser bloqueado de forma individual. Além disso, duas novas operações de bloqueio são adicionadas:

- **Bloqueio intencional de leitura (irl);**
- **Bloqueio intencional de escrita (iwl);**

Dessa forma, uma nova tabela de compatibilidade de bloqueios necessita ser definida (figura 2.13).

●	rl_i	wl_i	irl_i	iwl_i
rl_j	+	-	+	-
wl_j	-	-	-	-
irl_j	+	-	+	+
iwl_j	-	-	+	+

Figura 2.13 – Tabela de Compatibilidade de Bloqueios.

Um bloqueio intencional (ou de advertência) de leitura representa a intenção de obter um bloqueio compartilhado sobre um elemento secundário. As regras do protocolo são:

1) Para impor um bloqueio de leitura ou escrita sobre um determinado elemento (nó) N (por exemplo, a relação R2.1), devemos começar pela raiz da hierarquia (por exemplo, BD1).
2) Se estamos no elemento N que queremos bloquear (por exemplo, a relação R2.1), não precisamos examinar mais nada. Solicitamos o bloqueio desejado sobre esse elemento.
3) Se o elemento que desejamos bloquear estiver mais abaixo na hierarquia, devemos inserir uma advertência (intenção) no nó corrente. Isto é, se estamos no nó raiz, representado por BD1, e quisermos obter um bloqueio compartilhado sobre um elemento que está mais abaixo na hierarquia (por exemplo, R2.1), deve-se solicitar um bloqueio intencional de leitura sobre o nó BD1 e, se quisermos um bloqueio exclusivo sobre R2.1, solicitaremos um bloqueio intencional de escrita sobre BD1. Quando o bloqueio sobre o corrente (BD1) for concedido, prosseguiremos até chegar ao elemento desejado, no nosso exemplo R2.1. Vale observar que para que seja concedido um bloqueio compartilhado sobre o elemento R2.1 será necessário obter bloqueios intencionais de leitura sobre os nós BD1, A1 e F2.

A seguir discutimos alguns aspectos do protocolo:

- Quando solicitamos um bloqueio intencional de leitura sobre um nó N (por exemplo, BD1), pretendemos ler um dos nós descendentes de N (ou seja, abaixo de N na hierarquia), por exemplo R2.1. O único momento em que essa intenção de leitura (bloqueio intencional de leitura sobre BD1) poderia criar um problema (de incompatibilidade de bloqueios) é se alguma outra transação já tivesse reivindicado o direito de gravar uma nova cópia do elemento do BD1 inteiro, uma vez que um bloqueio intencional de leitura só é incompatível com um bloqueio de escrita sobre o mesmo elemento. Observe que, se alguma outra transação tiver a intenção de gravar apenas um elemento secundário na hierarquia (por exemplo, R2.2), indicado por um bloqueio intencional de escrita em BD1, então poderemos ter condições de conceder o bloqueio intencional de leitura em BD1 e permitir que o conflito seja resolvido em um nível mais baixo, se de fato a intenção de gravação e a intenção de leitura envolverem um elemento comum.

- Quando se pretende gravar um elemento secundário na hierarquia do nó N, devemos evitar a leitura e gravação do elemento inteiro representado por N. Contudo, um bloqueio intencional de escrita não entrará em conflito com outro bloqueio intencional de escrita em N ou com um bloqueio intencional de leitura em N.
- A operação de leitura sobre o elemento correspondente ao nó N não pode entrar em conflito com outro bloqueio de leitura sobre N ou com um bloqueio de leitura sobre um elemento secundário de N, representado por um bloqueio intencional de leitura em N. Porém, um bloqueio de escrita ou um bloqueio intencional de escrita implica que alguma outra transação gravará pelo menos uma parte do elemento representado por N. Portanto, não poderemos conceder o direito de ler N inteiro. Assim, tanto bloqueios de escrita quanto bloqueios intencionais de escrita são incompatíveis com bloqueios de leitura.
- Não podemos permitir a gravação de todo o nó N, se qualquer outra transação já tiver o direito de ler ou gravar N, ou já tiver adquirido esse direito sobre um elemento secundário (abaixo de N na hierarquia).

Resumindo, o protocolo de bloqueio de granularidade múltipla garante a serialização. Cada transação T_i pode bloquear um nó Q, usando as seguintes regras:

1. A função de compatibilidade precisa ser observada.
2. A raiz da árvore precisa ser bloqueada primeiro, e pode ser bloqueada em qualquer modo.
3. Um nó Q pode ser bloqueado por T_i no modo leitura ou intencional de leitura somente se o pai de Q for bloqueado por T_i no modo intencional de escrita ou intencional de leitura.
4. Um nó Q pode ser bloqueado por T_i no modo escrita ou intencional de escrita somente se o pai de Q for bloqueado por T_i no modo intencional de escrita.
5. T_i pode bloquear um nó somente se ele não desbloqueou outro nó anteriormente (isto é, T_i tem duas fases).
6. T_i pode desbloquear um nó Q somente se nenhum dos filhos de Q estiver bloqueado por T_i. A liberação de bloqueios começa de baixo para cima na hierarquia.

Vale destacar que o protocolo 2PL com múltipla granularidade exige que os bloqueios sejam feitos de cima para baixo (*top-down* – da raiz para as folhas), enquanto a liberação deve ser de baixo para cima (*bottom-up* – das folhas para a raiz).

Exemplo 9

Considere a tabela a seguir:

FILME (Título, Ano, Duração, Produtora)

Agora, considere as seguintes transações:

T_1 :	T_2 :
SELECT * FROM Filme WHERE Título = "Filme1";	**UPDATE Filme SET Ano = 1939 WHERE Título = "Filme2";**

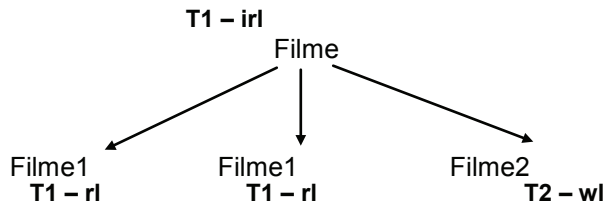


Figura 2.14 – Exemplo de Bloqueio com Granularidade Múltipla.

A transação T_1 começa obtendo um bloqueio intencional de leitura sobre a relação inteira. Em seguida, ele se desloca para as *tuplas* individuais (há 2 filmes com o nome Filme1) e obtém o bloqueio de leitura sobre cada uma delas.

Suponha que enquanto a transação T_1 está sendo executada, a transação T_2, que altera o ano de uma *tupla*, inicie sua execução. Nesse caso, T_2 precisa de um bloqueio intencional de escrita sobre a relação, pois planeja gravar um novo valor para uma das *tuplas*. O bloqueio intencional de leitura de T_1 sobre a relação é compatível, e assim o bloqueio é concedido. Quando T_2 chega na *tupla* Filme2, não encontra nenhum bloqueio lá, e assim recebe seu bloqueio de escrita e atualiza a *tupla*. Caso T_2 tentasse gravar um novo valor na *tupla* para um dos filmes Filme1, ela teria de esperar até T_1 liberar seu bloqueio de leitura, pois leitura e escrita não são compatíveis.

2.4 Bloqueio em Duas Fases com Múltiplas Versões

A ideia principal dos protocolos baseados em múltiplas versões consiste em reduzir a taxa de bloqueios através da utilização de mais de uma versão de cada item de dado. A seguir descreveremos o protocolo baseado em múltiplas versões mais popular: o protocolo de bloqueio em duas fases com duas versões (*two version two phase lock* – 2V2PL).

2.4.1 Protocolo 2V2PL

- **Princípios de Funcionamento**

Os princípios gerais que norteiam o protocolo 2V2PL são discutidos a seguir:

- Para cada objeto x duas versões são mantidas: x e x_i
- Uma operação de escrita de uma transação T_k sobre um objeto x gera uma nova versão x_i de x
- Para executar a operação de escrita:
  - T_k obtém primeiro um bloqueio de escrita sobre x para prevenir que nenhuma outra transação
    - Leia x_i ou
    - Escreva uma nova versão de x
  - Qualquer outra transação pode ler x
- Quando T_k executa seu *commit*, x_i torna-se a única versão de x
  - A versão anterior torna-se inacessível

- **Vantagens**

Podemos observar que o protocolo 2V2PL apresenta duas vantagens principais, são elas:

- Leituras sobre x não são atrasadas devido a uma escrita sobre x
- Escritas sobre x não são atrasadas devido a uma leitura sobre x

- **Implementação do Protocolo 2V2PL**

A implementação do protocolo 2V2PL baseia na utilização de um novo tipo de bloqueio: o bloqueio de certificação (ou *certify lock*), denotado por $cl_i(x)$. A compatibilidade do bloqueio de certificação em relação aos bloqueios de leitura (compartilhado) e escrita (exclusivo) é apresentada na figura 2.15.

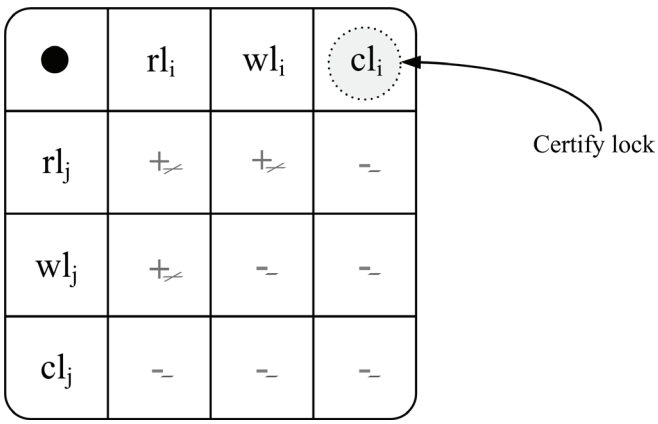


Figura 2.15 – Tabela de Compatibilidade de Bloqueios do Protocolo 2V2PL.

Para assegurar a geração de escalonamentos (*schedules*) precisos, os bloqueios só são liberados após a operação de *commit*. A Figura 2.16 ilustra um

algoritmo que pode ser utilizado para implementar o protocolo 2V2PL.

1. Quando o escalonador recebe uma operação de escrita $w_j(x)$

Tenta obter $wl_j(x)$ conforme a tabela de compatibilidade de bloqueios
Se existe $wl_i(x)$ ou $cl_i(x)$ ***então:***
O escalonador retarda $wl_j(x)$
Se não
Concede o bloqueio $wl_j(x)$
Converte $w_j(x)$ em $w_j(x_n)$ e envia $w_j(x_n)$ para processamento
Fim Se

2. Quando o escalonador recebe uma operação de leitura $r_j(x)$
Tenta obter $rl_j(x)$
Se $rl_j(x)$ for obtido **então**:
Se T_j já possuía $wl_j(x)$ e já executou $w_j(x_n)$ ***então:***
Mapear $r_j(x)$ em $r_j(x_n)$ e enviar $r_j(x_n)$ para processamento
Se não
Enviar $r_j(x)$
Fim Se
Fim Se

3. Quando o escalonador recebe uma operação de *commit* (c_j)
Tenta converter (*upgrade*) todos os wl_j em cl_j segundo a tabela de compatibilidade
Se existe $rl_k(x)$, com $0 < K \leq n$ e $k \neq j$, **então**:
Atrasar $cl_j(x)$ até $rl_k(x)$ seja liberado por T_k
Fim Se

- **Comentários**

O protocolo 2V2PL produz escalonamentos (*schedules*) serializáveis por conflito e precisos, ou seja, histórias corretas e que evitam *aborts* em cascata. Além disso, esse protocolo apresenta um grau de paralelismo maior, uma vez que um mesmo item pode ser lido e escrito por transações diferentes de forma simultânea. Por esses motivos, o protocolo 2V2PL ganhou enorme popularidade e, atualmente, já é implementado pelos principais SGBDs (Oracle, PostgreSQL, SQLServer, dentre outros).

Exemplo 10

Considere o seguinte conjunto de transações:

$T_1 = rl_1(x)\ r_1(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ c_1\ u_1(x)\ u_1(y)$

$T_2 = rl_2(x)\ r_2(x)\ rl_2(y)\ r_2(y)\ c_2\ u_2(x)\ u_2(y)$

Considere agora a seguinte execução (*schedule*):

$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ rl_2(y)\ r_2(y)\ c_2\ u_2(x)\ u_2(y)\ c_1\ u_1(x)\ u_1(y)$

O *Schedule* S_1 pode ser gerado por um escalonador que utiliza o protocolo 2V2PL. Observe que a operação $w_1(x)$ não será atrasada devido a existência (execução prévia) da operação $r_2(x)$. Note também que quando a transação T_1 tentar executar o seu *commit*, o bloqueio de escrita $wl_1(x)$ poderá ser convertido em um bloqueio de certificação (*certify lock*) uma vez que a transação T2 já executou o seu *commit* e, consequentemente, já liberou o bloqueio $rl_2(x)$.

Considere agora o *schedule* S_2.

$S_2 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ c_1$ ...

Observe que nesse exemplo a operação de *commit* da transação T_1 chegou ao escalonador antes do *commit* da transação T_2. Nesse caso, o escalonador irá tentar converter o bloqueio de escrita $wl_1(x)$ em um bloqueio de certificação $cl_1(x)$. Contudo, essa conversão não poderá ser realizada uma vez que existe um bloqueio, anteriormente obtido pela transação T_2 ($rl_2(x)$), o qual é incompatível com $cl_1(x)$. Nesse caso, a transação T_1 irá esperar pelo *commit* da transação T_2. Após o *commit* de T_2, a transação T_1 poderá realizar o seu *commit*, o que resultará no *schedule* S_1.

Vale destacar que o *schedule* S_1 não seria gerado por um protocolo 2PL Estrito (apesar de ser um escalonamento correto), uma vez que a operação $wl_1(x)$ não seria executada enquanto a transação T_2 não executasse o seu *commit*, liberando o bloqueio $rl_2(x)$. No caso de utilização do protocolo 2PL estrito o *schedule* gerado seria S_3. Assim, podemos concluir que o protocolo 2V2PL permite um grau de concorrência (entrelaçamentos) maior que o protocolo 2PL Estrito.

$S_3 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ rl_2(y)\ r_2(y)\ c_2\ u_2(x)\ u_2(y)\ wl_1(x)\ w_1(x)\ c_1\ u_1(x)\ u_1(y)$

Exemplo 11

Considere o seguinte conjunto de transações:

$T_1 = rl_1(x)\ r_1(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ c_1\ u_1(x)\ u_1(y)$

$T_2 = rl_2(x)\ r_2(x)\ rl_2(y)\ r_2(y)\ wl_2(y)\ w_2(y)\ c_2\ u_2(x)\ u_2(y)$

Considere agora a seguinte execução (*schedule*):

$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ rl_1(y)\ r_1(y)\ wl_1(x)\ w_1(x)\ rl_2(y)\ r_2(y)\ wl_2(y)\ w_2(y)\ c_2\ c_1$ ...

Observe que no momento em que a transação T_2 solicita a execução da sua operação de commit, o escalonador irá tentar converter o bloqueio de escrita $wl_2(y)$ em um bloqueio de certificação $cl_2(y)$. Contudo, essa conversão não será realizada, pois existe um bloqueio incompatível, nesse caso $rl_1(y)$. Logo, a transação T_2 irá esperar até que a transação T_1 libere o bloqueio $rl_1(y)$. Em seguida, a transação T_1 solicita o seu *commit* e o escalonador irá tentar

converter o bloqueio de escrita $wl_1(x)$ em um bloqueio de certificação $cl_1(x)$. Porém, essa conversão não será realizada, pois existe um bloqueio incompatível, nesse caso $rl_2(x)$. Logo, a transação T_1 irá esperar até que a transação T_2 libere o bloqueio $rl_2(x)$. Dessa forma, a situação de impasse (*deadlock*) está formada. Logo, podemos concluir que o protocolo 2V2PL não está livre da ocorrência de *deadlocks*.

Exemplo 12

Considere o seguinte conjunto de transações:

$T_1 = rl_1(x)\ r_1(x)\ wl_1(x)\ w_1(x)\ c_1\ u_1(x)$

$T_2 = rl_2(x)\ r_2(x)\ wl_2(x)\ w_2(x)\ c_2\ u_2(x)$

Considere agora a seguinte execução (*schedule*):

$S_1 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ wl_1(x)\ w_1(x)\ wl_2(y)\ w_2(y)\ c_1\ c_2$

Observe que o *Schedule* S_1 não seria gerado pelo protocolo 2V2PL, uma vez que o bloqueio de escrita $wl_2(y)$ não seria concedido devido à existência de um bloqueio incompatível ($wl_1(x)$). Nesse caso, a transação T_2 seria atrasada até que a transação T_1 fosse concluída. Assim, o protocolo 2V2PL iria gerar o *schedule* S_2.

$S_2 = rl_1(x)\ r_1(x)\ rl_2(x)\ r_2(x)\ wl_1(x)\ w_1(x)\ c_1$

Contudo, a transação T_1 ao tentar executar seu *commit* ficaria esperando pela conclusão de T_2, uma vez que o escalonador não iria conseguir converter o bloqueio de escrita $wl_1(x)$ em bloqueio de certificação $cl_1(x)$ devido à existência de um bloqueio incompatível ($rl_2(x)$). Novamente, temos uma situação de *deadlock*.

2.5 Arquitetura de Gerenciadores de Transações que Utilizam 2PL

O gerenciador de transações é o módulo do SGBD responsável por gerenciar as transações. A figura 2.14 ilustra uma possível arquitetura para um gerenciador de transações.

A interface de transações é o componente responsável por gerar o identificador (TRID) de cada transação.

O gerenciador de bloqueios (*lock manager*) tem objetivo de garantir que as requisições de bloqueios efetuadas pelos usuários sejam atendidas de forma eficiente, além de proporcionar alto grau de concorrência. Todo comando SQL, uma vez preparado para execução, precisa interagir com o *Lock Manager*, para informar o recurso que precisa ser bloqueado, caso os recursos necessários não possam ser bloqueados, o *Lock Manager* envia uma notificação de espera. Ao final de uma transação, interage-se com o *Lock Manager* avisando-o que bloqueios podem ser liberados.

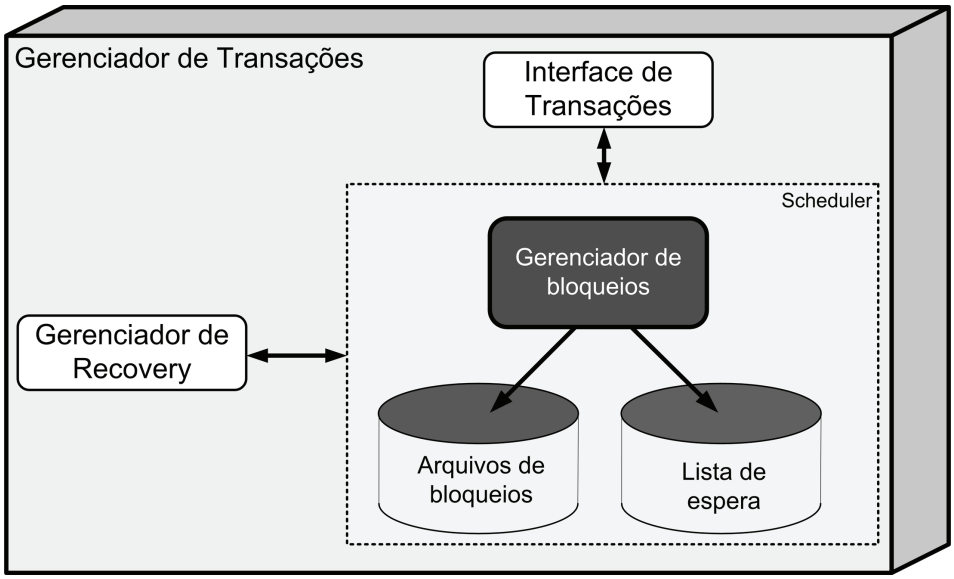


Figura 2.16 – Arquitetura de um Gerenciador de Transações.

O *Lock Manager* também gerencia uma estrutura de dados interna onde são armazenadas as informações sobre os recursos que estão bloqueados (identificador da transação, tipo de bloqueio, identificador do objeto bloqueado etc) e uma lista de espera, a qual indica que transações estão esperando por quais recursos.

Assim, o gerenciador de bloqueios executa as operações de bloqueio e de desbloqueio (liberação), além de gerenciar duas estruturas de dados:

- **Arquivo de Bloqueios**: Arquivo que contém as informações sobre os bloqueios. Cada entrada nesse arquivo armazena: o identificador da transação (TRID), o tipo de bloqueio e o identificador do objeto bloqueado (OID).
- **Lista de Espera**: Arquivo utilizado para armazenar as informações das transações que estão suspensas, esperando pela liberação de algum bloqueio.

Já o componente gerenciador de recuperação (*recovery*) é responsável por recuperar o estado consistente do banco de dados após uma situação de falha.

2.6 Níveis de Isolamento

O nível de isolamento é uma propriedade que define quando e como uma mudança (alteração) feita por uma transação se torna visível para outras transações concorrentes. O Isolamento é também uma das propriedades ACID.

A maioria dos SGBDs permite definir o nível de isolamento das

transações, permitindo assim controlar a relação entre o grau de concorrência e o nível de consistência desejado.

Um nível de isolamento define:

- **Escopo do bloqueio;**
- **Duração do bloqueio.**

Quanto ao escopo um bloqueio pode ser:

- **Bloqueio de objeto**: Bloqueia um objeto (*tupla*, por exemplo) específico do banco de dados;
- **Bloqueio de predicado**: Bloqueia um conjunto de objetos que satisfazem uma determinada condição. Assim, bloqueiam objetos existentes no banco de dados, objetos ainda não existentes no banco de dados, mas que satisfariam o predicado caso fossem incluídos no banco de dados, e, objetos a serem alterados, que passariam a satisfazer o predicado após a alteração.

Quanto à duração um bloqueio pode ser:

- **Bloqueio de curta duração**: O bloqueio é liberado imediatamente após a execução da operação associada ao bloqueio.

  A seguir mostramos um exemplo de bloqueio de objeto de curta duração:

  … $rl_i(x)$ $r_i(x)$ $u_i(x)$ …c_i

  O exemplo a seguir ilustra a utilização de um bloqueio de predicado de curta duração:

  … $rl_i(x \text{ in } P)$ $r_i(x \text{ in } P)$ $u_i(x \text{ in } P)$ … c_i

- **Bloqueio de longa duração**: O bloqueio é retido até que a operação de *commit* ou *abort* seja executada.

  A seguir apresentamos um exemplo de objeto de longa duração:

  … $rl_i(x)$ $r_i(x)$ …c_i $u_i(x)$

  O exemplo a seguir ilustra a utilização de um bloqueio de predicado de longa duração:

  … $rl_i(x \text{ in } P)$ $r_i(x \text{ in } P)$ … c_i $u_i(x \text{ in } P)$

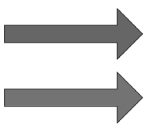
Nesse último exemplo, nenhuma outra transação T_J pode inserir ou atualizar objetos que satisfaçam o predicado P, até que T_i libere o bloqueio de leitura sobre P, o que somente ocorre após o *commit* de T_i.

Exemplo 13

Agora, observe um exemplo com a utilização de um cursor onde o bloqueio utilizado é de curta duração para objetos e de longa duração para predicados:

```
Select *
from Empregado
where salario>499
…
Commit
```

Estrutura de cursor



05	André	5000	1
08	Bárbara	2000	2
10	Inês	7000	1
15	Sofia	500	1
16	Natalya	2000	2
20	Caio	500	1
22	Lucas	550	1
30	Rebeca	550	2

Note que no momento que o cursor se mover da primeira *tupla* (05,'André', 5000, 1) para a segunda *tupla* (08, 'Bárbara', 2000, 2), a primeira *tupla* não estará mais bloqueada, pois o bloqueio utilizado é de curta duração para objetos. Contudo, uma nova *tupla* do tipo (07, 'José',500,1) não poderia ser inserida uma vez que essa nova *tupla* satisfaz o predicado utilizado no bloqueio (salario>499), o qual é de longa duração para predicado. Já se a *tupla* fosse (07, 'José', 499,1) a inserção seria permitida, uma vez que essa não satisfaz a condição utilizada no bloqueio.

2.6.1 Descrição dos Níveis de Isolamento

Existem quatro níveis de isolamento definidos no padrão ANSI/ ISO SQL. São eles: *Read Uncommitted, Read Committed, Repeatable Read* e o *Serializable*. Esses níveis são classificados de acordo com a possibilidade de ocorrência de determinados fenômenos indesejados, que podem ser "d*irty reads*", "*non-repeatable reads*" e "*phantons".* A figura a seguir ilustra o relacionamento entre os níveis de isolamento e os fenômenos.

	Dirty Read	Non-Repeatable Read	Phantom
Read Uncommitted	Sim	Sim	Sim
Read Committed	Não	Sim	Sim
Repeatable Read	Não	Não	Sim
Serializable	Não	Não	Não

Figura 2.17 – Níveis de Isolamento ANSI/ISSO SQL

A seguir, descrevemos cada um desses fenômenos:

Dirty Read – Ocorre quando uma transação (T_1) modifica determinada informação e, em seguida, outra transação (T_2) lê a mesma informação antes que T_1 realize sua operação de *commit* ou *rollback (abort)*. Assim, caso a transação T_1 seja cancelada ("abortada"), T_2 terá lido uma informação que nunca chegou a existir "oficialmente" na base de dados.

Non-Repeatable Read – Ocorre quando uma transação T_1 lê um determinado item de dado. Posteriormente, outra transação T_2 apaga ou modifica esse item. Em seguida, quando a transação T_1 tenta ler novamente o mesmo item de dado lido anteriormente, essa vai observar um valor diferente do valor lido inicialmente.

Phantom – Ocorre quando uma transação T_1 lê um conjunto de itens que satisfazem uma determinada condição de seleção. Em seguida, uma transação T_2 modifica ou insere itens de dados que satisfazem a condição de seleção utilizada por T_1. Depois disso, T_1 repete a consulta executada anteriormente, com a mesma condição de seleção, obtendo, porém, um resultado diferente, por exemplo, recuperando itens que não foram recuperados na primeira consulta (leitura).

Agora que conhecemos os fenômenos que deseja-se evitar, podemos definir formalmente os diferentes níveis de isolamento do padrão ANSI/ISSO SQL.

Read Uncommitted – Esse é o primeiro e menos restritivo nível de isolamento. Nesse nível podem ocorrer os três fenômenos estudados. Por outro lado, possibilita o maior grau de concorrência, dentre os diferentes níveis de isolamento do padrão SQL. Nesse nível de isolamento, temos que:

- **Bloqueios de leitura**: Não são necessários.
- **Bloqueio de escrita**: São de curta duração tanto para objetos (*tuplas* de uma tabela, por exemplo) quanto para predicados.

Read Commited – Esse nível é o usado por padrão no Oracle, PostgreSQL e SQL Server. Esse nível evita a ocorrência do fenômeno leitura suja (*Dirty Read*). Porém, não evita os fenômenos leitura não repetível (*Non-Repeatable*

Read) e fantasma (*Phantom*). Esse nível de isolamento é um pouco mais forte que o *Read Uncommitted*. Logo a consistência dos dados é um pouco mais rigorosa enquanto o grau de concorrência permitido é um pouco menor. Nesse nível de isolamento, temos que:

- **Bloqueios de leitura**: São de curta duração tanto para objetos (*tuplas* de uma tabela) quanto para predicados.
- **Bloqueio de escrita**: São de longa duração tanto para objetos (*tuplas* de uma tabela) quanto para predicados.

A seguir, exemplificamos algumas anomalias que podem ocorrer no nível de isolamento *Read Commited:*

- Nonrepeatable read

Considere o *Schedule* S_1:

$S_1 = r_2(x)\ w_1(x,2)\ c_1\ r_2(x)\ c_2$

A transação T_2 está lendo valores diferentes de x. O que deve ser evitado, apesar de ser improvável que uma mesma transação execute duas operações de leitura sobre o mesmo item de dado.

- Lost update

Considere o *Schedule* S_2:

$S_2 = r_2(x)\ r_1(x)\ w_1(x,x+20)\ c_1\ w_2(x, x+10)\ r_2(z)\ c_2$

Observe que a operação de escrita $w_1(x)$ foi sobreposta pela operação $w_2(x)$.

Cursor Stability – Esse nível de isolamento, apesar de não fazer parte originalmente do padrão SQL, passou a ser utilizado pela maioria dos SGBDs.

- **Bloqueios de leitura**: São de curta duração objetos (*tuplas* de uma tabela). Os bloqueios de predicados são de curta duração, contudo o bloqueio no item corrente do cursor é mantido até que o cursor seja movido ou o cursor seja fechado.
- **Bloqueio de escrita**: São de longa duração tanto para objetos (*tuplas* de uma tabela) quanto para predicados.

O nível de isolamento *Cursor Stability* evita o fenômeno atualização perdida (*lost update*). Para ilustrar esse fato, considere o exemplo a seguir:
$S_3 = r_2(x)\ w_1(x)\ c_1\ w_2(x)\ r_2(z)\ c_2$

Observe que o *Schedule* S_3 não poderia ser produzido no nível de isolamento *Cursor Stability* uma vez que o bloqueio de leitura obtido pela transação T_2 sobre o item x somente será liberado quando o cursor for movimentado para o próximo item de dado, o que só vai ocorrer após a transação T_2 executar sua operação de escrita em x (ou seja, $w_2(x)$).

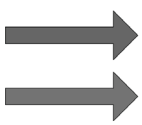
Repeatable Read – Esse nível de isolamento evita os fenômenos leitura suja (*Dirty Read)* e leitura não repetível (*Non-Repeatable Read)*. Contudo, não evita o fenômeno fantasma (Phantom).

- **Bloqueios de leitura**: São de longa duração para objetos (tuplas de uma tabela) e de curta duração para predicados.
- **Bloqueio de escrita**: São de longa duração tanto para objetos (tuplas de uma tabela) quanto para predicados.

Para ilustrar o funcionamento desse nível de isolamento, considere novamente o exemplo a seguir:

```
Select *
from Empregado
where salario>499
…
Commit
```

Estrutura de cursor



05	André	5000	1
08	Bárbara	2000	2
10	Inês	7000	1
15	Sofia	500	1
16	Natalya	2000	2
20	Caio	500	1
22	Lucas	550	1
30	Rebeca	550	2

Nesse caso, uma nova *tupla* (07,'José',4000,1) pode ser inserida na tabela uma vez que no nível de isolamento *Repeatable Read* os bloqueios de leitura são de curta duração para predicados. Já a *tupla* (05,'André', 5000, 1) ficará bloqueada até a execução do *commit* da transação, uma vez que bloqueios de leitura são de longa duração para objetos.

Nesse caso, se executarmos novamente a consulta:

```
Select *
from Empregado
where salario>499
```

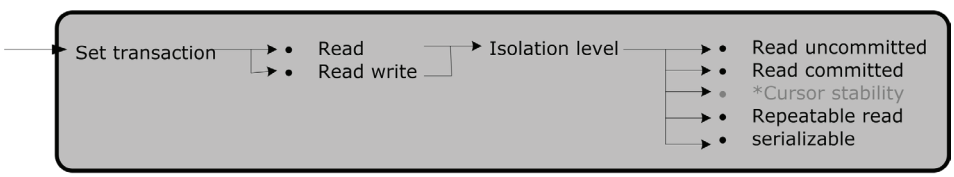
iremos observar uma tupla “fantasma” (ou seja, que não existia antes): (07,'José',4000,1).

Serializable – Esse é o nível de isolamento mais forte. Evita os três fenômenos estudados. Contudo, apresenta o menor grau de concorrência.

- **Bloqueios de leitura**: São de longa duração tanto para objetos (tuplas de uma tabela) quanto para predicados.
- **Bloqueio de escrita**: São de longa duração tanto para objetos (tuplas de uma tabela) quanto para predicados.

2.6.2 Sintaxe SQL 99

A seguir ilustramos a sintaxe SQL utilizada para definir o nível de isolamento a ser utilizado.



* Apenas DB2

Figura 2.18 – Sintaxe SQL 99

Vale destacar que o Oracle e PostGreSQL apenas implementam os níveis de isolamento Read Commited e Serializable. Já o MS SQLServer implementa os níveis de isolamento Read Uncommited, Read Commited, Repeatable Read e Serializable.

A escolha do nível de isolamento é importante decisão de projeto, que envolve decidir entre:

- Garantir consistência dos objetos do banco de dados, ou seja, definir que anomalias devem ser evitadas (*Nonrepeatable read*, *Lost update* etc)
- Garantir um alto grau de concorrência.

Exercícios

1. Sob que condições é melhor evitar conflitos do que permitir que eles ocorram para depois detectá-los?

2. Se um protocolo evita *deadlock*, o enfraquecimento ainda é possível? Justifique.

3. Apresente um início de escalonamento (*schedule*) 2PL Básico que recaia em uma situação de impasse (*deadlock*).

4. Apresente um escalonamento (*schedule*) 2PL Básico que não seja recuperável.

5. Apresente um início de escalonamento (*schedule*) 2PL Estrito que recaia em uma situação de impasse (*deadlock*).

6. Apresente exemplos de escalonamentos 2PL conservador, 2PL estrito e 2PL rigoroso para as seguintes transações:

   T_1: r(Y) w(Y) w(Z)
   T_2: r(X) r(T) w(T)
   T_3: r(Z) w(Z)
   T_4: r(X) w(X)

7. Demonstre que o protocolo de bloqueio em duas fases assegura a serializabilidade em conflito.

8. Considere o seguinte conjunto de transações T:

   T_1 : $r_1(A)$
   $r_1(B)$
   if A = 0 then B := B + 1;
   $w_1(B)$

   T_2 : $r_2(B)$
   $r_2(A)$
   if B = 0 then A := A + 1;
   $w_2(A)$

Adicione instruções *lock* e *unlock* para as transações T_1 e T_2 a fim de que eles observem o protocolo de bloqueio em duas fases. A execução dessas transações pode resultar em um impasse (*deadlock*)?

9. Considere um sistema de banco de dados que inclua uma operação atômica ***increment*** em adição às operações ***write*** e ***read***. Digamos que V seja o valor do item X. A operação *Increment(X)* por C ajusta o valor de X para V + C em um passo atômico. O valor de X não está disponível para a transação, a menos que ele execute um read(x). A Figura abaixo mostra uma matriz de compatibilidade de bloqueios para três modos de bloqueios: partilhado, exclusivo e incrementado.

S	S	X	I
S	verdadeiro	falso	falso
S	falso	falso	falso
I	falso	falso	verdadeiro

9.1. Mostre que, se todas as transações bloqueiam dados que elas acessam no modo correspondente ao bloqueio de duas fases, a serializabilidade está assegurada.

9.2. Mostre que a inclusão de bloqueios no modo incrementado permite aumentar a concorrência.

10. Explique o fenômeno fantasma. Por que esse fenômeno leva à execução incorreta apesar do uso de um protocolo que assegura a serializabilidade?

11. Prove que os protocolos *wait-die* e *wound-wait* evitam *deadlock* e enfraquecimento (*livelock*).

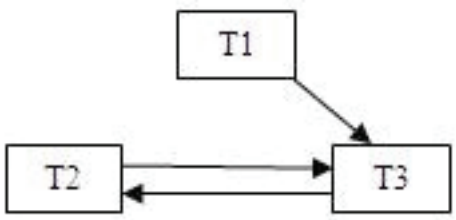
12. Por que as operações *lock* e *unlock* devem ser atômicas?

13. Considere o seguinte *schedule* S (incompleto) :

$r_1(x)\ r_1(y)\ w_1(x)\ r_2(y)\ w_3(y)\ w_1(x)\ r_2(y)$

1. Assumindo que as três transações executam suas operações de *commit*, mostre o grafo de serialização para o *schedule* S.

Solução:



2. Para cada uma das opções baixo modifique o *schedule* S a fim de criar um novo *schedule* (completo) T tal que:

a) T evita *aborts* em cascata e não é recuperável.
b) T é recuperável.
c) T é serializável por conflito.

Para criar o *schedule* T você pode adicionar novas operações em qualquer posição do *schedule* S, mas não pode alterar a ordem das operações já existentes. Caso não seja possível criar o *schedule* T como se pede, comente brevemente.

Solução:

a) T evita aborts em cascata e não é recuperável.

Lembrando (Definições):

Recuperável: Se wi(x) < rj(x), então ci < cj.
Evita abort em cascata: Se wi(x) < rj(x), então ci < rj(x).

Vamos provar por contradição que não é possível criar o *Schedule* pedido:

Suponha que T evita aborts em cascata. (1)
Suponha que T não é recuperável. (2)

Por (1), se wi(x) < rj(x), então ci < rj(x).

Suponha que wi(x) < rj(x). (3)

Por (1) e (3), ci < rj(x).

Consequentemente, ci < cj, pois, por definição, qualquer operação de leitura ou escrita antecede seu commit.

Logo, T é recuperável.

Chegamos, portanto, em uma contradição.

Podemos concluir que não é possível T evitar aborts em cascada e não ser recuperável.

b) T é recuperável.

r1(x) r1(y) w1(x) r2(y) w3(y) w1(x) r2(y) c1 c3 c2

c) T é serializável por conflito.
Não é possível criar o *Schedule* T, pois o prefixo exposto na questão não

é serializável por conflito. Como o prefixo não é serializável por conflito, então o *Schedule* completo também não será.

14. Como o protocolo 2V2PL consegue garantir um maior grau de concorrência do que o protocolo 2PL.

15. Em que situação o protocolo 2V2PL pode provocar que uma transação espere indefinidamente para terminar sua execução?

16 Mostre que o protocolo 2PL Preciso evita *aborts* em cascata.

17 Demonstre que se dois *schedules* são equivalentes por conflito, ou seja, seus grafos de serialização são idênticos. Prove que o inverso não é verdade.

18 Prove ou refute a assertiva a seguir: O protocolo 2V2PL evita a formação de deadlocks.

Solução:

Vamos refutar a assertiva dada por meio de um contraexemplo.

Dessa forma, vamos mostrar um exemplo de um *schedule* que seria gerado pelo protocolo 2V2PL e que possui um *deadlock*.

O *Schedule* incompleto a seguir serve como prova de que a assertiva em questão é falsa:

wl1(y) w1(y) wl2(x) w2(x) wl1(x) wl2(y)

Como podemos observar, é dado um bloqueio de escrita sobre y para T1. Em seguida, é concedido um bloqueio de escrita sobre x para T2. Logo após, T1 solicita um bloqueio sobre x e T2 sobre y. Ambas as transações são retardadas, causando, consequentemente, um *deadlock*.

19. Verifique se o seguinte *schedule* pode ser produzido por um escalonador (*scheduler*) 2V2PL. Justifique sua resposta.

$T_1=r_1(y)r_1(x)w_1(y)c_1$ $T_2= w_2(u)r_2(x)r_2(y)w_2(x)c_2$
$T_3=r_3(y)r_3(x)w_3(u)w_3(z)c_3$ $T_4=r_4(v)w_4(u)c_4$

$S=r_4(v)r_3(y)r_1(y)r_1(x)w_2(u)r_2(x)w_1(y)c_1r_2(y)r_3(x)w_2(x)c_2w_4(u)c_4w_3(u)w_3(z)c_3$

20 O nível de isolamento *repeatable read* sofre do fenômeno de fantasma (*phanton*). Considere que o DBA de sua empresa definiu o nível de

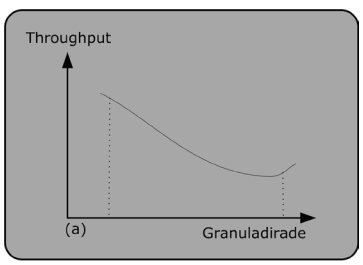
isolamento *repeatable read* para todas as transações. Construa um *schedule* onde esse fenômeno pode ocorrer no nível de isolamento *repeatable read*. Justifique sua resposta.

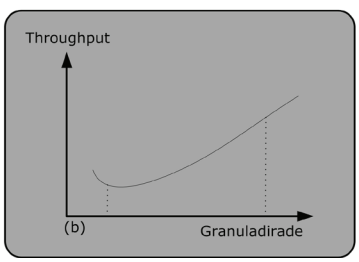
21 Identifique se o *schedule* S, definido sobre T ={T_1,T_2,T_3}, abaixo é livre de *deadlock*. Considere que as operações das transações devem ser escalonadas utilizando o protocolo 2PL preciso. Justifique sua resposta.

$T_1=r_1(y)r_1(x)c_1$ $T_2= r_2(v)w_2(x)r_2(z)c_2$ $T_3=r_3(v)w_3(z)w_3(y)c_3$

$S=r_2(v)r_1(y)w_2(x)r_3(v)r_1(x)w_3(z)r_2(z)w_3(y)c_3c_1c_2$

22 Considere os seguintes gráficos:





O gráfico (a) refere-se à execução de transações longas, onde se pode observar o comportamento do *throughput* (número de transações executadas em um intervalo de tempo) em relação a uma variação da granularidade de bloqueios. Por sua vez, o gráfico (b) representa a execução de transações de curta duração. Justifique os comportamentos representados pelas curvas nos gráficos (a) e (b).

23 Pode-se provar que a classe de *schedules* produzidos por um escalonador 2PL sobre um conjunto $\Im$ de transações está contida no conjunto de todos os *schedules* serializáveis por conflito sobre $\Im$**.** Formalmente, 2PL $\subseteq$ CSR. Construa um *schedule* serializável por conflito S (S CSR), mas que não

possa ter sido produzido por um escalonador que implemente o protocolo 2PL (S $\not\in$ 2PL).

24 Identifique se o seguinte *schedule* pode ser produzido no nível de isolamento *cursor stability.* Justifique sua resposta.

$\mathbf{S_A = r_2(y)r_1(x)r_1(v)w_2(x)w_3(y)r_3(v)r_3(u)w_1(v)c_3c_1c_2}$

25 O nível de isolamento *read committed* sofre do fenômeno da atualização perdida *(lost update).* Considere que o DBA de sua empresa definiu o nível de isolamento *read committed* para todas as transações. Construa um *schedule* onde esse fenômeno pode ocorrer no nível de isolamento *read committed,* para mostrar que o DBA pode está sendo responsável por inconsistências no banco de dados. Justifique sua resposta.

26. Considere o seguinte cenário de execução em um SGBD:

$T_1 = r_1(y)w_1(x)$ $T_2 = w_2(u)r_2(x)w_2(z)$ $T_3 = r_3(u)r_3(v)$

Considere que o SGBD utilize o protocolo 2PL-preciso para controlar concorrência.

a) Elabore um *Schedule* S que possa ser produzido em um SGBD que utiliza a estratégia *wound-wait* para prevenir situações de *deadlocks*. Justifique sua resposta.

b) Elabore um *Schedule* S que não possa ser produzido em um SGBD que utiliza a estratégia *wound-wait* para prevenir situações de *deadlocks*. Justifique sua resposta.

c) Elabore um *Schedule* S que possa ser produzido em um SGBD que utiliza a estratégia *wait-die* para prevenir situações de *deadlocks*. Justifique sua resposta.

d) Elabore um *Schedule* S que não possa ser produzido em um SGBD que utiliza a estratégia *wait-die* para prevenir situações de *deadlocks*. Justifique sua resposta.

27. Considere o seguinte *schedule* S (incompleto):

$r_1(x)\ r_1(y)\ w_1(x)\ r_2(y)\ w_3(y)\ w_1(x)\ r_2(y)$

a) Assumindo que as três transações executam suas operações de *commit*, mostre o grafo de serialização para o *schedule* S.

b) Para cada uma das opções abaixo modifique o *schedule* S a fim de criar um novo *schedule* (completo) T tal que:

- T evita *aborts* em cascata e não é serializável.
- T é recuperável e não é serializável por conflito.
- T é serializável por conflito e não evita *aborts* em cascata.

Para criar o *schedule* T você pode adicionar novas operações em qualquer posição do *schedule* S, mas não pode alterar a ordem das operações já existentes. Caso não seja possível criar o *schedule* T como se pede, comente brevemente.

28. Considere a base de dados das Eleições de 2002, a qual é descrita a seguir:

Candidatos (nr_candidato : integer , nome: string, cargol: integer, partido: string)

ce_voto_secao (cd_municipio: integer, nr_zona: integer, nr_secao: integer, cd_cargo: integer, nr_candidato: integer, qtd_votos: integer)

PS. : Os campos sublinhados constituem a chave primária de cada relação.

Considere ainda a existência de dois procedimentos, os quais são mostrados a seguir:

```
SET TRANSACTION ISOLATION
LEVEL SERIALIZABLE

BEGIN TRAN proc1

SELECT * FROM CANDIDATOS
WHERE NR_CANDIDATO = 13

WAITFOR DELAY '000:02:00'

UPDATE CANDIDATOS
SET NR_CANDIDATO = 128
WHERE NR_CANDIDATO = 13

COMMIT TRAN proc1
```

```
SET TRANSACTION ISOLATION LEVEL
SERIALIZABLE

BEGIN TRAN proc2

SELECT * FROM CANDIDATOS
WHERE NR_CANDIDATO = 13

WAITFOR DELAY '000:02:00'

UPDATE CANDIDATOS WITH (NOLOCK)
SET NR_CANDIDATO = 127
WHERE NR_CANDIDATO = 13

COMMIT TRAN proc2
```

Assuma que o procedimento “proc1” foi executado inicialmente, e, logo em seguida foi executado o procedimento “proc2”.

Descreva e comente o *schedule* gerado pela execução desses dois procedimentos.

Dicas:

1. Observe que o comando *TransactSQL* WAITFOR DELAY '000:02:00' faz com que o procedimento pare sua execução e espere por 2 minutos.
2. Note que o comando SET TRANSACTION ISOLATION LEVEL seta (ajusta) o nível de isolamento.

29. Prove ou refute a assertiva a seguir: “Todo *Schedule* recuperável evita a ocorrência de aborts em cascata”.

Considere a base de dados apresentada na questão anterior. Considere ainda a existência de dois procedimentos, os quais são mostrados a seguir:

```
SET TRANSACTION ISOLATION LEVEL READ
UNCOMMITTED

BEGIN TRAN proc1

SELECT * FROM CANDIDATOS
WHERE NR_CANDIDATO = 13

WAITFOR DELAY '000:02:00'

UPDATE CANDIDATOS
SET NR_CANDIDATO = 128
WHERE NR_CANDIDATO = 13

COMMIT TRAN proc1
```

```
SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED

BEGIN TRAN proc2

SELECT * FROM CANDIDATOS
WHERE NR_CANDIDATO = 13

WAITFOR DELAY '000:02:00'

UPDATE CANDIDATOS
SET NR_CANDIDATO = 127
WHERE NR_CANDIDATO = 13

COMMIT TRAN proc2
```

Assuma que o procedimento “proc1” foi executado inicialmente, e, logo em seguida foi executado o procedimento “proc2”.
Descreva e comente o *schedule* gerado pela execução desses dois procedimentos.

30. Para cada uma das opções abaixo crie um *Schedule* que satisfaça as propriedades requisitadas:

a) Não recuperável e não serializável.
b) Serializável e recuperável.
c) Preciso e serializável.
d) Não serializável e que não evita *aborts* em cascata.

3. DESCRIÇÃO DOS PROTOCOLOS AGRESSIVOS

Protocolos de controle de concorrência baseados em bloqueio utilizam uma abordagem conservadora para a resolução de conflitos entre transações. Eles assumem que na maioria das transações ocorrerão conflitos, por esse motivo eles bloqueiam essas transações. Contudo em sistemas onde não há grande acesso de escrita a objetos, essa abordagem acaba sendo prejudicial. Nesse contexto, novos protocolos, que não utilizam o conceito de bloqueio, começaram a ser estudados.

3.1 Protocolos Baseados em Marcadores de Tempo (Timestamps)

No protocolo de ordenação por marca de tempo (*timestamp*), o gerenciador de transações (GT) atribui um *timestamp*, $ts(T_i)$, a cada transação T_i. Para quaisquer duas transações T_i e T_j, com $(i \neq j)$, $ts(T_i) \neq ts(T_j)$. O gerenciador de transações associa o *timestamp* da transação T_i a todas as operações $pi(x) \in T_i$. Em geral, os valores dos *timestamps* representam a ordem em que as transações são submetidas ao sistema. As estratégias mais usadas para gerar os valores dos *timestamps* baseiam-se na utilização de números sequenciais (1,2,3...) e da composição Data+Hora. Por conveniência iremos utilizar o termo *timestamp* de uma operação $p_i(x)$, embora esse seja nada mais que o *timestamp* da transação T_i. Um escalonador, baseado na ordenação por *timestamp*, ordena as operações em conflito de acordo com os seus *timestamps*, segundo a regra mostrada a seguir:

Regra: Se $p_i(x)$ e $q_j(x)$ são operações em conflito, então a operação $p_i(x)$ será executada antes da operação $q_j(x)$ se e somente se $ts(T_i) < ts(T_j)$.

O teorema a seguir mostra que se essa regra for seguida os escalonamentos (*schedules*) produzidos serão serializáveis por conflito.

Teorema:

Se S é um *schedule* que representa uma execução concorrente produzida pela regra da ordenação por *timestamp,* S é serializável por conflito.

Prova: Seja GS(S) o grafo de serialização para o *schedule* S. Se $T_i \rightarrow T_j$ é uma aresta de GS(S), devem existir duas operações $p_i(x)$ e $q_j(x)$, que estão em conflito em S, tais que $p_i(x) <_s q_j(x)$. Assim, pela regra da ordenação por *timestamp*, temos que $ts(T_i) < ts(T_j)$. Vamos supor que existe um ciclo $T_1 \rightarrow T_2 \rightarrow .... \rightarrow T_n \rightarrow T_1$, onde n>1, em GS(S). Usando indução, podemos concluir que $ts(T_1) < ts(T_1)$, o que é uma contradição. Logo, GS(S) é acíclico, e pelo teorema da serializabilidade, S é serializável por conflito.

3.1.1 Protocolo Básico de Ordenação por *Timestamp*

O protocolo básico (*Basic Timestamp Ordering*, ou simplesmente Basic TO) é uma implementação simples e agressiva da regra de ordenação por *timestamp* (Regra OT). Nesse protocolo, o *scheduler* escalona as operações de forma imediata, ou seja, na medida em que essas são recebidas do gerenciador de transações (*first-come-first-served order*). Para garantir que o *schedule* gerado não viole a regra OT, o escalonador rejeita as operações que chegarem atrasadas. Isto é feito reiniciando (abortando) a transação que submeteu a operação atrasada. Uma operação $p_i(x)$ é dita atrasada se ao chegar ao escalonador esse já tenha executado uma operação $q_j(x)$, que conflita com $p_i(x)$, com $ts(T_j) > ts(T_i)$$. Nesse caso, a operação $p_i(x)$ deve ser rejeitada, ou seja, a transação T_i deve ser reiniciada (abortada).

Funcionamento

O algoritmo associa a cada item de dão x do banco de dados (BD) dois *timestamps*:

- **Read_TS(x):** Maior entre todos os *timestamps* de transações que leram o item x, e
- **Write_TS(x):** Maior entre todos os *timestamps* de transações que modificaram o item x.

Quando uma transação T_i deseja ler ou escrever sobre o item x, o algoritmo compara o *timestamp* de T_i com o Read_TS(x) e Write_TS(x), conforme mostra o algoritmo a seguir:

```
Ordenação por timestamp

        Se ( Ti solicita uma operação write(x) )
                Se ( read_TS(X) > ts(Ti) ou write_TS(X) > ts(Ti) )
                        Então
                                abortar Ti;
                        Se não
                                executa write(x);
                                write_TS(X) <- ts(Ti);

        Se ( Ti solicita uma operação read(x) )
                Se ( write_TS(X) > ts(Ti) )
                        Então
                                abortar Ti;
                        Se não
                                executa read(x);
                                read_TS(X) <- Max(read_TS(X), ts(Ti))
```

Figura 3.1 – Algoritmo do Protocolo de Ordenação por *Timestamp*.

Principais Características

- A ordem de execução das transações é determinada pela ordem em que as transações são iniciadas;
- O *schedule* produzido é equivalente a algum *Schedule* serial, o qual corresponde à ordem dos *timestamps*;
- As operações conflitantes sempre são executadas na ordem dos *timestamps* de suas transações;
- Não utiliza bloqueios. Logo, está livre de problemas de *deadlock*;
- Não evita *aborts* em cascata;

Exemplo 14

Considere o seguinte conjunto de transações:

$T_1 = r_1(x)\ r_1(y)\ w_1(x)\ c_1$

$T_2 = r_2(x)\ r_2(y)\ c_2$

Considere agora a seguinte execução (*schedule*):

$S_1 = r_1(x)\ r_2(x)\ r_1(y)\ w_1(x)\ r_2(y)\ c_2\ c_1$

Observe que o *schedule* S_1, apesar de correto (pois se montarmos o grafo de serialização veremos que esse não possui ciclo) não será gerado pelo protocolo da ordenação por *timestamp*. Note que no momento em que a operação $w_1(x)$ chegar ao escalonador esse irá verificar que já existe uma operação, anteriormente escalonada, que conflita com $w_1(x)$, mais precisamente $r_2(x)$. Nesse caso, o escalonador irá testar se Read_TS(x), que nesse momento tem valor 2, é maior que $ts(T_1)$, que estamos assumindo ter valor 1. Como o resultado desse teste será verdadeiro a transação T_1 será cancelada (abortada), apesar do *schedule* S_1 ser correto.

3.2 Teste do Grafo de Serialização

Seja S um *schedule* sobre um conjunto de transações $T=\{T_1, T_2, \ldots, T_n\}$. O grafo de serialização para S, representado por GS(S), é definido como o grafo direcionado GS(S) = (N,A) no qual cada nó em N corresponde a uma transação em T. O conjunto A contém as arestas na forma $T_i \rightarrow T_j$, se e somente se T_i, $T_j \in N$ e existem duas operações $p \in OP(T_i)$, $q \in OP(T_j)$, onde p conflita com q e $p <_S q$. Um *schedule* global S é serializável por conflito se e somente se o grafo de serialização para S é acíclico. Um *schedule* S é correto se ele for serial ou serializável por conflito.

O protocolo do teste do grafo de serialização baseia-se no monitoramento e gerenciamento do grafo de serialização. A ideia básica do protocolo consiste em manter um grafo de serialização sempre acíclico, uma vez que se esse grafo

for acíclico o *schedule* gerado é serializável por conflito. Vale destacar que existe um algoritmo para verificar se um grafo é acíclico em tempo polinomial.

A seguir descrevemos um possível algoritmo para implementar esse protocolo:

Para cada nova transação T_i, **Faça**:

Incluir um nó no grafo de serialização;
Fim Para;

Para cada operação $p_i(x) \in OP(T_i)$ recebida **Faça**:

Verificar se existe uma operação $q_j(x) \in OP(T_j)$ que conflite com $p_i(x)$ e que já tenha sido enviada para processamento;

Caso afirmativo incluir aresta $T_j \rightarrow T_i$;

Se a nova aresta introduzir um ciclo no grafo, **então**:

Abortar T_i;
Retirar do grafo o nó referente a T_i e todas as arestas que partem ou chegam nesse nó;
Fim Se;
Fim Para;

Garbage Collection

Se o nó correspondente a uma transação terminada Ti só possui arestas saindo, então o nó pode ser removido do grafo

Porque??

Contudo, é razoável esperar que o grafo de serialização cresça substancialmente, uma vez que existe um nó no grafo para cada transação. Assim, o tempo necessário para verificar se o grafo é acíclico tende a aumentar. Com a finalidade de evitar esse problema uma estratégia de coleta de lixo (*Garbage Collection*) pode ser utilizada:

Se o nó correspondente a uma transação terminada T_i só possui arestas saindo, então o nó pode ser removido do grafo. Isto se deve ao fato de que se uma transação concluiu sua execução então nenhuma nova aresta deve ser inserida no grafo de maneira a chegar no nó referente a T_i, uma vez que T_i não executa nenhuma nova operação. Logo, esse nó jamais se envolverá em um ciclo, podendo ser retirado do grafo.

Exemplo 15

Considere o seguinte conjunto de transações:

$T_1 = r_1(x)\ r_1(y)\ w_1(x)\ c_1$

$T_2 = r_2(x)\ r_2(y)\ c_2$

Considere agora a seguinte execução (*schedule*):

$S_1 = r_1(x)\ r_2(x)\ r_1(y)\ w_1(x)\ r_2(y)\ c_2\ c_1$

Observe que o *schedule* S_1 será gerado pelo protocolo TGS, uma vez que o grafo de serialização de S_1 não possui ciclo (figura 3.3).

T1 ← T2

Figura 3.3 – Grafo de Serialização do *Schedule* S_1.

Exemplo 16

Considere o seguinte conjunto de transações:

$T_1 = r_1(x)\ w_1(x)\ c_1$

$T_2 = r_2(x)\ w_2(x)\ c_2$

Considere agora a seguinte execução (*schedule*):

$S_2 = r_1(x)\ r_2(x)\ w_1(x)\ w_2(x)\ c_1\ c_2$

Observe que momento em que o escalonador receber a operação $w_1(x)$, será inserido no grafo de serialização uma resta $T_2 \rightarrow T_1$. Contudo, a inserção dessa aresta não introduz um ciclo no grafo de serialização. Logo, a operação $w_1(x)$ será escalonada. Já no momento em que o escalonador receber a operação $w_2(x)$, será inserido no grafo de serialização uma resta $T_1 \rightarrow T_2$. Como a inserção dessa nova aresta introduz um ciclo no grafo de serialização (figura 3.4), a operação $w_2(x)$ será rejeitada e a transação T_2 abortada. Logo, o *schedule* S_2 (que é incorreto) não será gerado por um escalonador que utiliza o protocolo TGS.

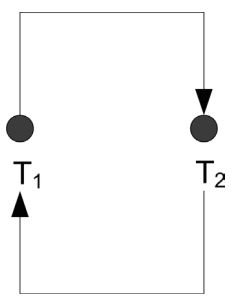


Figura 3.4 – Grafo de Serialização do *Schedule* S_2.

3.3 Protocolos Otimistas

Em todas as técnicas de controle de concorrência discutidas até aqui, sempre se faz alguma verificação antes que uma operação, de leitura, escrita ou *commit*, por exemplo, seja executada. Essas verificações geram uma sobrecarga (*overhead*) durante a execução das transações, reduzindo assim a velocidade com as transações são concluídas.

Os protocolos otimistas partem do pressuposto que a maioria das transações não irão se envolver em conflitos. Assim, nas técnicas de controle de concorrência otimista nenhuma verificação é realizada enquanto a transação estiver sendo executada. No momento em que a transação solicitar o seu *commit* é que algum tipo de verificação é realizado. Dentre as técnicas otimistas destacam-se os protocolos baseados em certificadores e os protocolos baseados em validação.

Um escalonar que utiliza um protocolo otimista baseado em certificadores funciona da seguinte forma:

O escalonador, ao receber uma operação de leitura ou escrita envia imediatamente essa operação para processamento (execução) e atualiza o escalonamento (*schedule*) que está sendo gerado.

Quando uma transação T_i envia uma solicitação de *commit* o escalonador verifica se o escalonamento gerado até o presente momento, incluindo a operação c_i, é serializável. Em caso afirmativo, o escalonador realiza o *commit* de T_i. Em caso negativo, o escalonador aborta a transação T_i e retira as operações de T_i do escalonamento por ele gerenciado.

A seguir, descrevemos um segundo protocolo otimista, denominado protocolo baseado em validação. Nesses protocolos, as transações executam em três fases:

- **Leitura**: Nessa fase a transação lê os valores dos itens do banco de dados que necessita e guarda uma cópia desses valores em uma área local, chamada *workspace* ou *cache*;
- **Validação**: Nessa fase, o SGBD verifica se a transação pode conflitar com alguma outra transação que está executando concorrentemente. Caso exista esse risco, a transação é abortada, sua *cache* local esvaziada e, em seguida, a transação é reiniciada; e
- **Escrita**: Caso a transação passe pela fase de validação, as alterações realizadas nos valores armazenados em sua cópia (*cache*) local são persistidas (gravadas) no banco de dados.

Nesse protocolo, as atualizações realizadas por uma determinada transação T_i não são aplicadas diretamente aos itens do banco de dados até que a transação T_i seja concluída (ou seja, execute sua operação de *commit*). Durante a execução de T_i, todas as atualizações (alterações) realizadas por T_i são aplicadas em cópias locais dos itens de dados (*cache*). Ao final da execução de T_i, uma fase de validação é iniciada. Durante essa fase, é verificado se alguma das atualizações realizadas por T_i violou a serialização. Se a serialização não for violada, a transação T_i é efetivada, e o banco de dados é atualizado com as cópias locais de T_i. Caso contrário, a transação T_i é abortada e, mais tarde, reiniciada. A ideia do protocolo é fazer todas as verificações de uma só vez. Assim, a execução de uma transação T_i continua com um mínimo de sobrecarga até que a fase de validação seja alcançada.

Importante observar que as abordagens otimistas somente resultarão em um melhor desempenho se houver poucos conflitos e a validação puder ser executada de maneira eficiente. Caso contrário, o custo de reiniciar as transações continuamente será bastante elevado.

Os protocolos que não são baseados em bloqueio:

- Apresentam um maior grau de concorrência, pois tendem a permitir um número maior de entrelaçamentos;
- Não sofrem do problema de *deadlocks*;
- Em geral, apresentam altas taxas de *abort*;
- Não evitam *aborts* em cascata;

Exercícios

1. Mostre que existem *schedules* que são possíveis sob o protocolo de bloqueio em duas fases, mas não são possíveis sobre o protocolo baseado em marcador de tempo e vice-versa.

2. Identifique diferenças na utilização de *timestamps* na prevenção de *deadlocks* e no controle de concorrência.

3. Prove ou refute a assertiva a seguir: Todo *schedule* gerado pelo protocolo de ordenação por *timestamp* (TO) pode ser gerado pelo protocolo do teste do grafo de serialização (TGS).

4. Prove ou refute. Todo *schedule* gerado pelo protocolo do TGS também pode ser gerado pelo protocolo de Ordenação por Marcador de Tempo (*Timestamp*).

5. Prove ou refute. Todo *schedule* gerado pelo protocolo do TGS também pode ser gerado pelo protocolo 2V2PL.

6. Identifique se o seguinte *schedule* pode ser produzido por um escalonador (*scheduler)* implementando o protocolo de marca de tempo. Justifique sua resposta.

$$S_A = r_1(y)r_2(u)r_2(x)w_3(u)r_3(v)c_3w_1(v)c_2c_1;$$

UNIDADE 4

RECUPERAÇÃO DE BANCO DE DADOS

Resumindo

Esta unidade discute em detalhes a recuperação de banco de dados. São discutidos e detalhados os diversos tipos de falhas. A estrutura de log é apresentada e a recuperação baseada em log é discutida. Noções de gerenciamento de buffer são apresentadas e examinadas. Técnicas de recuperação, incluindo os meios de armazenamentos são apresentadas, destacando suas vantagens e desvantagens. Por fim, técnicas baseadas em páginas sombras são discutidas.

4 RECUPERAÇÃO DE BANCO DE DADOS

1 RECUPERAÇÃO DE BANCO DE DADOS

1.1 Introdução

Um sistema de computador, como qualquer outro dispositivo elétrico ou mecânico, está sujeito a falhas. Essa simples observação tem consequências de longo alcance para o projeto de sistemas de computadores em geral e de Sistemas de Gerenciamento de Banco de Dados (SGBD) em particular.

Uma parte essencial do SGBD é o Sistema de Recuperação, responsável pela detecção de falhas e pela restauração do BD para um estado consistente que existia antes da ocorrência da falha.

O Sistema de Recuperação trata de duas propriedades das transações: atomicidade e durabilidade. Para tanto, o SGBD deve ser tolerante a falhas, ou seja, ser capaz de conduzir o BD a um estado passado consistente, após a ocorrência de uma falha que o deixou em um estado inconsistente.

O Sistema de Recuperação baseia-se em redundância de dados. Entende-se por redundância a criação de cópias dos dados e operações ou a replicação desses dados. No caso de falha dos dados principais, podem-se utilizar as cópias para recuperar os dados e então retornar o BD a um estado consistente.

É importante lembrar que a redundância de dados não é um mecanismo 100% seguro. Suponha que o SBGD está fazendo cópia dos dados em um disco rígido externo e ocorre uma falha no disco principal. Nesse caso, o SGBD vai tentar recuperar os dados do disco externo. Contudo, esse disco também pode sofrer uma falha e, assim, o SGBD não poderá retornar a um estado consistente.

O SGBD executa alguns procedimentos durante seu funcionamento normal, como manter informações sobre o que foi atualizado no BD pelas transações e realizar cópias periódicas do BD. Quando ocorre uma falha, o SGBD deve executar ações para retornar o BD a um estado consistente. As

duas ações básicas são:

- UNDO: desfazer uma atualização no BD.
- REDO: refazer uma atualização no BD.

1.2 Tipos de Falhas

Existem diferentes tipos de falhas que podem afetar a consistência de um BD. A seguir detalhamos cada uma dessas falhas.

Falha de Transação

A falha de transação ocorre quando uma transação ativa termina de forma anormal. As principais causas para esse tipo de falha são: violação de uma restrição de integridade, lógica da transação mal definida, deadlock, cancelamento pelo usuário, entre outras.

Esse tipo de falha não compromete a memória principal nem a memória secundária (disco, em geral). Como as causas são muito comuns, essa falha tem maior probabilidade de ocorrência. Entretanto, seu tempo de recuperação é pequeno, pois não compromete as memórias principais e secundárias.

Falha de Sistema

A falha de sistema ocorre quando o SGBD encerra a sua execução de forma anormal. As principais causas para esse tipo de falha são: interrupção de energia, falha no sistema operacional erro interno no software do SGBD, falha de hardware, entre outras.

Nesse caso, as transações em execução não podem mais continuar, porém é possível recuperá-las. Pode perguntar por que quedas de energia geram perda de estado de uma transação? A resposta é simples: porque muitas vezes o conteúdo que estava na memória principal (memória volátil) não foi passado para o disco (memória estável).

Esse tipo de falha compromete a memória principal, mas não compromete a memória secundária. Como as causas são mais comuns do que a falha de transação, esse tipo de falha tem probabilidade média de ocorrência. Entretanto, seu tempo de recuperação é médio, pois compromete a memória principal.

Falha de Meio de Armazenamento

A falha de meio de armazenamento ocorre quando o BD torna-se total ou parcialmente inacessível. As principais causas para esse tipo de falha estão relacionadas ao disco: setores corrompidos no disco, falha no cabeçote de leitura/gravação, entre outras.

Esse tipo de falha não compromete a memória principal mas compromete a memória secundária. Como as causas não são comuns, essa falha tem menor probabilidade de ocorrência. Entretanto, seu tempo de recuperação é grande, pois compromete a memória secundária.

Falha Catastrófica

Na falha catastrófica estão várias situações em que a mídia que contém o BD é completamente destruída. Os exemplos incluem explosões ou incêndios no local do banco de dados, vandalismo ou vírus.

Em todos os casos, os princípios em que se baseia a recuperação são bastante simples e podem ser resumidos em uma única palavra: redundância. Isto é, o meio de proteger o BD é assegurar que qualquer informação nele existente possa ser reconstruída a partir de alguma outra informação armazenada redundantemente.

Para evitar ou contornar essas falhas, os seguintes procedimentos são executados: (a) toda vez que é feita uma modificação no BD, é mantido automaticamente pelo sistema um registro num arquivo especial (b) periodicamente é feito um backup de todo o BD (normalmente em fita magnética).

1.3 Recuperação Baseada em Log

Registro de Log

A estrutura largamente usada para registrar modificações no banco de dados é o log (ou Journal). Um log registra sequencialmente as operações feitas por transações no banco de dados. Esse arquivo é geralmente consultado em caso de falhas para a realização de UNDO e/ou REDO de transações e é mantido em uma ou mais cópias em memória secundária, tais como disco e fita.

Tipos de Log

Existem alguns tipos de log e esses são usados para recuperação em diversas situações, de acordo com o tipo de falha ou problema. Os principais tipos de log são:

- log UNDO: esse log mantém apenas o valor antigo do dado (before image).
- log REDO: esse mantém apenas o valor atualizado do dado (after image).
- log UNDO/REDO: esse é o log mais comum e mantém os valores antigo e atualizado do dado.

Outros registros especiais de log existem para gravar eventos significativos durante o processamento de transações, tais como o início de uma transação e o compromisso ou aborto da transação. Para fins de recuperação de falhas, operações leitura não precisam ser gravadas e são úteis apenas para outros fins, tais como auditoria ou estatística.

Representação do Log

Para diferenciar as transações, o SGBD atribui a uma dessas um identificador único. Os vários tipos de registros de log são representados de acordo com a Figura 1.

- Início de transação: <start T_x>
- Commit de transação: <commit T_x>
- Atualização: <write T_x, X, beforeImage, afterImage>

Não é necessário em log REDO
Não é necessário em log UNDO

Figura 1 – Representação do log.

- Identificação da transação: identificação única da transação que executa a operação WRITE.
- Nome do item de dado: o nome único do item de dado gravado.
- Valor antigo: o valor do item de dado anterior à gravação.
- Novo valor: o valor que o item de dado terá depois da gravação.

Um exemplo de um log completo pode ser observado na Figura 2:

Log

<start T_3>
<write T_3, B, 15, 12>
<start T_2>
<write T_2, B, 12, 18>
<start T_1>
<write T_1, D, 20, 25>
<commit T_1>
<write T_2, D, 25, 26>
<write T_3, A, 10, 19>
<commit T_3>
<commit T_2>
...

T_1

Read(A)
Read(D)
Write(D)

T_2

Read(B)
Write(B)
Read(D)
Write(D)

T_3

Read(C)
Write(B)
Read(A)
Write(A)

Figura 2 – Representação do log.

Nessa Figura, a transação T_3 inicia a transação e depois altera o item B do valor 15 para o valor 12. Em seguida, a transação T_2 inicia e modifica o item B do valor 12 para o valor 18. A transação T_1 inicia e modifica o item D do valor 20 para o valor 25. A transação T_1 é efetivada e o processo de preenchimento do log continua a a efetuação das transações T_2 e T_3.

É importante observar que apenas as operações de escrita são armazenadas no log. Outra forma de representar o log é considerar a operação feita no BD:

- insert: <write Tx, X, INSERT, afterImage>
- update: <write Tx, X,UPDATE, beforeImage, afterImage>
- delete: <write Tx, X,DELETE, beforeImage>

OBS.: Como o log armazena informações sobre todas as atividades do BD, o volume de dados armazenados no log pode tornar-se excessivamente grande. Existem meios seguros para apagar informações do log ou para compactá-lo.

1.4 Gerenciamento de Buffer

O buffer é um conjunto de blocos da memória principal. O uso do buffer surge da necessidade de evitar acessos frequentes ao meio físico (LOG e BD). O SGBD é responsável pela gerência de alguns buffers, tais como buffers para dados, para processamento de transações e para o log. Nesse caso, o SGBD assume o controle desses buffers, ao invés do sistema operacional, requisitando apenas serviços de leitura/escrita de blocos ao Sistema Operacional. A Figura 3 mostra um exemplo de buffers.

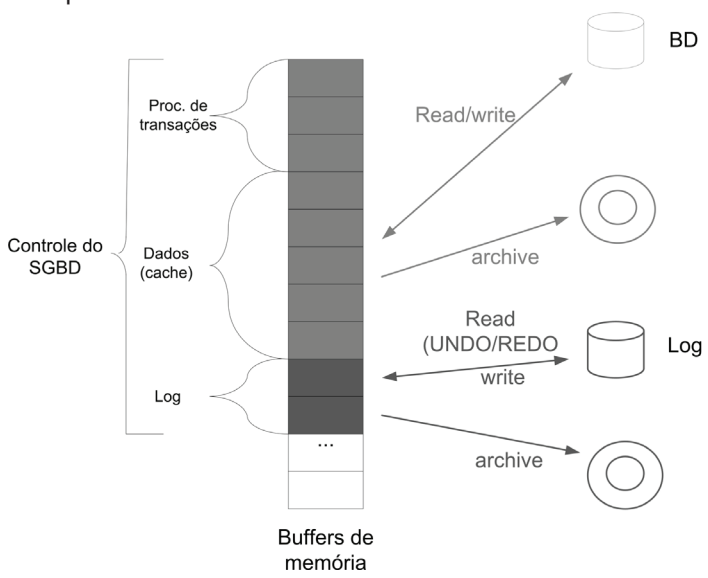


Figura 3 – Exemplo de buffers.

Buffer do Log

Esse buffer agrupa um conjunto de registros de log. Esses registros podem ser enviados para o meio físico, quando um bloco de registros equivalente ao tamanho de um bloco do arquivo de log for completado.

Os registros do log devem ser armazenados no arquivo na ordem em que eles foram criados. Por exemplo, antes que o registro <commit Ti> seja armazenado no arquivo, todos os outros registros dessa transação já devem estar no arquivo.

Uma transação só é considerada efetivada quando todos os registros de log dessa transação estiverem no meio físico, ou seja, persistidas em memória secundária.

Buffer do Banco de Dados

Esse buffer guarda na memória principal alguns blocos do banco de dados físico. Quando um dado do banco de dados precisa ser lido e o bloco físico onde esse dado está armazenado não está no buffer, então esse bloco é recuperado do banco de dados físico.

Técnicas para o Gerenciamento de Buffer

As técnicas de recuperação devem sincronizar os buffers de log e de dados. Para tato, essas técnicas utilizam o seguinte principio básico: “um bloco atualizado na cache só pode ser gravado no banco de dados após o histórico dos dados atualizados nesse bloco ter sido gravado no Log em disco”. Esse princípio é chamado Write-Ahead-Log (WAL).

Nesse caso, uma transação Tx só pode passar para o estado efetivada (committed) após todas as suas atualizações terem sido gravadas no banco de dados segundo o princípio WAL.

O SGBD aplica técnicas de gerenciamento de buffer e essas técnicas influenciam as técnicas de recuperação. As principais técnicas são:

NOT-STEAL

Nessa técnica, um bloco na cache utilizado por uma transação Tx não pode ser gravado antes do commit de Tx. Esse bloco possui um bit de status indicando se foi (1) ou não (0) modificado. Com isso, o processo de recuperação é mais simples, pois evita dados de transações inacabadas sendo gravadas no banco de dados.

STEAL

Na técnica STEAL, um bloco na cache utilizado por uma transação Tx pode ser gravado antes do commit de Tx. Assim, é necessário se algum dado é

requisitado do banco de dados por outra transação e não há blocos disponíveis na cache. Então, um bloco a ser eliminado deve ser escolhido através de alguma técnica, como o FIFO, ou seja, primeiro a entrar, primeiro a sair. A vantagem dessa técnica é que não há necessidade de manter blocos bloqueados por transações.

FORCE

Com o FORCE, os blocos que mantêm dados atualizados por uma transação Tx são imediatamente gravados no BD quando Tx alcança o commit. Para tanto, deve-se saber quais os blocos que Tx atualizou dados. A vantagem com isso é garantia da durabilidade de Tx o mais cedo possível, permitindo o REDO de Tx em caso de falha.

NOT-FORCE

Com o NOT-FORCE, os blocos que mantêm dados atualizados por Tx não são imediatamente gravados no BD quando Tx alcança o commit. Assim, blocos atualizados podem permanecer na cache e serem utilizados por outras transações, após o commit de Tx, o que reduz o custo de acesso a disco.

1.5 Checkpoint

Na existência de uma falha, o sistema de recuperação deve, a princípio, percorrer todo o log para saber quais transações devem ser desfeitas. Contudo, esse processo de percorrer o log consome muito tempo, visto que o log pode ser muito grande.

Além disso, muitas das transações que necessitam ser refeitas já escreveram suas atualizações no BD e refazê-las se tornará muito oneroso. Para resolver esse problema, os SGBDs introduzem o conceito de checkpoints ou pontes de controle. Checkpoints são pontos em que se garante que o conteúdo dos buffers do log e do banco de dados foram descarregados nos respectivos meios físicos.

Os checkpoints são registros inseridos no log periodicamente e exigem a execução da sequência de operações abaixo:

1. Suspender, temporariamente, a execução de todas as transações;
2. Descargar o buffer do log para o arquivo de log;
3. Descargar do buffer do BD para o BD físico;
4. Gravar de um “registro checkpoint” no arquivo log;
5. Reassumir a execução das transações.

Quando ocorre uma falha de sistema, os seguintes passos devem ser executados:

- Todas as transações cujos registros <commit Ti> estejam depois do "registro de Checkpoint" mais recente gravado no log devem ser refeitas (REDO);
- Todas as transações cujos registros de <start Ti> forem encontrados no log e os seus respectivos <commit Ti> não forem encontrados devem ser desfeitas (UNDO);
- A técnica de checkpoint não interfere no processo de UNDO, somente no REDO.

O ponto crítico em relação falha de sistema é que os conteúdos da memória principal são perdidos (em particular, os buffers). Não é possível prever o estado preciso de qualquer transação que estava em andamento no momento da falha. Tal transação pode nunca terminar com sucesso, logo precisa ser desfeita (rollback) quando o sistema reiniciar.

Entretanto, pode também ser necessário refazer algumas transações que possam ter terminado com sucesso antes da falha ocorrer, mas que não tiveram seus resultados transferidos dos buffers para o banco de dados físico.
Exemplo:

Como o SGBD sabe, no momento do seu reinicio, quais transações desfazer e quais transações refazer?

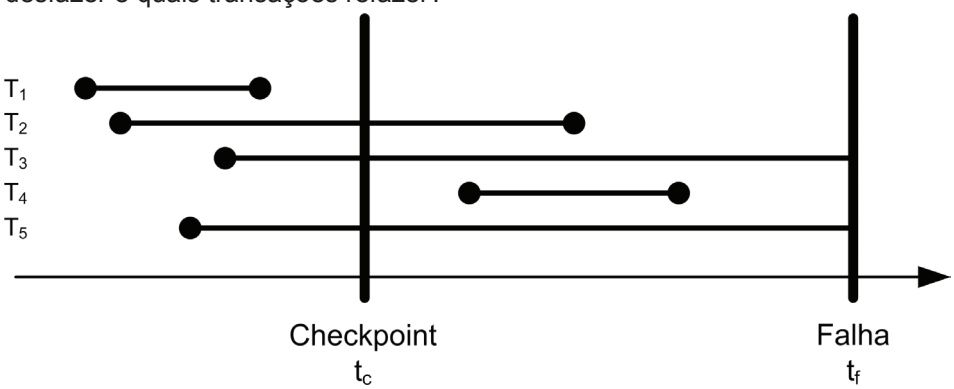


Figura 4 – Exemplo de checkpoint.

A Figura 4 mostrar que ocorreu uma falha de sistema ocorre no tempo t_f e o mais recente checkpoint ocorrido antes do tempo t_f ocorreu no tempo t_c.

Em relação às transações, a transação T_1 terminou antes do tempo t_c, a transação T_2 começou antes do tempo t_c e terminou depois do tempo t_c, mas antes do tempo t_f , a transação T_3 também começou antes do tempo t_c, mas não terminou antes do tempo t_f, a transação T_4 começou depois do tempo t_c e terminou antes do tempo t_f e a transação T_5 também começou depois do tempo t_c, mas não terminaram antes do tempo t_f.

Deve ficar claro que, quando o sistema é reiniciado, as transações do tipo T_3 e T_5 devem ser desfeitas (UNDO), e as transações do tipo T_2 e T_4 devem ser refeitas (REDO).

Entretanto, as transações T_1, não entram no processo de recuperação, porque todos os seus resultados já foram escritos no banco de dados físico no tempo t_c.

1.6 Técnicas de Recuperação

Existem duas técnicas para usar o log com vistas a assegurar a atomicidade das transações apesar das falhas. São elas: (a) Modificação Adiada do BD e (b) Modificação Imediata do BD.

Modificação Imediata

Na modificação imediata, todos os dados que foram alterados por transações efetivadas ou não, são persistidos no arquivo de dados. Por essa razão, algumas transações precisam ser desfeitas (UNDO) e outras refeitas (REDO) no momento da recuperação. Com a necessidade do UNDO, alguns controles adicionais precisam ser adicionados ao arquivo de log. O SGBD precisa conhecer, além dos valores alterados, os valores antes da modificação. A ideia básica é que a atualização é feita no banco de dados no momento exato em que foi executada, então, no procedimento de recuperação após falha, tem que desfazer tudo e refazer.

As técnicas baseadas em modificação imediata consistem em realizar modificações no banco de dados antes do término das transações, ou seja, permite que as modificações no banco de dados sejam realizadas enquanto as transações ainda estão em execução. Nesse sentido, as escritas realizadas por transações ativas são denominadas de modificações não efetivadas. Nessa abordagem, o log deverá armazenar o valor antigo e o valor novo, efetuados pelas ações de modificações.

Diante de uma falha, uma transação é recuperada por meio da operação UNDO, onde os valores antigos serão recuperados para o banco de dados. Se existir, no log, informação sobre o término da transação, a operação REDO será executada para que todos os novos valores sejam refletidos no banco de dados.

Durante o processo de recuperação pode haver um cenário onde todos os dados já tenham sido escritos no banco de dados. Mas não haverá inconsistência, pois serão sobrescritos pelos mesmos valores. Essa característica está ligada à propriedade de idempotência de operações UNDO e REDO, ou seja, fazer UNDO ou REDO uma vez ou várias vezes produz o mesmo resultado.

Do ponto de vista do gerenciamento de buffer, essas técnicas utilizam a abordagem STEAL. Isso torna o gerenciamento de buffer bem mais simples,

pois na abordagem STEAL não há necessidade de manter blocos bloqueados por transações, o que minimiza o esforço do gerenciador de buffer ao alocar e desalocar espaços de memória.

Técnica UNDO/REDO

Na técnica UNDO/REDO é gravada a efetivação de uma transação "x" no log depois que todas as modificações, realizadas por "x", terem sido gravadas no log, e antes dessas modificações terem sido gravadas no banco de dados. Para isso, essa técnica requer a existência de um log UNDO e REDO.

Essa abordagem utiliza duas listas, uma lista de REDO e uma lista de UNDO, que contém identificadores de transações. A lista de REDO exibe os identificadores das transações que efetivaram, ou seja, possuem indicativo de efetivação gravado no log. Já a lista de UNDO ilustram os identificadores das transações que estavam "ativas".

O procedimento de recuperação é dado, basicamente, por duas regras. A primeira regra consiste em fazer uma varredura no log do final para o começo, executando UNDO das transações contidas na lista de UNDO, ou seja, as transações que efetivaram. Já a segunda regra faz a varredura do começo para o final do log, realizando REDO das transações inseridas na lista de REDO, ou seja, as transações "ativas".

Durante a aplicação da primeira regra é possível que, ao percorrer o log do começo para o final para realizar REDO, um dado "x" tenha sido atualizado mais de uma vez por transações efetivadas. Nesse caso, por medidas de otimização, é necessário realizar, somente, o REDO da última atualização do dado "x".

Exemplo:

Nesse exemplo, vamos expor o comportamento da técnica UNDO/REDO com checkpoint diante do processo de recuperação. Para isso, seguiremos um cenário que contém cinco transações dispostas em uma linha do tempo e o momento da falha. Maiores detalhes sobre esse cenário podem ser visualizados na Figura 5.

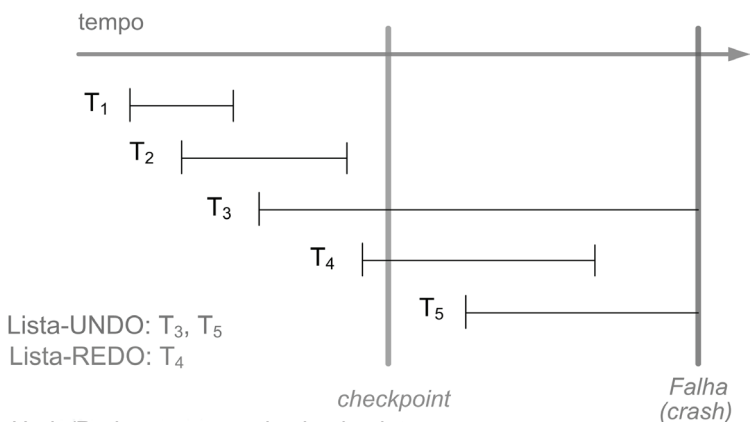


Figura 5. Técnica Undo/Redo com o uso de checkpoint.

Podemos perceber que as transações T_1 e T_2 iniciaram e terminaram antes da execução do checkpoint. A transação T_3 iniciou antes do checkpoint e não chegou a sua efetivação. Já a transação T_4 iniciou antes do checkpoint e efetivou antes da falha. A transação T_5 iniciou após o checkpoint e não consegui efetivar por completo.

Seguindo as regras impostas pela técnica em questão, as transações T_1 e T_2 foram efetivadas antes do checkpoint. Portanto, essas transações não precisam ser desfeitas e nem refeitas. Isso porque o checkpoint faz com que transações efetivadas contenham suas atualizações persistidas no banco de dados.

A transação T_4 efetivou antes do momento da falha, mas suas atualizações não foram persistidas no banco de dados. Para esse caso, vamos supor que a técnica utiliza a estratégia NOT-FORCE. Com isso é necessário, durante o processo de recuperação, que a transação T_4 sobre a operação de REDO. Já as transações T_3 e T_5 não efetivaram nem antes do checkpoint e nem antes do momento da falha. Nesse caso, essas transações devem sofrer UNDO.

Algoritmo: Procedimento para recuperar

RM_Restart ( )

1. Considere limpas as partições da cache

2. Faça refeito = { } e desfeito = { }

3. Comece com a última entrada do log e percorra o arquivo no sentido inverso (em direção ao início).

Repita os passos abaixo até que: (i) refeito U desfeito = Banco de Dados ou (ii) não existam mais entradas no log

a) Se x não estiver na cache, leia x.

b) Se Ti estiver na lista de transações efetivadas:

i) Atualize a partição de x com o valor ImDp.

ii) refeito = refeito U { x }

c) Caso contrário, se: (i) Ti estiver na lista de transações abortadas ou (ii) na lista de transações ativas e não estiver na lista de transações efetivadas:

i) Atualize a partição de x com o valor ImAn.

ii) desfeito = desfeito U { x }

4. Para cada Ti da lista de transações efetivadas:

a) Se Ti estiver na lista de transações ativas, remova Ti desta lista.

5. Informe o final do processamento de restauração ao escalonador

Técnica UNDO/NO-REDO

Na técnica UNDO/NO-REDO é gravada a efetivação de uma transação "x" no log depois que todas as modificações, realizadas por "x", terem sido gravadas no log, e depois que essas modificações terem sido gravadas no banco de dados. Nesse caso, é necessário a existência de um log UNDO.

Nessa técnica, se existir um registro de efetivação de uma transação "x" no log, então "x" está garantidamente efetivada no banco de dados. A principal vantagem dessa abordagem é que não há a necessidade de realizar REDO. Porém, pode-se fazer UNDO de uma transação que foi gravada com sucesso no banco de dados, mas sua efetivação não foi gravada a tempo no log.

O procedimento de recuperação dessa técnica é bem mais simples do que a técnica UNDO/REDO. Nessa abordagem, é necessário realizar varredura do final para o começo do log, executando UNDO das transações existentes na lista de UNDO, ou seja, desfazendo os efeitos produzidos pelas transações "ativas".

Exemplo:

Para esse exemplo vamos supor um cenário que existam cinco transações em uma linha do tempo e momento em que ocorre a falha. Nesse cenário, a técnica UNDO/NO-REDO será demonstrada sem o uso de checkpoint. Maiores detalhes sobre a disposição das transações na linha do tempo e o momento da falha podem ser encontradas na Figura 6.

Podemos observar que as transações T_1 e T_2 foram efetivadas antes do momento da falha. As transações T_3 e T_5 não conseguiram efetivar antes da falha. Já a transação T_4 efetivou bem próximo do momento da falha. A disposição de T_4 na linha do tempo, nesse cenário, permite-nos observar uma característica dessa técnica.

As transações T_1 e T_2 efetivaram antes do momento da falha e possuem suas efetivações em log. Nesse sentido, essas duas transações não necessitam sofrer REDO durante a recuperação. Já as transações T_3 e T_5 não chegaram a efetivar por completo. Com isso, devem passar por uma operação de UNDO para que suas modificações sejam desfeitas.

Por último, a transação T_4 conseguiu terminar antes do momento da falha. Porém, esse momento de efetivação foi bem próximo do momento da falha e, infelizmente, seu registro de efetivação não se encontra no log. Portanto, assim como T_3 e T_5, a transação T_4 precisa passar pela operação de UNDO durante o processo de recuperação após falha.

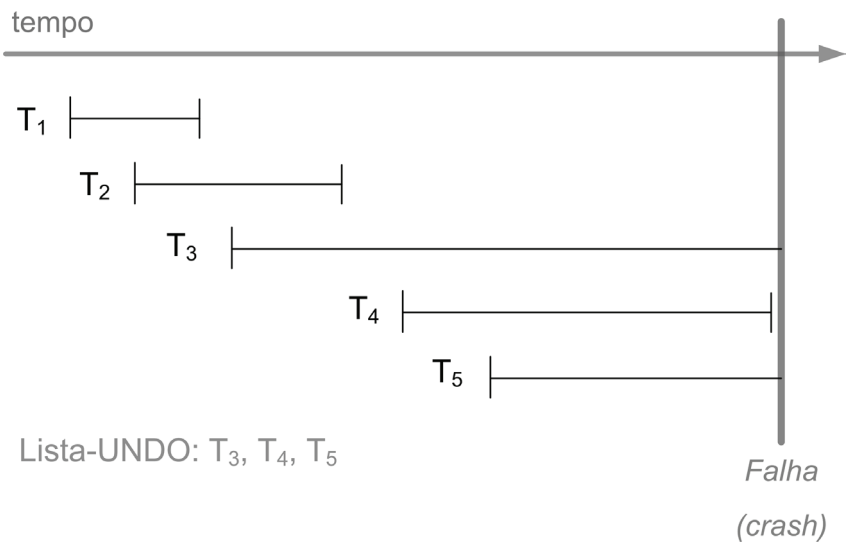


Figura 6 – Técnica Undo/Redo sem o uso de checkpoint.

Algoritmo: Procedimento para recuperar

RM_Restart ( )

1. Considere limpas as partições da cache

2. Faça desfeito = { }

3. Comece com a última entrada do log e percorra o arquivo no sentido inverso (em direção ao início).

Repita os passos abaixo até que: (i) desfeito = Banco de Dados ou (ii) não existam mais entradas no log para examinar.

Para cada [Ti , x , ImAn, ImDp], faça:

a) Se Ti não estiver na lista de transações efetivadas e x não estiver em desfeito:

i) Selecione uma partição da cache para x

ii) Atualize a partição de x com o valor ImAn.

iii) desfeito = desfeito U { x }

4. Para cada Ti da lista de transações efetivadas:

a) Se Ti estiver na lista de transações ativas, remova Ti desta lista

5. Informe o final do processamento de restauração ao escalonador

Modificação Adiada

A modificação adiada e a imediata diferem entre si pela estratégia utilizada durante o checkpoint e, por consequência, em como esses dados são restaurados no caso de falha. Nessa estratégia, o checkpoint só grava no arquivo de dados às transações que estão marcadas como efetivadas na memória principal. As operações de outras transações são somente feitas no buffer de banco de dados e registradas no arquivo de log. Assim sendo, transações não efetivadas têm suas operações ignoradas no momento da recuperação (veja isso como um rollback do sistema), enquanto as transações que foram efetivadas devem ser refeitas e persistidas no arquivo de dados.

O procedimento básico é adiar a execução de qualquer atualização sobre o banco de dados até que a transação seja concluída com sucesso e atinja o seu ponto de efetivação. Durante a execução de uma transação, as operações são apenas registradas no log e na área de trabalho da transação. Depois que a transação atinge seu ponto de efetivação e o log é escrito em disco, as atualizações são registradas no banco de dados.

As técnicas baseadas em modificação adiada no log realizam alterações no banco de dados somente quando todas as transações, que modificam o banco de dados, são escritas no log. Nesse sentido, quando uma transação é parcialmente efetivada, as informações no log associadas àquela transação são usadas para executar as escritas adiadas. Com isso, se uma transação for efetivada com sucesso, então existe um registro no log correspondente a essa transação. Quando todas as modificações do banco de dados são escritas no log, garante-se a propriedade de atomicidade de transação.

A ideia dessa técnica é que modificações adiadas, ou melhor, postergadas, adia-se a execução de todas as operações de modificação do banco de dados por uma transação até que ela tenha seja parcialmente efetivada, ou seja, tenha executado todas suas ações.

Diante de uma falha, uma transação só será recuperada por meio da operação REDO (refazer), se contiver um registro no log que sinalize o fim da mesma. Nessa abordagem não existe a operação UNDO (desfazer), pois o banco de dados só ficará em um estado inconsistente se o sistema cair durante a recuperação. Caso aconteça uma falha durante o processo de recuperação, a operação REDO deve ser executada novamente.

Do ponto de vista do gerenciamento de buffer, essas técnicas utilizam a abordagem NOT-STEAL. Isso torna o gerenciamento de buffer mais complexo, pois a abordagem NOT-STEAL faz com que os blocos atualizados por uma transação "x" não sejam substituídos enquanto "x" não efetivar. Porém, o processo de recuperação após falha é bem mais simples, pois as transações não precisam realizar UNDO.

Técnica NO-UNDO/REDO

Na técnica NO-UNDO/REDO força-se a gravação do log em disco após uma transação "x" concluir suas modificações. Com isso, se uma transação "x" falhar antes de alcançar sua efetivação, não é necessário realizar UNDO de "x", pois nenhuma atualização de "x" foi gravada no banco de dados.

Nessa técnica o processo de recuperação ocorre por meio de uma varredura do começo do log até o final, realizando REDO nas transações contidas na lista de REDO. Como a gravação do log, em disco, é feita após a efetivação das transações, o REDO acontece somente nas transações que foram efetivadas. Caso, durante a falha, haja transações "ativas", ou seja, transações que alcançaram a efetivação, essas devem ser recomeçadas.

Esse tipo de técnica limita a execução concorrente das transações porque itens de dados ficam bloqueados até a efetivação das transações. Consequentemente, uma transação não grava as modificações no BD até sua efetivação. Logo, uma transação nunca é desfeita por causa da ocorrência de falhas. Outra característica importante é que uma transação nunca vai ler o valor de um item gravado por outra não acabada, porque os itens estão bloqueados. Portanto, não ocorrerá UNDO em cascata de transações.

Exemplo:

Nesse primeiro exemplo, vamos mostrar um cenário da técnica NO-UNDO/REDO sem o uso de checkpoint. Para isso, colocamos em nosso exemplo cinco transações descritas na linha do tempo e o momento em que ocorre a falha. Maiores detalhes sobre esse cenário pode ser visto na Figura 7.

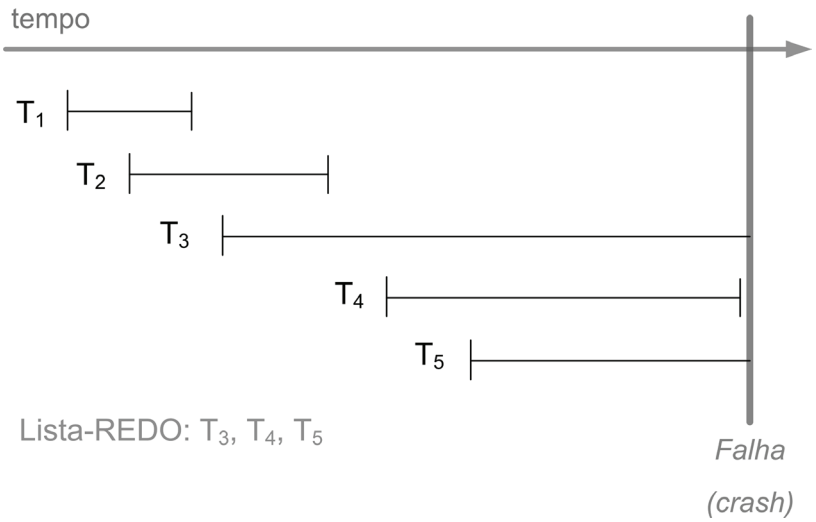


Figura 7 – Técnica No-Undo/Redo sem o uso de checkpoint.

Podemos perceber que as transações T_1, T_2 e T_4 efetivaram antes do momento da falha. Nesse sentido, essas transações estão inseridas na lista de

REDO. Para esse exemplo vamos supor que as transações T_1 e T_2 concluíram e atualizaram o banco de dados. Já a transação T_4 concluiu mas não persistiu suas modificações no banco de dados.

Seguindo as regras da técnica aqui descrita, as transações T_1 e T_2 concluíram e atualizaram o banco de dados. Portanto, essas precisam sofrer operações de REDO no processo de recuperação. Já a transação T_4 concluiu, mas não chegou a atualizar o banco de dados. Com isso, assim como T_1 e T_2, a transação T_4 deve sofrer operação de REDO durante a recuperação.

Analisando, na linha do tempo, as transações T_3 e T_5 percebe-se que essas não chegaram as suas efetivações. Portanto, essas não atualizaram, de forma persistente, o banco de dados e não sofrem UNDO. Com isso, as transações T_3 e T_5 devem ser ressubmetidas pelos usuários ou aplicações.

Exemplo:

Vamos ver um cenário em que a técnica NO-UNDO/REDO será combinada com o uso do recurso de checkpoint. Para isso, disponibilizamos em nosso exemplo cinco transações descritas na linha do tempo e o momento em que ocorre o checkpoint e a falha. A Figura 8 exibe o cenário que queremos discutir.

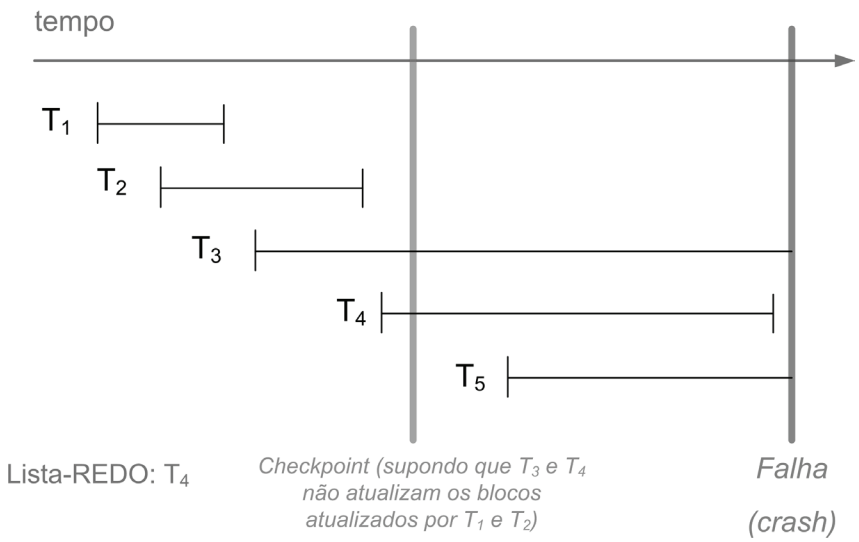


Figura 8 – Técnica No-Undo/Redo com o uso de checkpoint.

Nesse exemplo é mostrado que as transações T_1 e T_2 efetivaram antes da aplicação do checkpoint. Já a transação T_4 concluiu após o checkpoint e antes da falha. Enquanto as transações T_3 e T_5 não concluíram durante a linha do tempo mostrada nesse cenário.

Como visto, as transações T_1 e T_2 efetivaram antes do checkpoint. Com isso, essas duas transações não sofrem operação de REDO porque o checkpoint garantiu seus efeitos no banco de dados. A transação T_4 iniciou antes

do checkpoint, mas terminou antes da falha. Nesse caso, a transação T_4 deverá sofrer REDO, assim como na técnica sem o uso do checkpoint. Com relação às transações T_3 e T_4, assim como na técnica sem o uso de checkpoint, devem ser ressubmetidas pelos usuários ou aplicações.

Algoritmo: Procedimento para recuperar

RM_Restart ( )

1. Considere limpas as partições da cache

2. Faça desfeito = { }

3. Comece com a última entrada do log e percorra o arquivo no sentido inverso (em direção ao início).

Repita os passos abaixo até que: (i) desfeito = Banco de Dados ou (ii) não existam mais entradas no log para examinar.

Para cada [Ti , x , ImAn, ImDp], faça:

a) Se Ti não estiver na lista de transações efetivadas e x não estiver em desfeito:

i) Selecione uma partição da cache para x

ii) Atualize a partição de x com o valor ImAn.

iii) desfeito = desfeito U { x }

4. Para cada Ti da lista de transações efetivadas:

a) Se Ti estiver na lista de transações ativas, remova Ti desta lista

5. Informe o final do processamento de restauração ao escalonador.

1.7 Técnicas de Recuperação de Meio de Armazenamento

Essa técnica é utilizada quando ocorre falha do meio de armazenamento. Nesse caso, necessita-se de uma cópia do BD atualizada periodicamente. Os seguintes passos são necessários para a operação de cópia periódica do BD:

1. Conclusão de todas as transações ativas: nenhuma transação pode estar ativa e se existem transações nesse estado, deve-se aguardar até elas encerrarem com sucesso.
2. Descarga do buffer do log para o disco;
3. Descarga do buffer do BD para o BD físico.

No caso de falha, os passos para o procedimento de recuperação são:

1. Restaurar o BD a partir da última cópia;
2. Percorrer o arquivo de LOG, realizando o REDO de todas as transações

concluídas desde a última cópia.

As transações ativas no momento da falha podem ser ressubmetidas à execução pelo SGBD, já que não houve perda de dados na memória principal. Quando ocorre a operação de backup, o log corrente é “descartado” (excluído ou mantido associado ao backup anterior do BD) e um novo log, nesse caso “zerado”, é iniciado.

A Figura 9 mostra um exemplo da técnica de recuperação de meio armazenamento. Nessa Figura, as transações T_1 , T_2 e T_3 são iniciada e então, quando nenhuma destas transações está ativa, ocorre a operação de archive ou backup, ou seja, armazenamento de todas as operações referentes as transações. Em seguida, as transações T_4, T_5 e T_6 são iniciadas e a ocorre uma falha do BD quando a transação ainda está ativa. Nesse caso, é realizada a operação de recuperação do banco de dados. Como as transações T_1, T_2 e T_3 já foram armazenadas, não é necessário desfazê-las ou refazê-las. Já as transações T_4 e T_6, que não estavam no backup, são refeitas. Para a transação T_5, essa precisa ser desfeita, para manter a consistência do banco de dados.

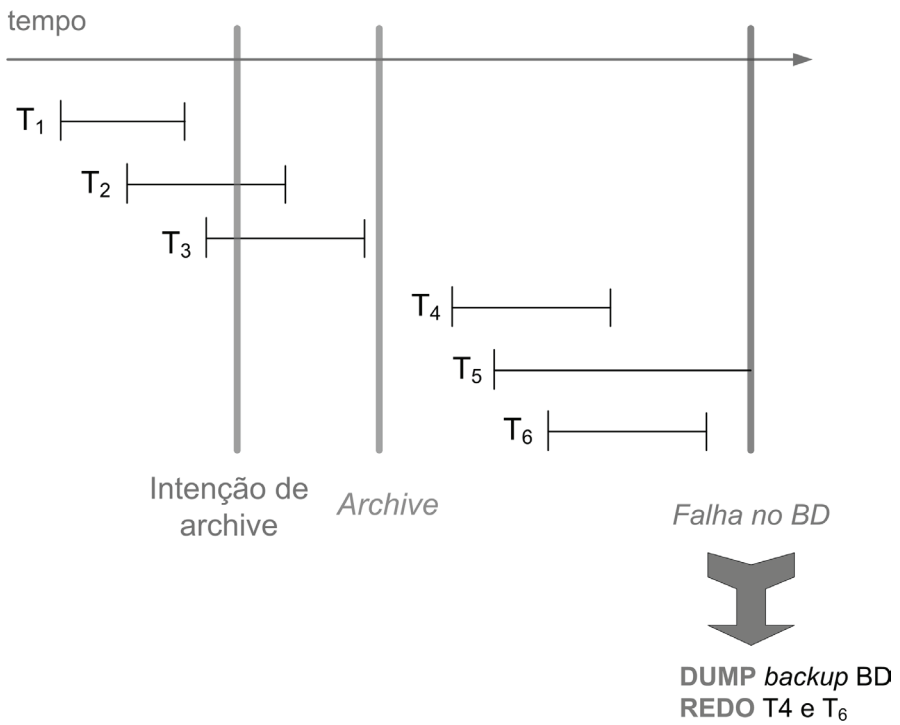


Figura 9. T Exemplo da técnica de recuperação de meio de armazenamento.

Vale a pena ressaltar as técnicas baseadas em log requerem um log seguro, isto é, redundância de log, pois o meio de armazenamento onde esse é salvo pode sofrer falhas. Normalmente os SGBD’s provem utilitários que realizam

essas tarefas.

1.8 Recuperação baseada em Páginas Sombras

Na técnica de recuperação baseada em páginas sombras, o BD é constituído de páginas. Existe uma Tabela de Páginas Correntes (TPC) e uma Tabela de Páginas Sombras (TPS), ambas em memória principal.

Princípio de Funcionamento

1. Quando uma transação T_i inicia a execução, a TPC é copiada para TPS, sendo está salva em disco;
2. Durante a execução, a TPS não muda;
3. Quando uma operação de escrita ocorre em uma página, uma nova cópia da página modificada é criada e é apontada pela TPC;
4. Para a recuperação de uma falha, ocorre a liberação das páginas modificadas e o descarte das TPC. Assim, a TPS disponibilizará o estado anterior à falha.
5. Com a confirmação da transação T_i, então a TPS é descartada.

A Figura 10 mostra um exemplo do uso de páginas sombras. No início, tanto a TPC como a TPS apontam para as mesmas páginas. Com a alteração dos valores dos itens contidos nas páginas, a TPC aponta para novas páginas. Em caso de falha, as TPS passam a ser definidas com TPC, garantindo a consistência das operações, já que foi possível retornar ao estado anterior.

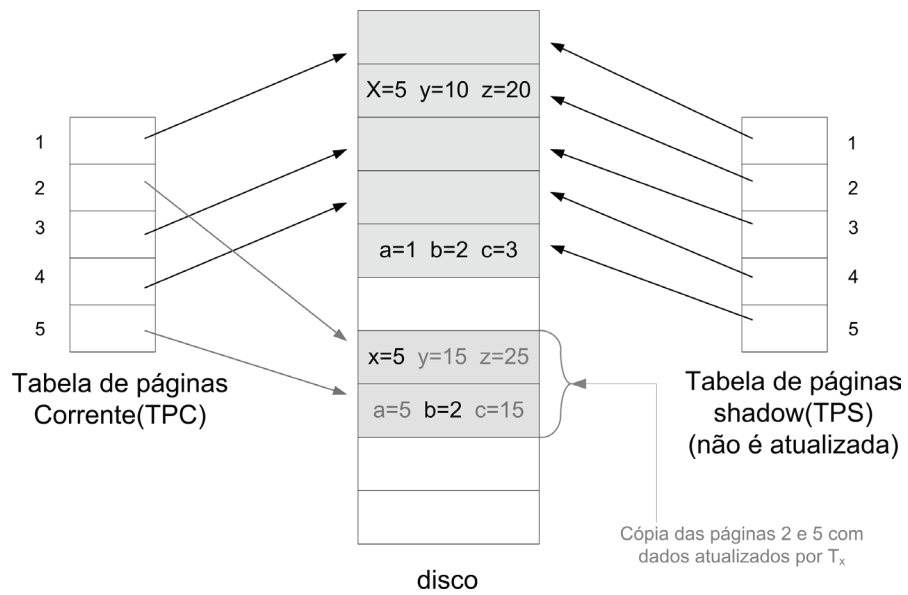


Figura 10 – Técnica de recuperação de baseada em páginas sombras.

As vantagens da técnica de recuperação de páginas sombras é que essa não necessita realizar as operações de UNDO nem REDO. Contudo, essa técnica apresenta muitas desvantagens, a saber: (a) gerenciamento complexo dos dados, já que ocorrem mudanças constantes nas páginas no disco; (b) requer coleta de "lixo", pois quando uma transação encerra, existem páginas obsoletas; (c) não suporta SGBD multiusuários, visto que apenas uma transação é executada por vez. Por esses motivos, essa técnica não é utilizada em SGBD atuais.

Exercícios

1. Quais são os objetivos do sistema de recuperação de um SGBD?

2. Quais são as duas ações básicas realizadas quando ocorre uma falha em um banco de dados?

3. Qual o função do *buffer* de banco de dados?

4. Discuta os diferentes tipos de falhas de transação.

5. Por que é necessário usar um arquivo de log?

6. Quais são os tipos de entradas em um *log* do sistema?

7. O que são *checkpoints* e por que eles são importantes? Apresente um exemplo da sua utilização.

8. Que tipos de técnicas de recuperação de falhas não requerem o *rollback*?

9. Em quais técnicas de recuperação de falhas são necessárias operações de UNDO e REDO? Explique.

10. Qual a principal vantagem das técnicas de atualização adiada? Por que são chamadas de método NO-UNDO/REDO?

11. Quais as vantagens e as desvantagens das técnicas de atualização imediata?

12. Assinale V (verdadeiro) ou F (falso) nas afirmativas abaixo:

    ( ) As técnicas de recuperação de falhas garantem a atomicidade, durabilidade e integridade das transações.
    ( ) Os tipos de falhas são: de transações, de sistema e de meio de

armazenamento.

( ) A técnica UNDO/REDO é usada na recuperação do meio de armazenamento.

( ) O *log* é uma arquivo aleatório utilizada na recuperação do BD que armazena operações de escrita.

( ) Uma das vantagens da utilização do *checkpoint* é que as transações *committed* antes do *checkpoint* não precisam sofrer UNDO em caso de falha.

13. Considere as seguintes sentenças e assinale a resposta abaixo:

i) A técnica ARCHIVE é ocorre durante o funcionamento normal do SGBD.
ii) As operações DUMP/REDO realizam uma varredura *forward* do *Log*, realizando UNDO das transações *committed*.
iii) A técnica *Shadow Pages* não é adequada a SGBD monousuário.

a) Apenas **iii** está correta.
b) **ii** e **iii** estão incorretas.
c) **i** e **iii** estão incorretas.
e) Todas estão erradas.
d) **i** e **ii** estão corretas.

14. Descreve as técnicas de recuperação de meio de armazenamento.

15. Explique o funcionamento da técnica de páginas sombras.

16. (QUESTÃO DESAFIO)

Na exemplo da Figura 4.8, suponha que a operação de *checkpoint* correu antes do final da transação T_2. Assim sendo, quais operações devem sofrer REDO? Justifique.

Webliografia

Página da Universidade Aberta do Piauí – UAPI
http://www.ufpi.br/uapi

Página da Universidade Aberta do Brasil – UAB
http://www.uab.gov.br

Página da Secretaria de Educação a Distância do MEC – SEED
http://www.seed.mec.gov.br

Página da Associação Brasileira de Educação a Distância – ABED
http://www.abed.org.br

Curso de Banco de Dados – UFSC
Professor Ronaldo Mello
http://www.inf.ufsc.br/~ronaldo/

Curso de Banco de Dados – UFSC
Professor Frank Siqueira
http://www.inf.ufsc.br/~frank/

Curso de Banco de Dados – PUC-RIO
Professor Casa Nova
http://www.inf.puc-rio.br/~casanova/

Curso de Banco de Dados – PUC-RIO
Professora Karin Breitman
http://www-di.inf.puc-rio.br/~karin/

Referências

SILBERSCHATZ, A.; KORTH, H.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed., Editora Campus, 2006.

GARCIA-MOLINA, H.; ULLMAN, Jeffrey D.; WIDOM, Jennifer. **Implementação de Sistemas de Bancos de Dados**. 1. ed. Lugar: Editora Campus, 2001.

O'NEIL, Patrick; O'NEIL, Elizabeth. **Database**: Principles, Programming and Performance. Second Edition, IE-ELSEVIER , 2001.

ELSMARI, R.; NAVATHE, Shamkant B. **Sistemas de Banco de Dados**. 4. ed., Addison-Wesley, 2005.

RAMAKRISHNAN, R. **Database Management Systems**, 3 ed., McGraw-Hill, 2002.

BERNSTEIN, P. A., V.; HADZILACOS; GOODMAN, N. **Concurrency Control and Recovery in Dabase Systems**. Addison-Wesley, 1987.

BRAYNER, A.. **Transaction Management in Multidatabase Systems**. Shaker Verlag, 1999.

CASANOVA, M. A. The Concurrency Problem of Database Systems. In: **Lectures Notes in Computer Science**, 116, 1981.

ESWARAN, K. P.; GRAY, J. R.; LORIE, A.; TRAIGER, I. L. **The Notions of Consistency and Predicate Locks in a Database System**. Communications of the ACM, 9 (11), 1976.

GRAY, J. N. The Transaction Concept: Virtues and Limitations. **Proceedings of**

the 7th International Conference on VLDB, pages 144-154, 1981.

GRAY, J. N.; LORIE, R. A.; PUTZOLU, G. R.; TRAIGER, I. L.. Granularity of Locks and Degrees of Consistency in a Shared Database. In: STONEBRAKER, M. editor, **Readings in Database Systems**, pages 94-121. Morgan-Kaufmann, 1988.

V. Hadzilacos. A Theory of Reliability in Database Systems. **Journal of the ACM**, 25:121-145, 1988.

HÄRDER, T. **Observations on Optimistic Concurrency Control Schemes**. Informations Systems, 9(2):111-120, 1984.

PAPADIMITRIOU, C. H. **The Theory of database Concurrency Control**. Computer Science Press, 1986.

Minicurrículo

Autores



Flávio Rubens de Carvalho Sousa

Doutor em Ciência da Computação pela Universidade Federal do Ceará (UFC), Fortaleza-CE Brasil (2013). Mestre em Ciência da Computação pela UFC e Graduado em Ciência da Computação pela Universidade Federal do Piauí (UFPI), Teresina-PI Brasil (2004). Atualmente é Professor Assistente da UFC em Quixadá. Atua principalmente nos seguintes temas: Gerenciamento de Dados Distribuídos, Big Data e Cloud Computing.



José Maria da Silva Monteiro Filho

Possui graduação em Bacharelado em Ciência da Computação pela Universidade Federal do Ceará – UFC (1998), mestrado em Ciência da Computação pela Universidade Federal do Ceará – UFC (2001) e doutorado em Informática pela Pontifícia Universidade Católica do Rio de Janeiro – PUC-Rio (2008). Atualmente é Professor Adjunto do Departamento de Computação da Universidade Federal do Ceará. Tem experiência na área de Ciência da Computação, com ênfase em Banco de Dados, atuando principalmente nos seguintes temas: bancos de dados autogerenciados e autônomos, bancos de dados móveis e bancos de dados em nuvem.





www.uapi.ufpi.br